Rollno: CO2105 , Name: Sandhya Raghunath Magadum

1. Adverstising Dataset

Import Libraries

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [ ]:
```
```

1] Load Dataset

In [2]:
```
df = pd.read_csv("Advertising.csv")
df.head()
```

Out[2]:

	TV	radio	newspaper	sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	9.3
3	151.5	41.3	58.5	18.5
4	180.8	10.8	58.4	12.9

In [3]:
```
df.shape
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype  
---  ------     --------------  -----  
 0   TV         200 non-null    float64
 1   radio      200 non-null    float64
 2   newspaper  200 non-null    float64
 3   sales      200 non-null    float64
dtypes: float64(4)
memory usage: 6.4 KB
```

In [ ]:
```
```

2] Data Cleaning

- Handle missing value
- Remove duplicate records
- Correct data types (if required)

2.1 Check Missing Values

In [4]:
```
df.isnull().sum()
```

Out[4]:
```
TV           0
radio        0
newspaper    0
sales        0
dtype: int64
```

In [ ]:
```

```

2.2] Remove duplicate records

In [5]:
```
df.duplicated().sum()
```

Out[5]:
```
0
```

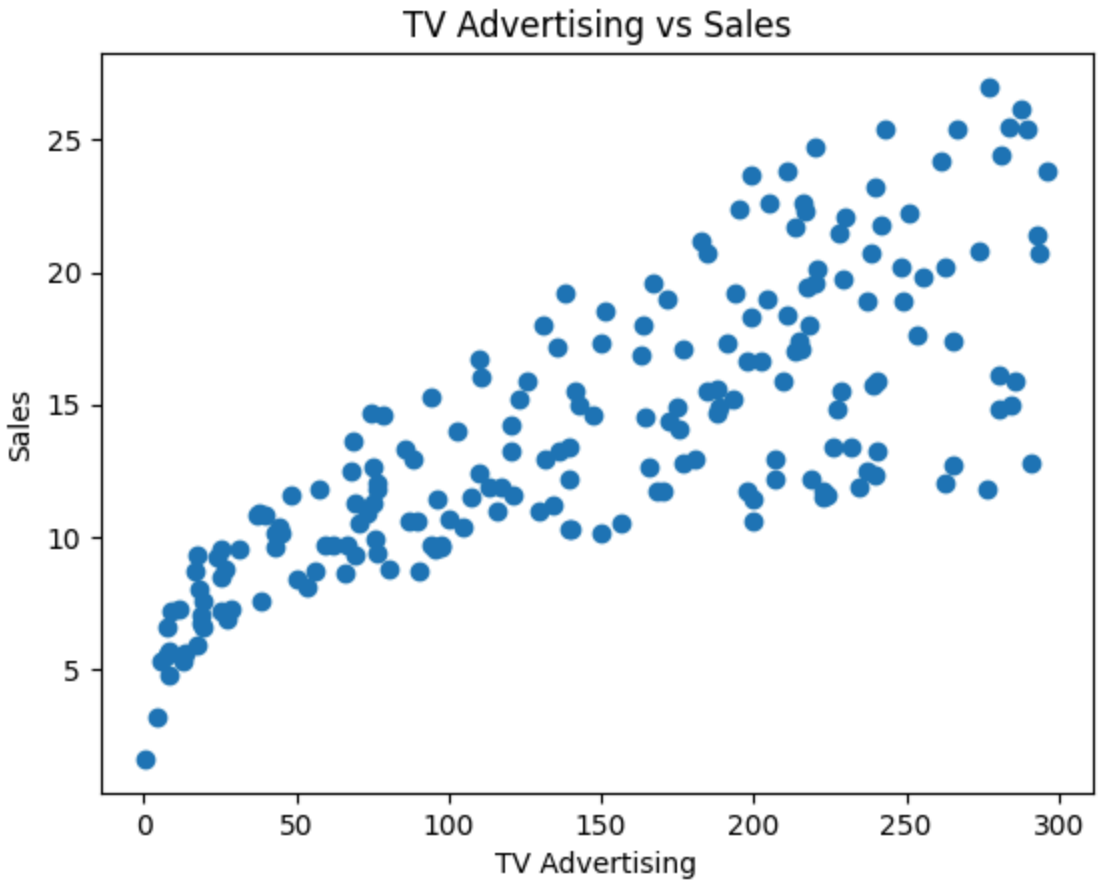
In [ ]:
```

```

3] Exploratory Data Analysis (EDA)

Create 10 Diff Visualizations:

1} TV Advertising VS Sales

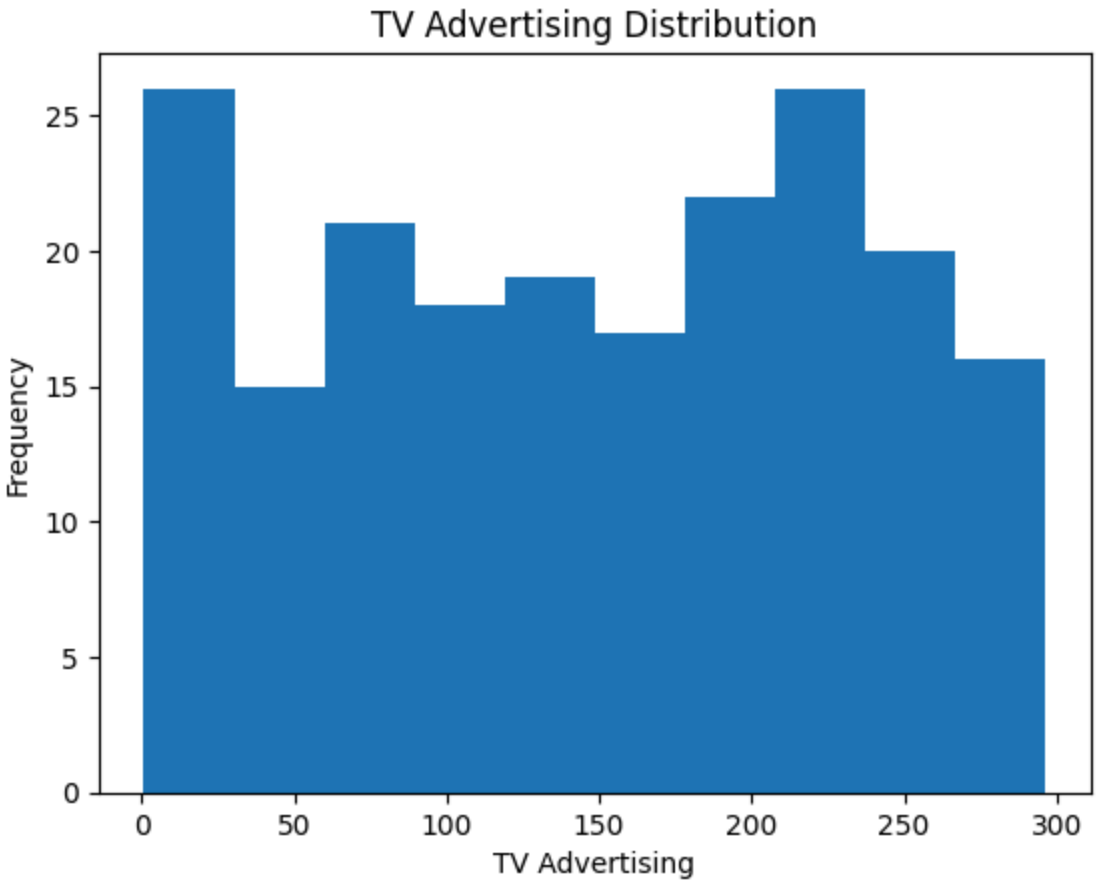
In [6]:
```
plt.scatter(df['TV'], df['sales'])
plt.xlabel('TV Advertising')
plt.ylabel('Sales')
plt.title('TV Advertising vs Sales')
plt.show()
```



In [ ]:

2} TV Distribution

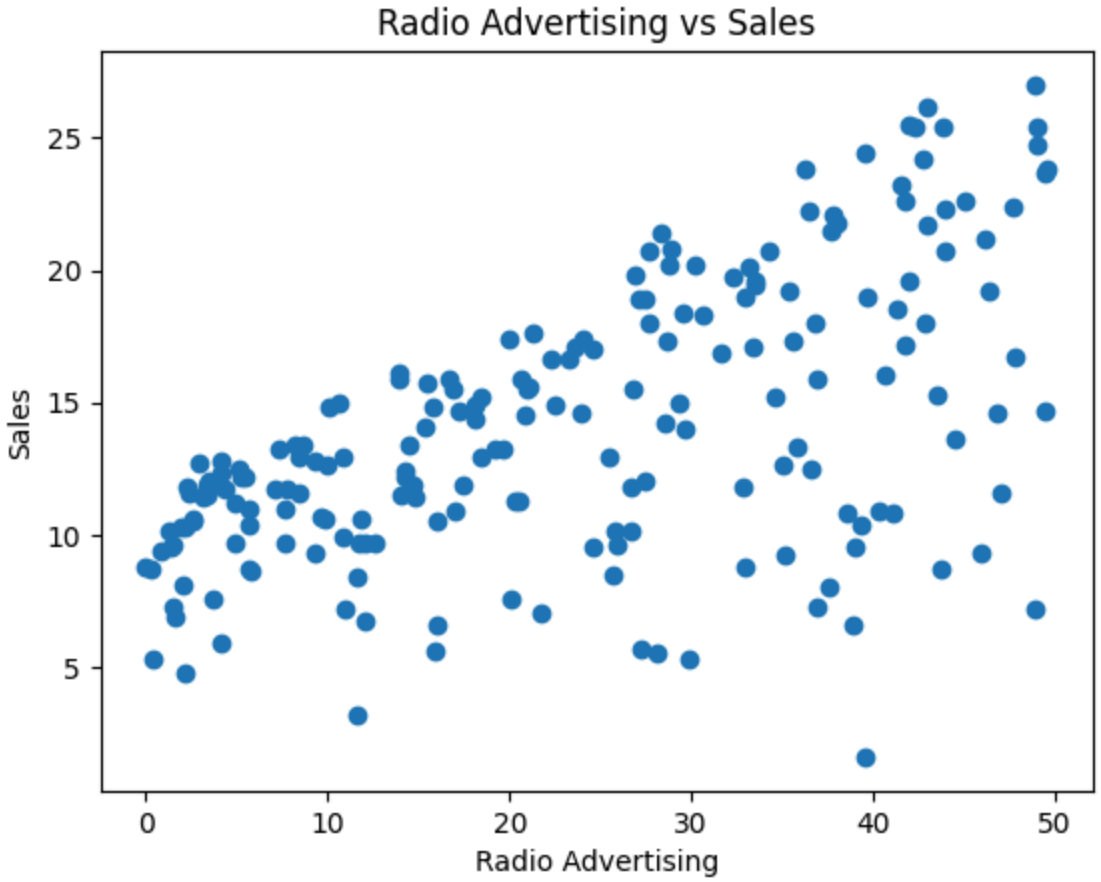
In [7]:
```
plt.hist(df['TV'])
plt.xlabel('TV Advertising')
plt.ylabel('Frequency')
plt.title('TV Advertising Distribution')
plt.show()
```



In [ ]:

3} Radio Advertising VS Sale

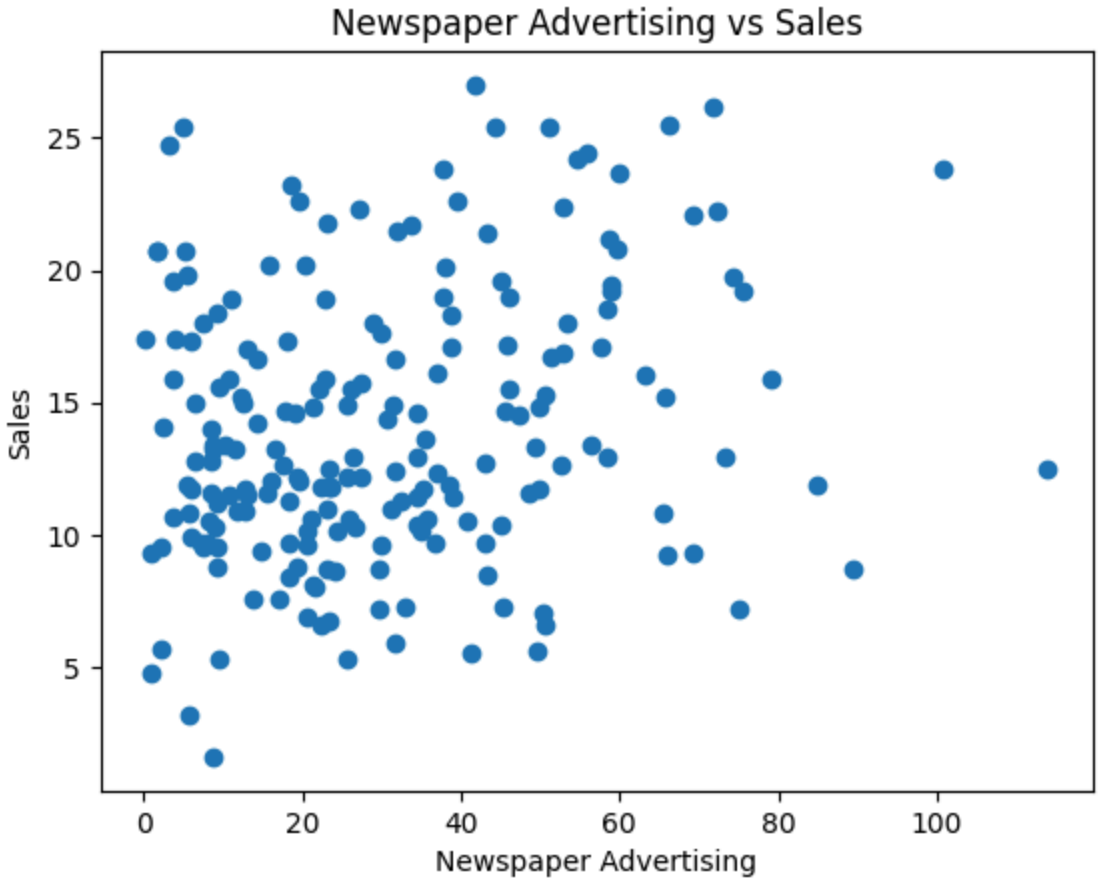
In [8]:
```
plt.scatter(df['radio'], df['sales'])
plt.xlabel('Radio Advertising')
plt.ylabel('Sales')
plt.title('Radio Advertising vs Sales')
plt.show()
```



In [ ]:

4} Newspaper Advertising vs Sales

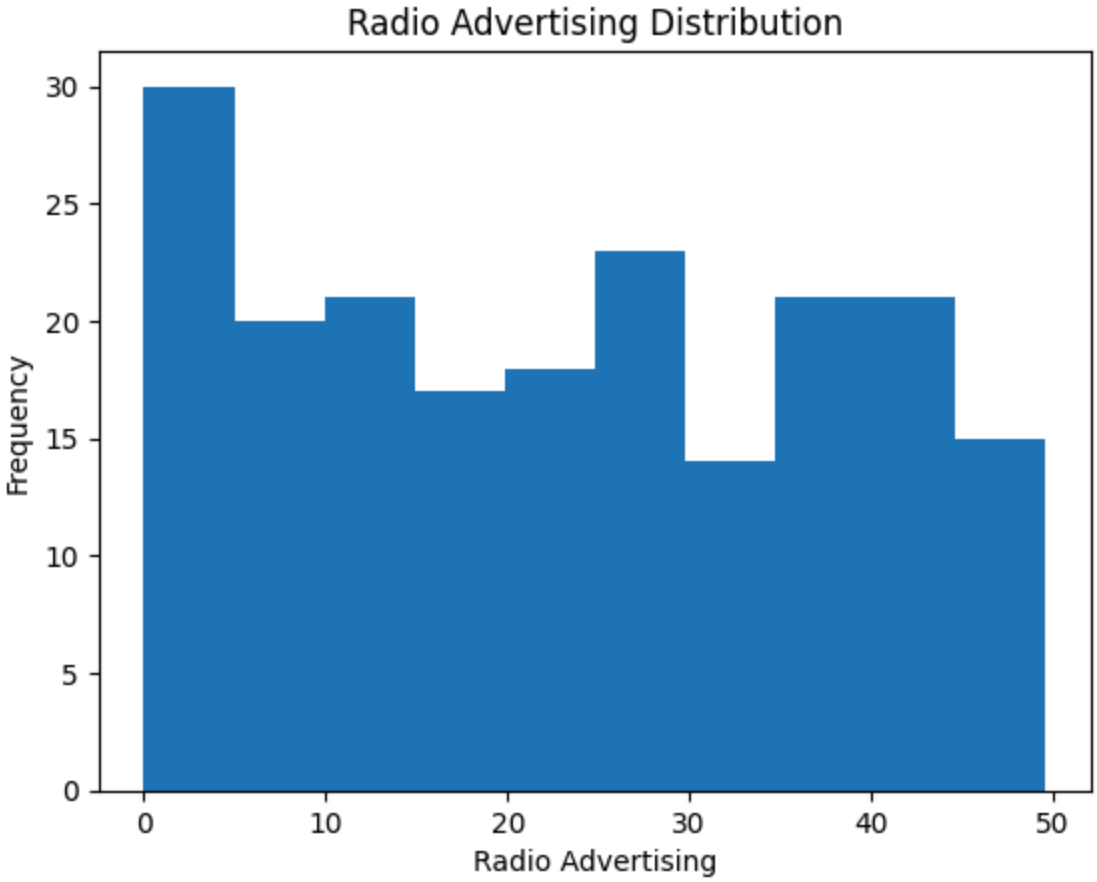
In [9]:
```
plt.scatter(df['newspaper'], df['sales'])
plt.xlabel('Newspaper Advertising')
plt.ylabel('Sales')
plt.title('Newspaper Advertising vs Sales')
plt.show()
```



In [ ]:

5} Radio Distribution

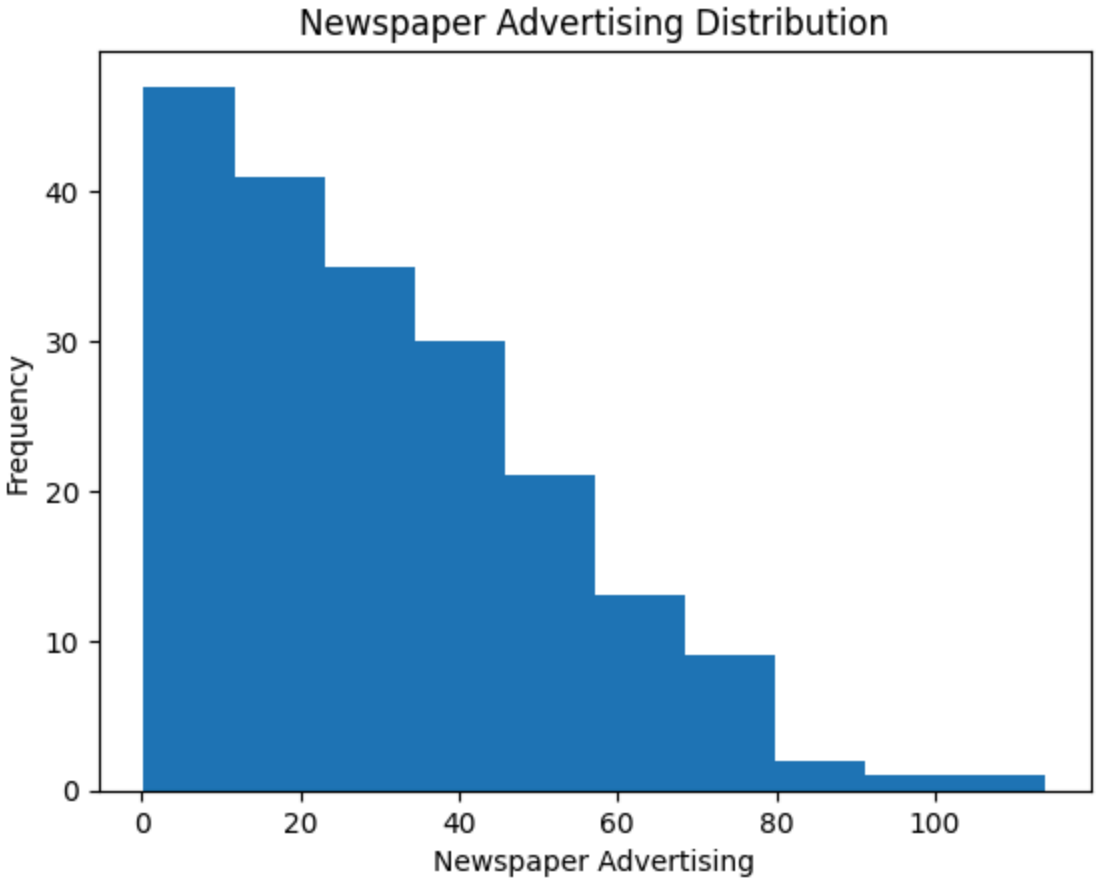
In [10]:
```
plt.hist(df['radio'])
plt.xlabel('Radio Advertising')
plt.ylabel('Frequency')
plt.title('Radio Advertising Distribution')
plt.show()
```



In [ ]:

6} Newspaper Distribution

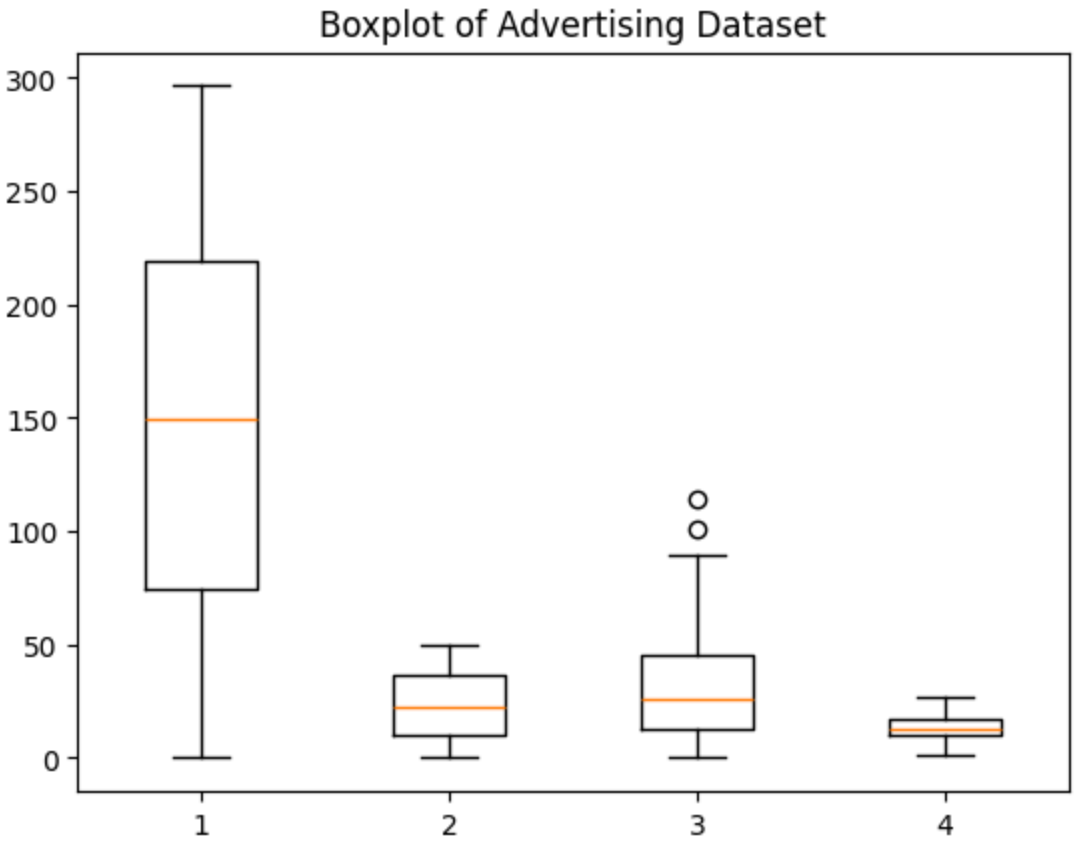
In [11]:
```
plt.hist(df['newspaper'])
plt.xlabel('Newspaper Advertising')
plt.ylabel('Frequency')
plt.title('Newspaper Advertising Distribution')
plt.show()
```



In [ ]:

7} Boxplot

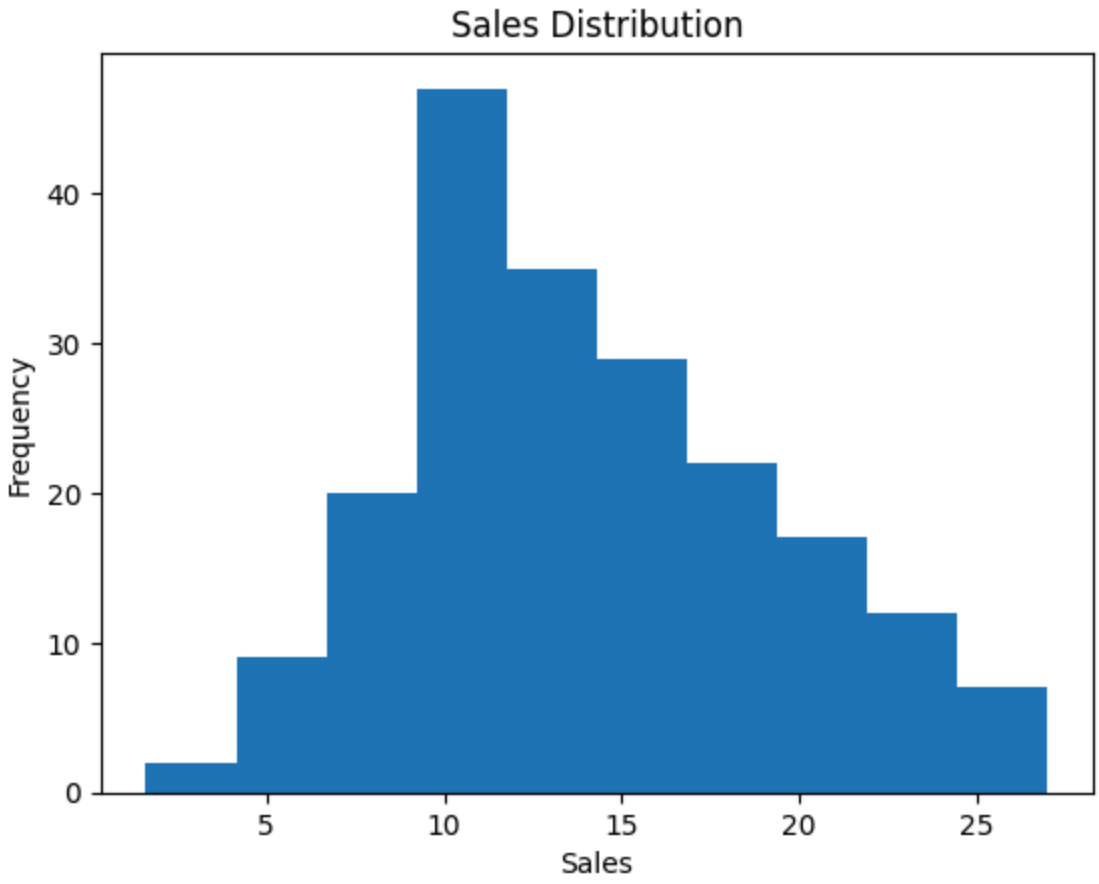
In [12]:
```
plt.boxplot(df[['TV', 'radio', 'newspaper', 'sales']])
plt.title('Boxplot of Advertising Dataset')
plt.show()
```



In [ ]:

8} Sales Distribution

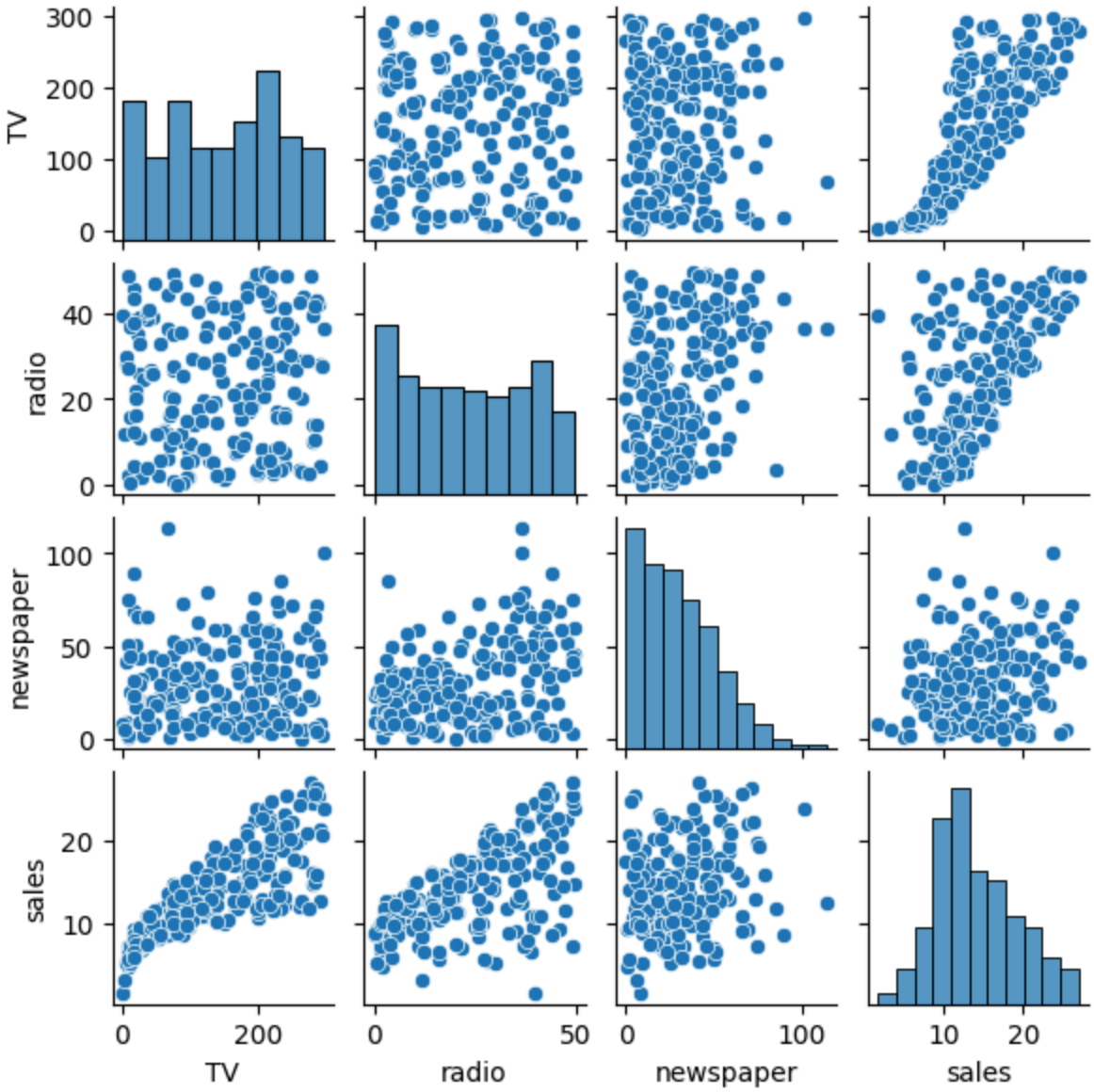
In [13]:
```
plt.hist(df['sales'])
plt.xlabel('Sales')
plt.ylabel('Frequency')
plt.title('Sales Distribution')
plt.show()
```



In [ ]:

9} Pairplot

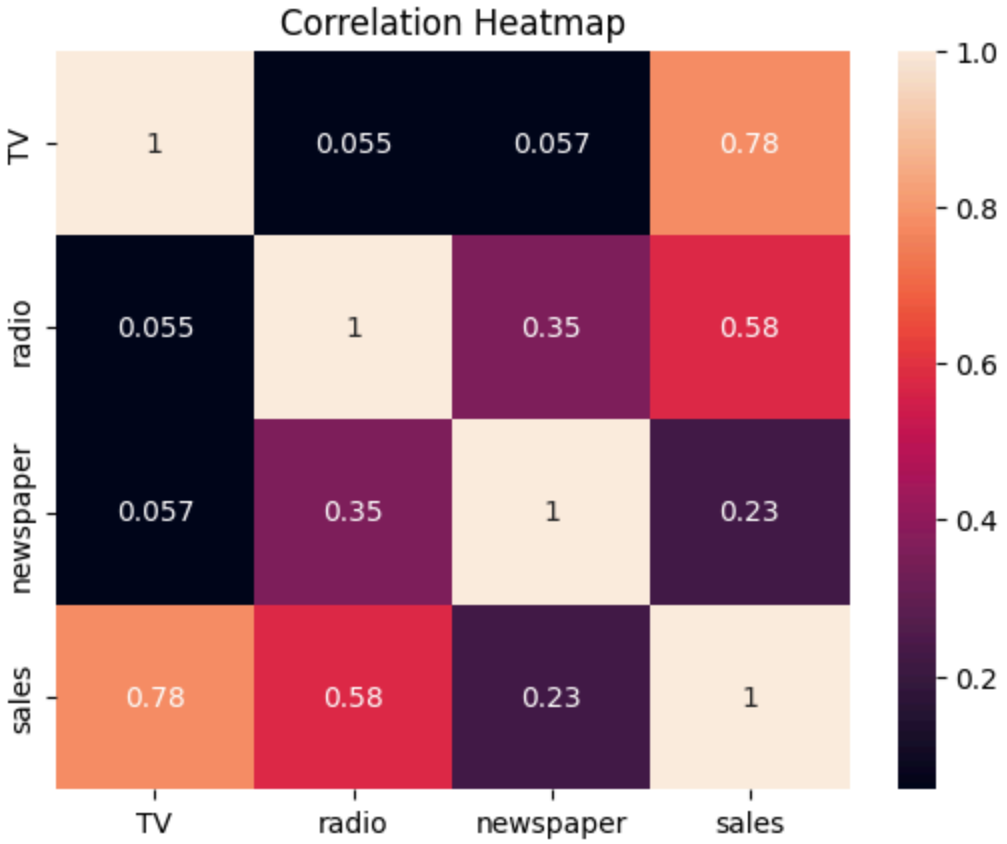
In [14]:
```
sns.pairplot(df, height=1.5)
plt.show()
```



In [ ]:

10} Correlation Heatmap

In [15]:
```
sns.heatmap(df.corr(), annot=True)
plt.title('Correlation Heatmap')
plt.show()
```



In [ ]:

4] Feature Engineering

X and Y separation :

X = Independent Features (TV, radio, newspaper)

y = Target Feature (sales

In [16]:
```
X = df[['TV', 'radio', 'newspaper']]
y = df['sales']
```

• Encoding categorical features. (Advertising dataset madhe categorical features nahit,mhanun encoding chi garaj nahi)

• Feature Scaling

Feature Scaling

In [17]:
```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

In [ ]:

5] Split the dataset into Training and Testing sets

In [18]:
```
# Import train_test_split
from sklearn.model_selection import train_test_split

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y,
    test_size=0.2,
    random_state=42
)
```

Check Training and Testing Data

In [19]:
```
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (160, 3)
X_test shape: (40, 3)
y_train shape: (160,)
y_test shape: (40,)
```

In [ ]:

In [ ]:

2. Home Prices Dataset

1] Load Dataset

In [20]:
```
df = pd.read_csv("homeprices.csv")
df.head()
```

Out[20]:

	town	area	price
0	Chennai	2600	5500000
1	Chennai	3000	5650000
2	Chennai	3200	6100000
3	Chennai	3600	6800000
4	Bangalore	2600	5850000

In [21]:
```
df.shape
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12 entries, 0 to 11
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype 
---  ------  --------------  ----- 
 0   town    12 non-null     object
 1   area    12 non-null     int64 
 2   price   12 non-null     int64 
dtypes: int64(2), object(1)
memory usage: 420.0+ bytes
```

In [ ]:
```

```

2] Data Cleaning

2.1] Check Missing Values

In [22]:
```
df.isnull().sum()
```

Out[22]:
```
town     0
area     0
price    0
dtype: int64
```

In [ ]:
```

```

2.2] Remove Duplicate Records

In [23]:
```
# Check duplicates
df.duplicated().sum()
```

Out[23]:
```
0
```

In [ ]:
```

```

2.3] Correct Data Types

In [24]:
```
df.dtypes
```

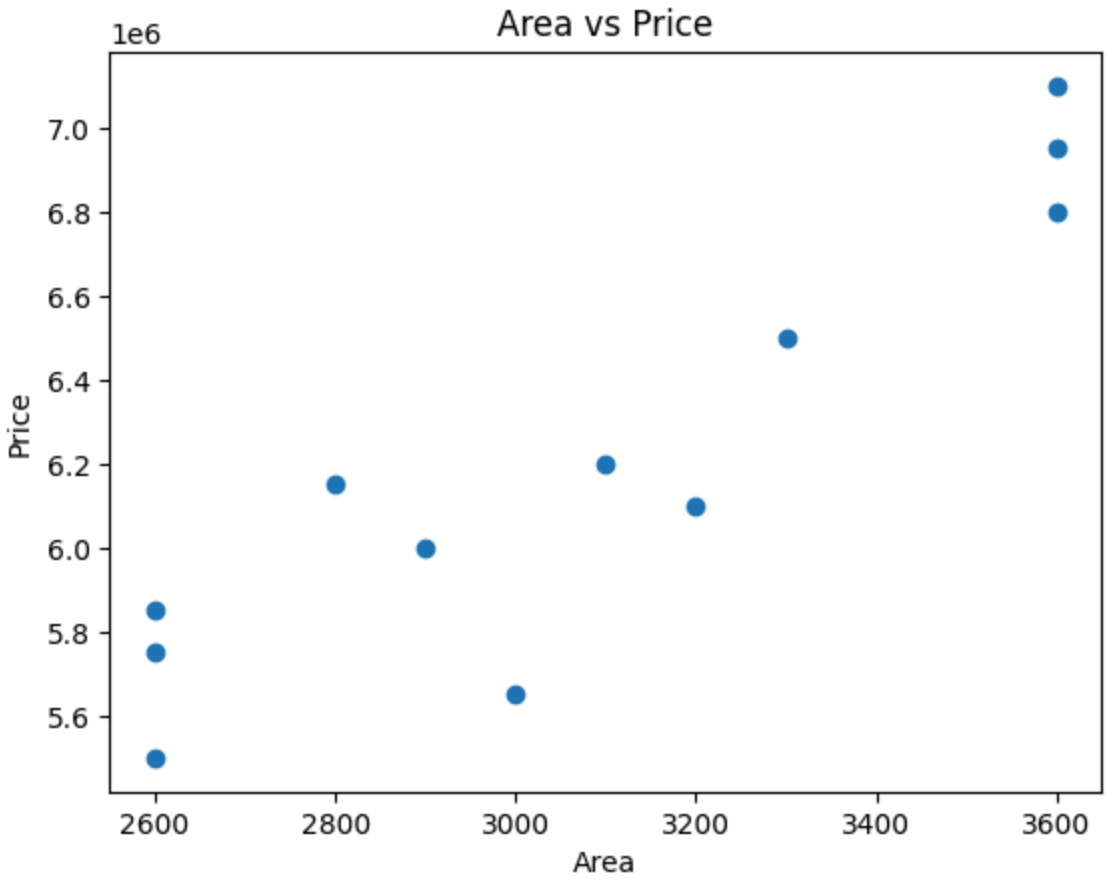
Out[24]:
```
town     object
area      int64
price     int64
dtype: object
```

In [ ]:
```

```

3] EDA

1} Area vs price

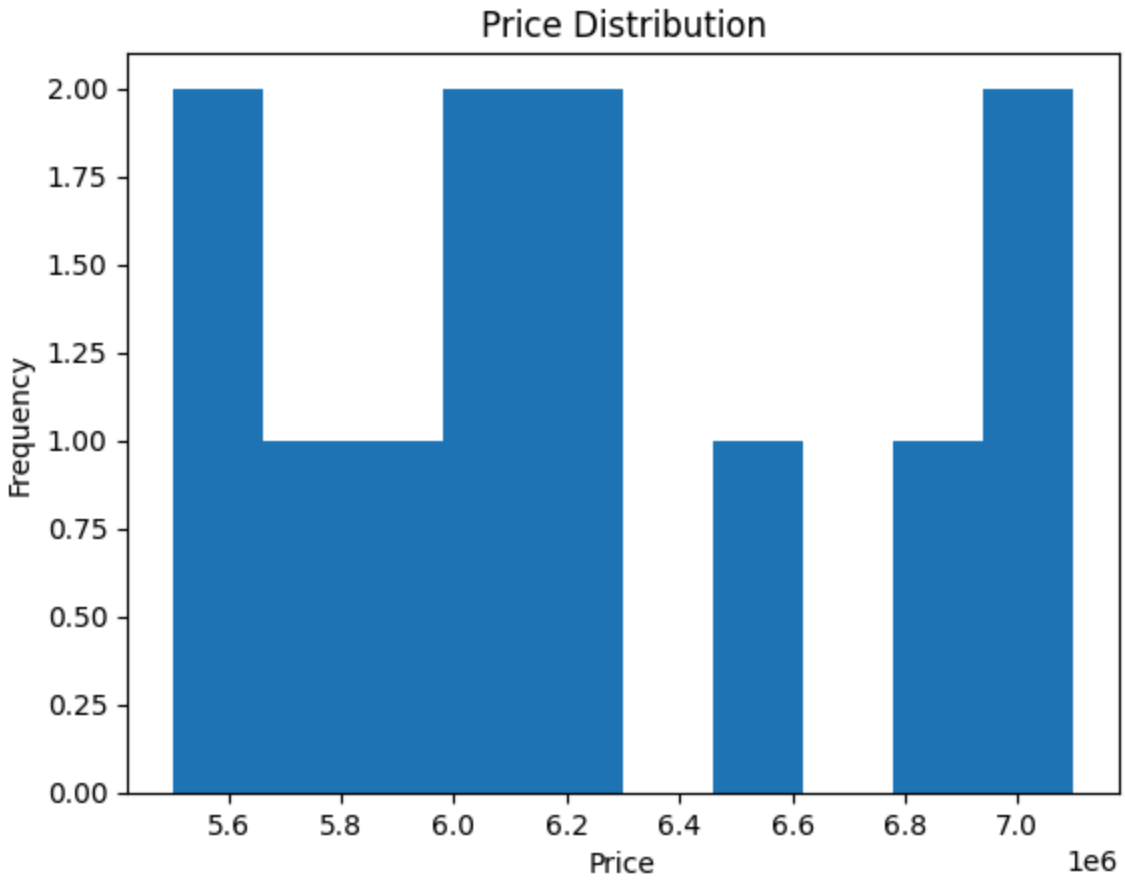
In [25]:
```
plt.scatter(df['area'], df['price'])
plt.xlabel('Area')
plt.ylabel('Price')
plt.title('Area vs Price')
plt.show()
```



In [ ]:
```

```

2} Price distribution

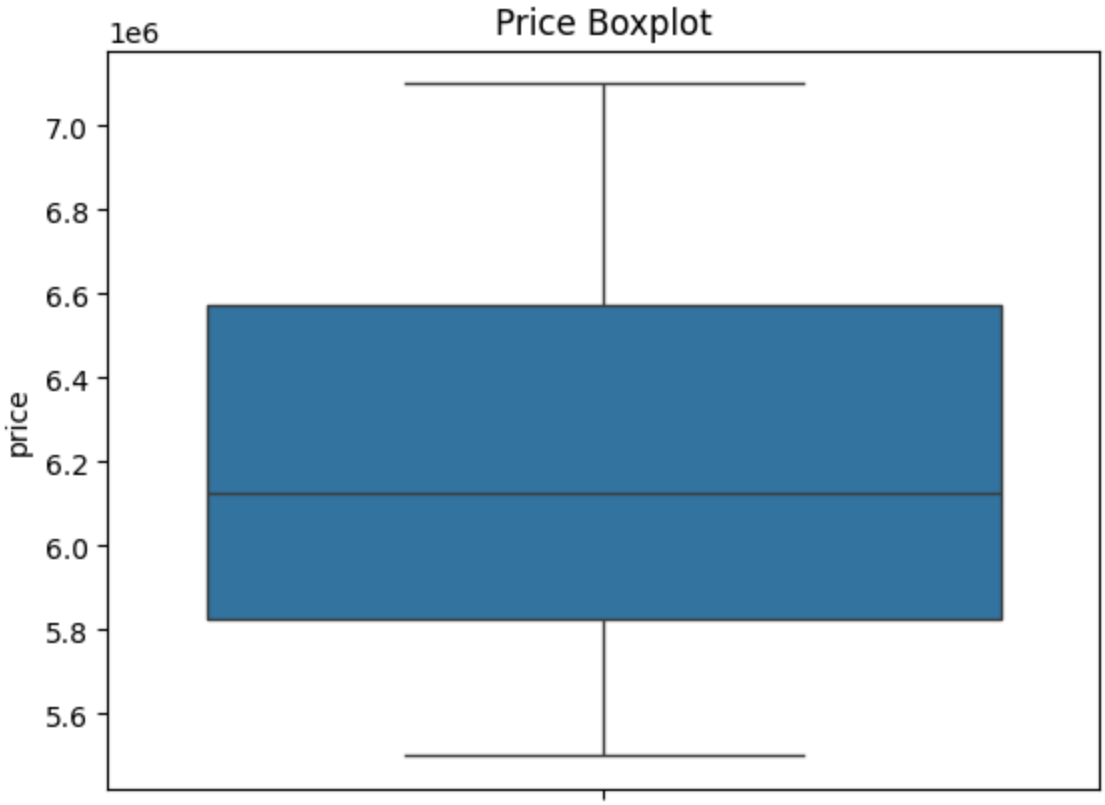
In [26]:
```
plt.hist(df['price'])
plt.xlabel('Price')
plt.ylabel('Frequency')
plt.title('Price Distribution')
plt.show()
```



In [ ]:

3} Boxplot

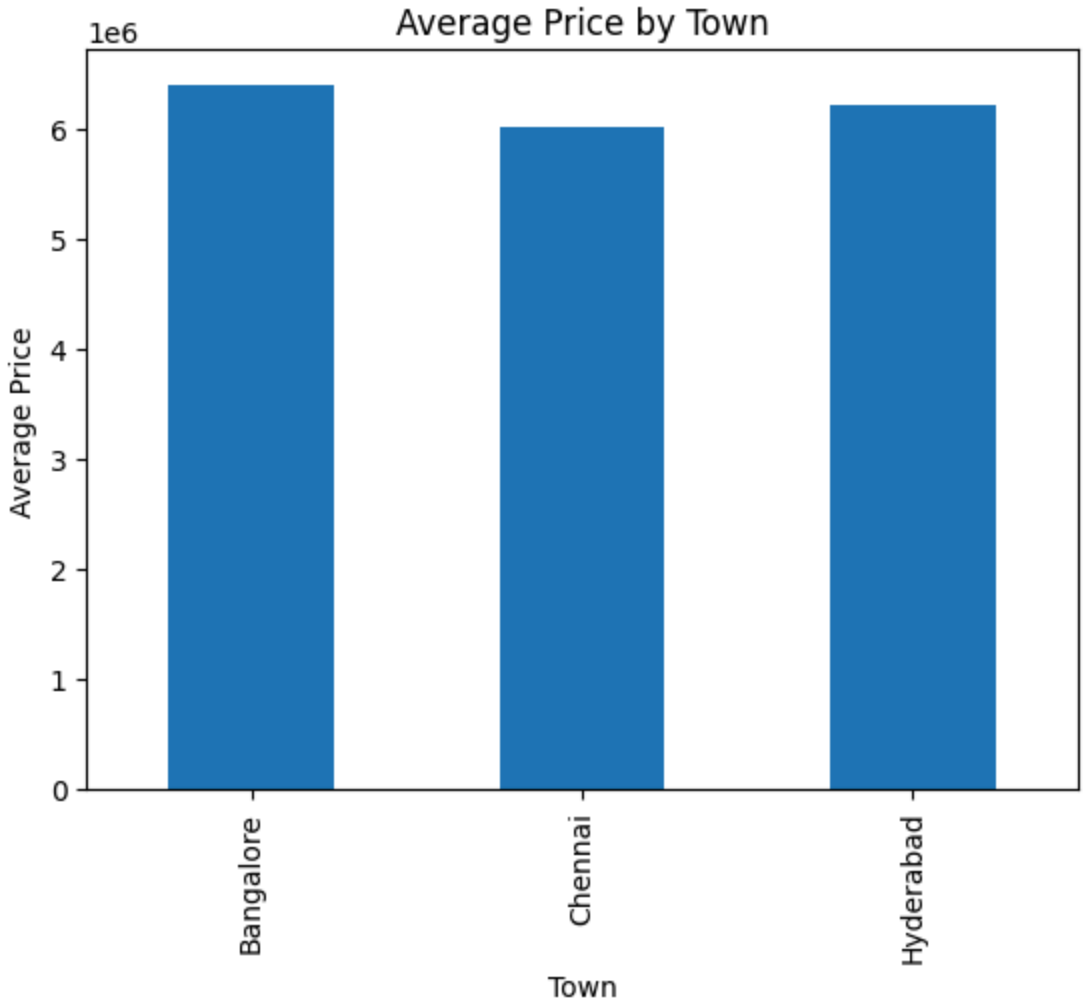
In [27]:
```
sns.boxplot(y=df['price'])
plt.title('Price Boxplot')
plt.show()
```



In [ ]:

4} Bar Chart

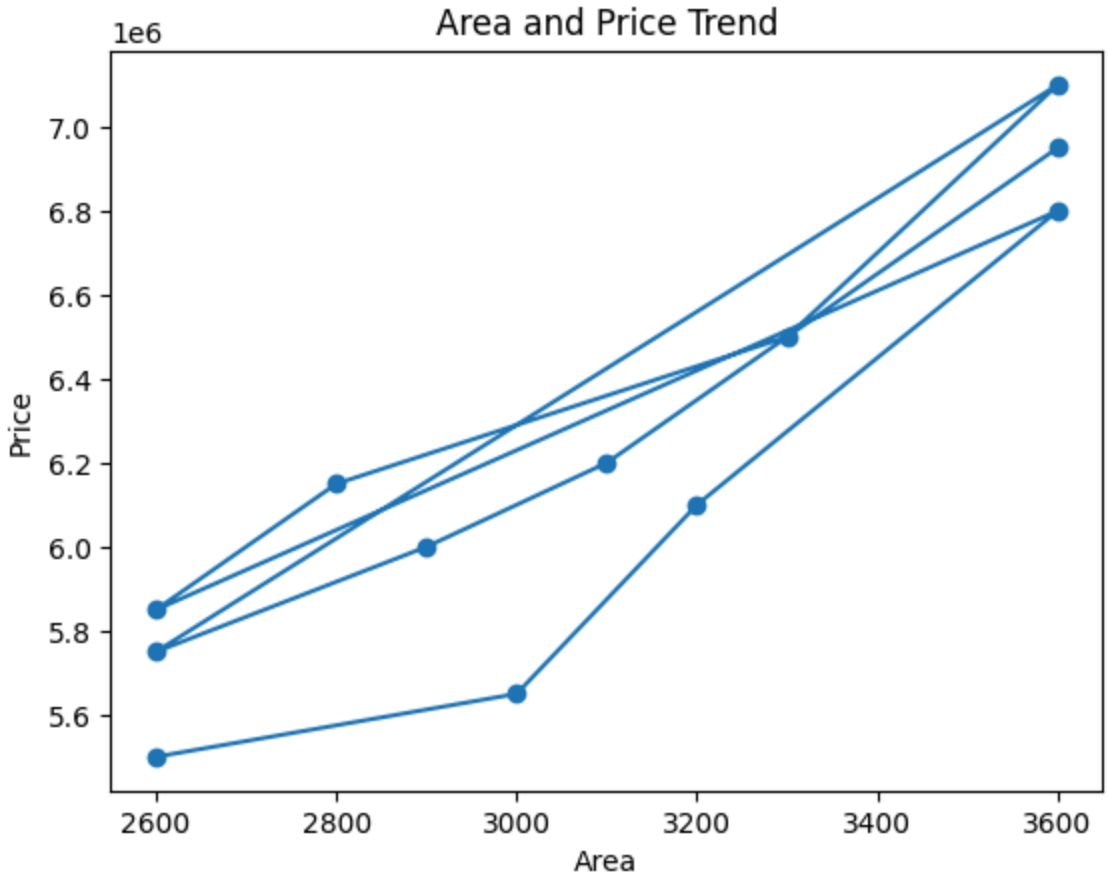
In [28]:
```
df.groupby('town')['price'].mean().plot(kind='bar')
plt.xlabel('Town')
plt.ylabel('Average Price')
plt.title('Average Price by Town')
plt.show()
```



In [ ]:

5} Line Plot

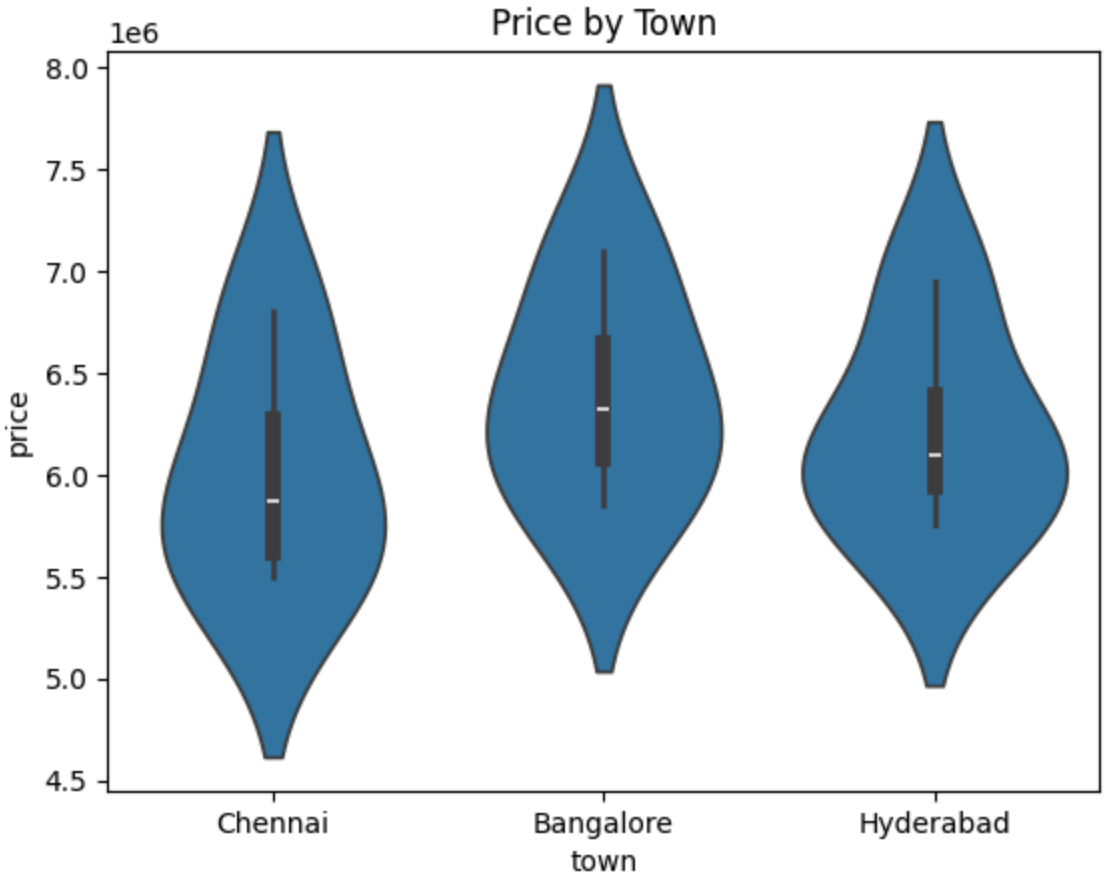
In [29]:
```
plt.plot(df['area'], df['price'], marker='o')
plt.xlabel('Area')
plt.ylabel('Price')
plt.title('Area and Price Trend')
plt.show()
```



In [ ]:

6} Violin Plot

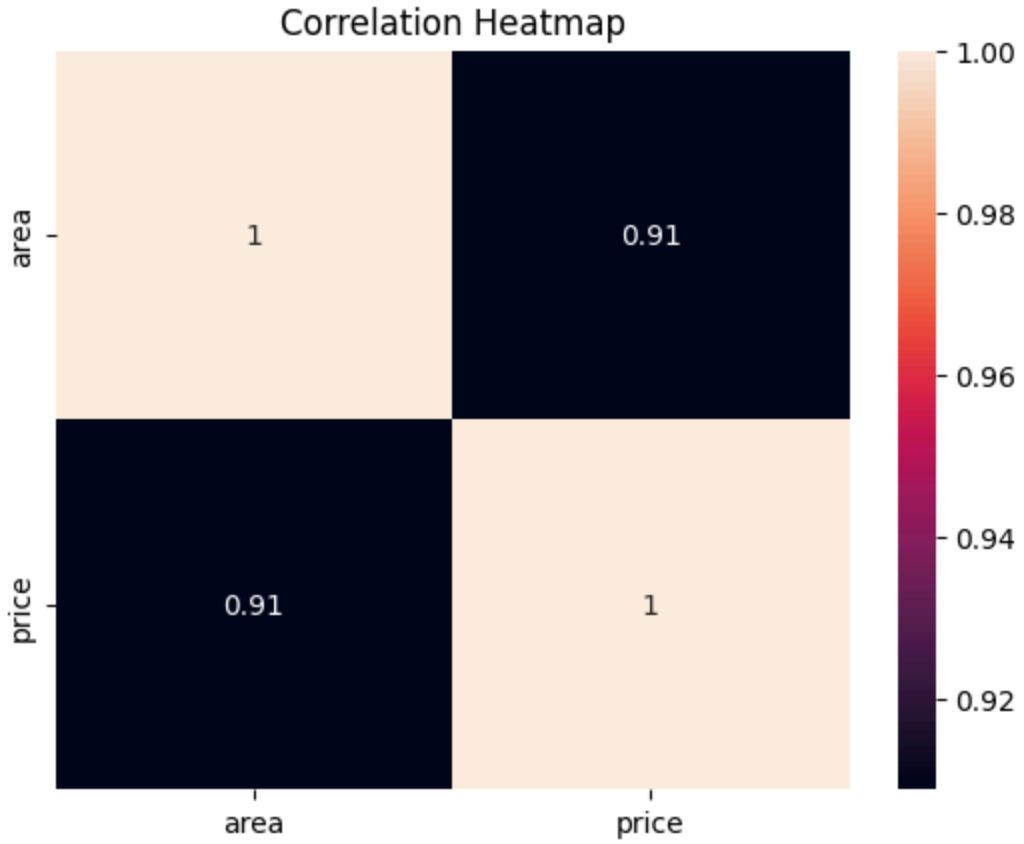
In [30]:
```
sns.violinplot(x='town', y='price', data=df)
plt.title('Price by Town')
plt.show()
```



In [ ]:

7} Heatmap

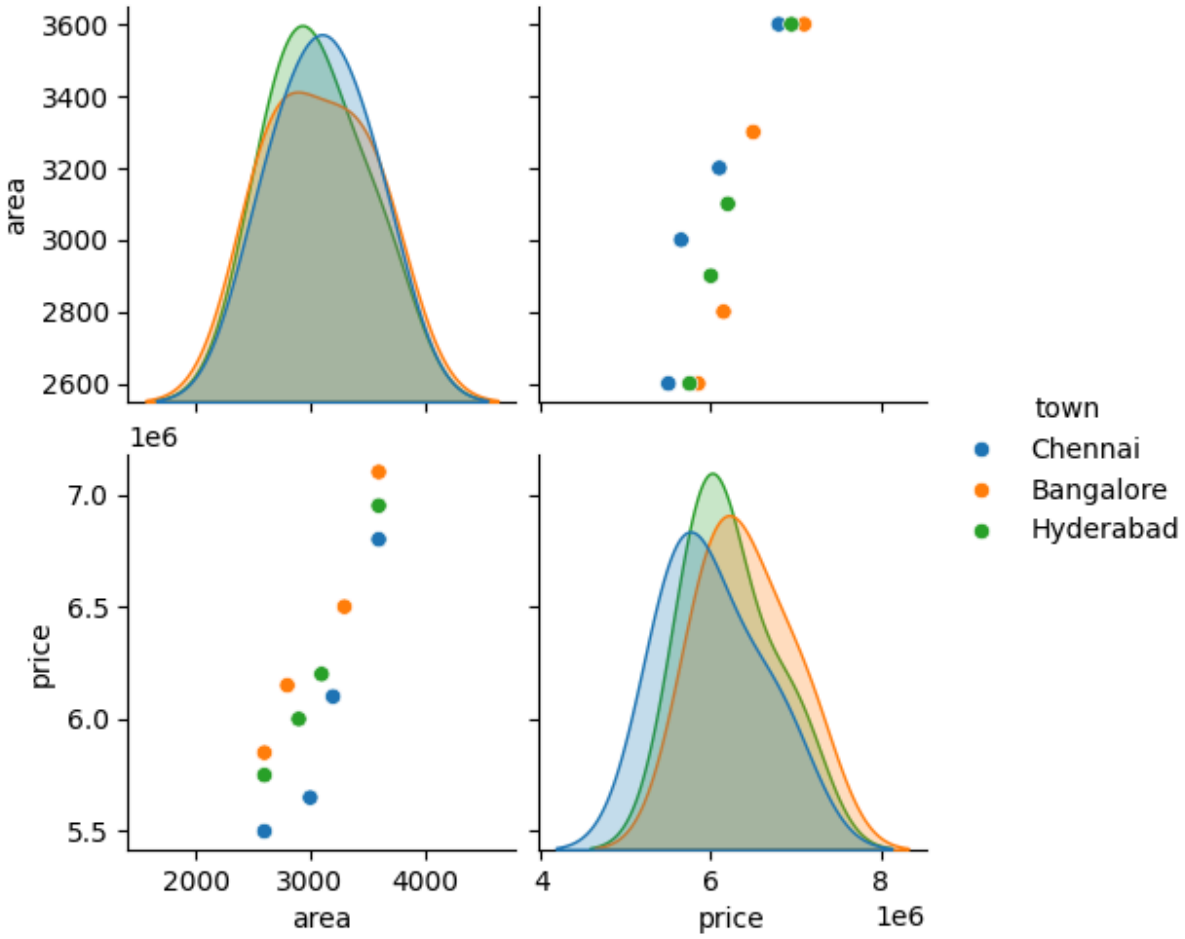
In [31]:
```
sns.heatmap(df.select_dtypes('number').corr(), annot=True)
plt.title('Correlation Heatmap')
plt.show()
```



In [ ]:

8} Pairplot

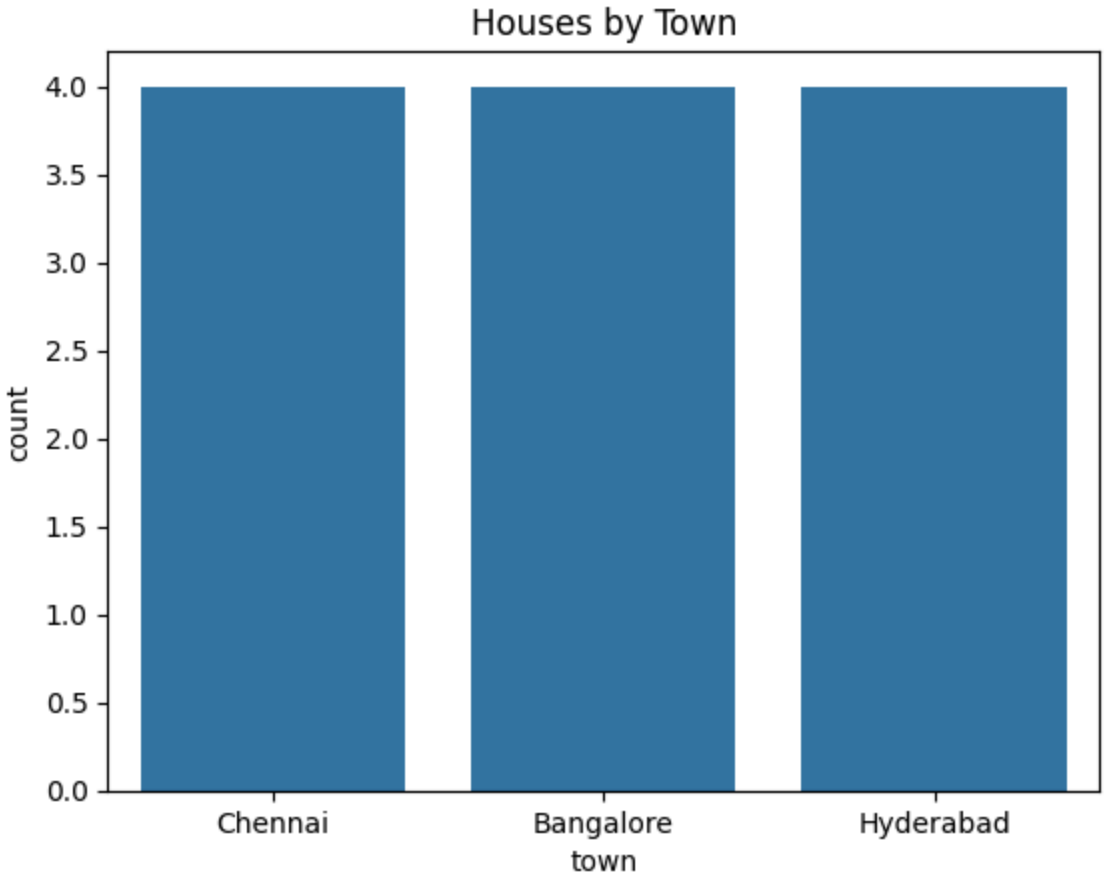
In [32]:
```
sns.pairplot(df, hue='town')
plt.show()
```



In [ ]:

9} Countplot

In [33]:
```
sns.countplot(x='town', data=df)
plt.title('Houses by Town')
plt.show()
```



In [ ]:

4] Feature Engineering

X and y Separation:

In [34]:
```
X = df[['town', 'area']]
y = df['price']
```

4.1] Encoding Categorical Feature

In [35]:
```
X = pd.get_dummies(X, columns=['town'], drop_first=True)
```

4.2] Feature Scaling

In [36]:
```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

In [ ]:

5] Train-Test Split

In [37]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
X_scaled,
  y,
    test_size=0.2,
    random_state=42
)
```

Check Training and Testing Data

In [38]:
```
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (9, 3)
X_test shape: (3, 3)
y_train shape: (9,)
y_test shape: (3,)
```

In [ ]:

In [ ]:

3. Mushrooms Dataset

1] Load Dataset

In [39]:
```
df = pd.read_csv("mushrooms.csv")
df.head()
```

Out[39]:

	class	cap-shape	cap-surface	cap-color	bruises	odor	gill-attachment	gill-spacing	gill-size	gill-color	...	sta surfa belo ri
0	p	x	s	n	t	p	f	c	n	k	...	
1	e	x	s	y	t	a	f	c	b	k	...	
2	e	b	s	w	t	l	f	c	b	n	...	
3	p	x	y	w	t	p	f	c	n	n	...	
4	e	x	s	g	f	n	f	w	b	k	...	

5 rows × 23 columns

In [40]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8124 entries, 0 to 8123
Data columns (total 23 columns):
 #   Column                    Non-Null Count  Dtype 
---  ------                    --------------  ----- 
 0   class                     8124 non-null   object
 1   cap-shape                 8124 non-null   object
 2   cap-surface               8124 non-null   object
 3   cap-color                 8124 non-null   object
 4   bruises                   8124 non-null   object
 5   odor                      8124 non-null   object
 6   gill-attachment           8124 non-null   object
 7   gill-spacing              8124 non-null   object
 8   gill-size                 8124 non-null   object
 9   gill-color                8124 non-null   object
 10  stalk-shape               8124 non-null   object
 11  stalk-root                8124 non-null   object
 12  stalk-surface-above-ring  8124 non-null   object
 13  stalk-surface-below-ring  8124 non-null   object
 14  stalk-color-above-ring    8124 non-null   object
 15  stalk-color-below-ring    8124 non-null   object
 16  veil-type                 8124 non-null   object
 17  veil-color                8124 non-null   object
 18  ring-number               8124 non-null   object
 19  ring-type                 8124 non-null   object
 20  spore-print-color         8124 non-null   object
 21  population                8124 non-null   object
 22  habitat                   8124 non-null   object
dtypes: object(23)
memory usage: 1.4+ MB
```

In [41]:
```
df.columns
```

Out[41]:
```
Index(['class', 'cap-shape', 'cap-surface', 'cap-color', 'bruises', 'odor',
       'gill-attachment', 'gill-spacing', 'gill-size', 'gill-color',
       'stalk-shape', 'stalk-root', 'stalk-surface-above-ring',
       'stalk-surface-below-ring', 'stalk-color-above-ring',
       'stalk-color-below-ring', 'veil-type', 'veil-color', 'ring-number',
       'ring-type', 'spore-print-color', 'population', 'habitat'],
      dtype='object')
```

2] Data Cleaning

Check Missing Values

In [42]:
```
df.isnull().sum()
```

Out[42]:
```
class                     0
cap-shape                 0
cap-surface               0
cap-color                 0
bruises                   0
odor                      0
gill-attachment           0
gill-spacing              0
gill-size                 0
gill-color                0
stalk-shape               0
stalk-root                0
stalk-surface-above-ring  0
stalk-surface-below-ring  0
stalk-color-above-ring    0
stalk-color-below-ring    0
veil-type                 0
veil-color                0
ring-number               0
ring-type                 0
spore-print-color         0
population                0
habitat                   0
dtype: int64
```

Check Duplicate Values (remove if required)

In [43]:
```
df.duplicated().sum()
```

Out[43]:
```
0
```

Correct Data Types

In [44]:
```
df.dtypes
```

Out[44]:
```
class                       object
cap-shape                   object
cap-surface                 object
cap-color                   object
bruises                     object
odor                        object
gill-attachment             object
gill-spacing                object
gill-size                   object
gill-color                  object
stalk-shape                 object
stalk-root                  object
stalk-surface-above-ring    object
stalk-surface-below-ring    object
stalk-color-above-ring      object
stalk-color-below-ring      object
veil-type                   object
veil-color                  object
ring-number                 object
ring-type                   object
spore-print-color           object
population                  object
habitat                     object
dtype: object
```

In [45]:
```
for column in df.columns:
    df[column] = df[column].astype("category")
```

In [46]:
```
df.dtypes
```

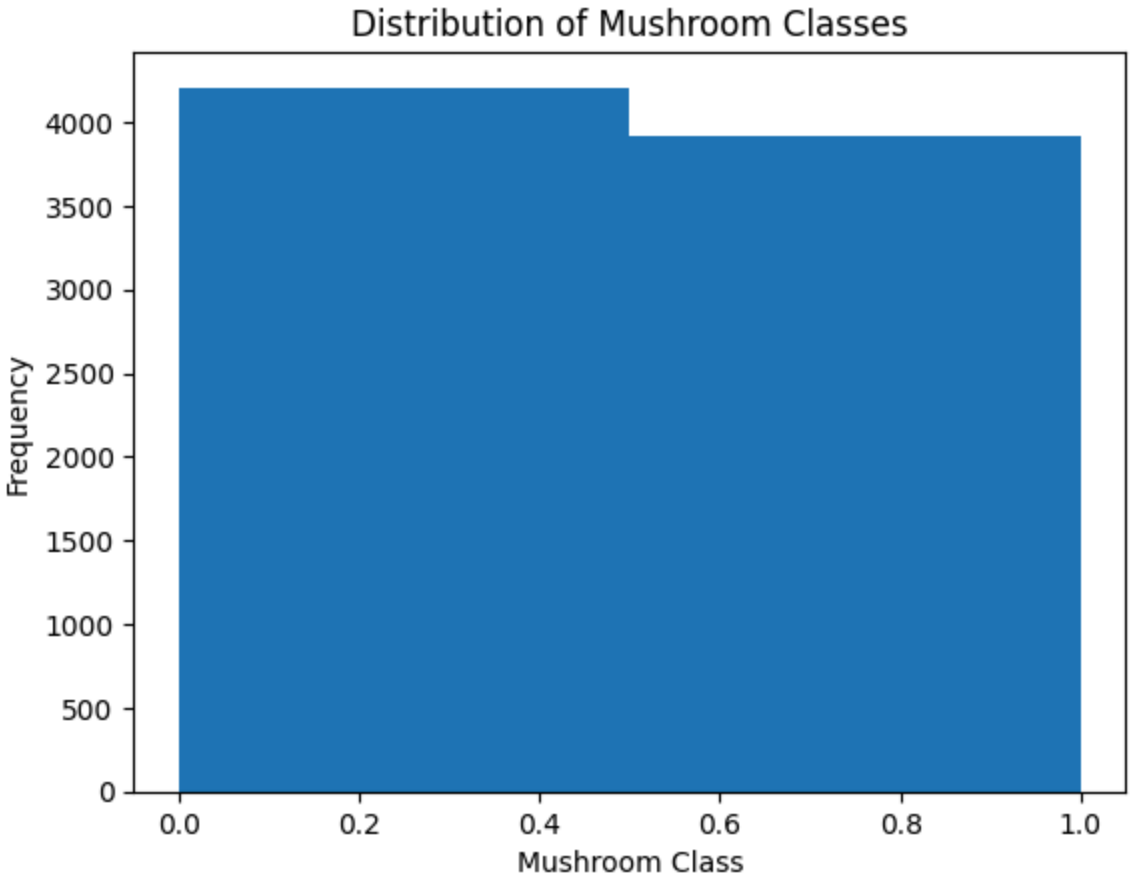
Out[46]:
```
class                       category
cap-shape                   category
cap-surface                 category
cap-color                   category
bruises                     category
odor                        category
gill-attachment             category
gill-spacing                category
gill-size                   category
gill-color                  category
stalk-shape                 category
stalk-root                  category
stalk-surface-above-ring    category
stalk-surface-below-ring    category
stalk-color-above-ring      category
stalk-color-below-ring      category
veil-type                   category
veil-color                  category
ring-number                 category
ring-type                   category
spore-print-color           category
population                  category
habitat                     category
dtype: object
```

In [ ]:

3] Exploratory Data Analysis (EDA)

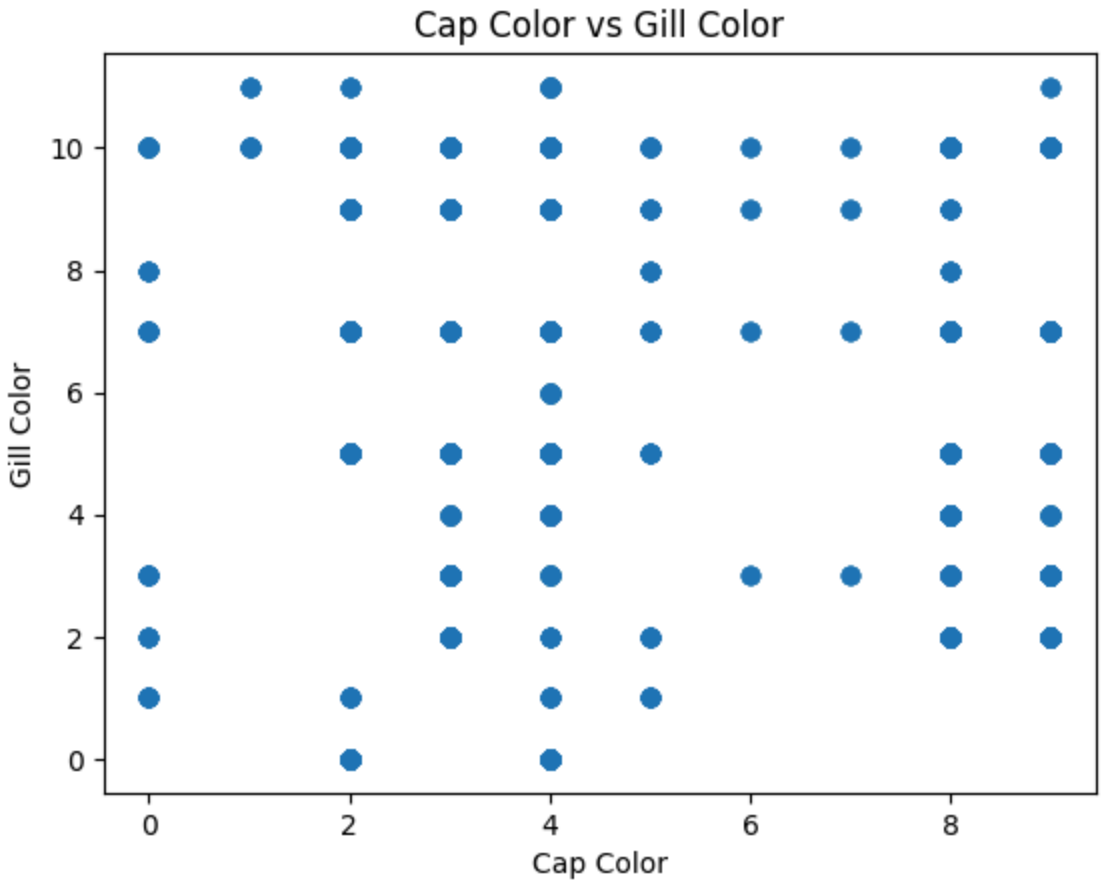
1} Histogram

In [47]:
```
class_count = df["class"].cat.codes
plt.hist(class_count, bins=2)
plt.title("Distribution of Mushroom Classes")
plt.xlabel("Mushroom Class")
plt.ylabel("Frequency")
plt.show()
```



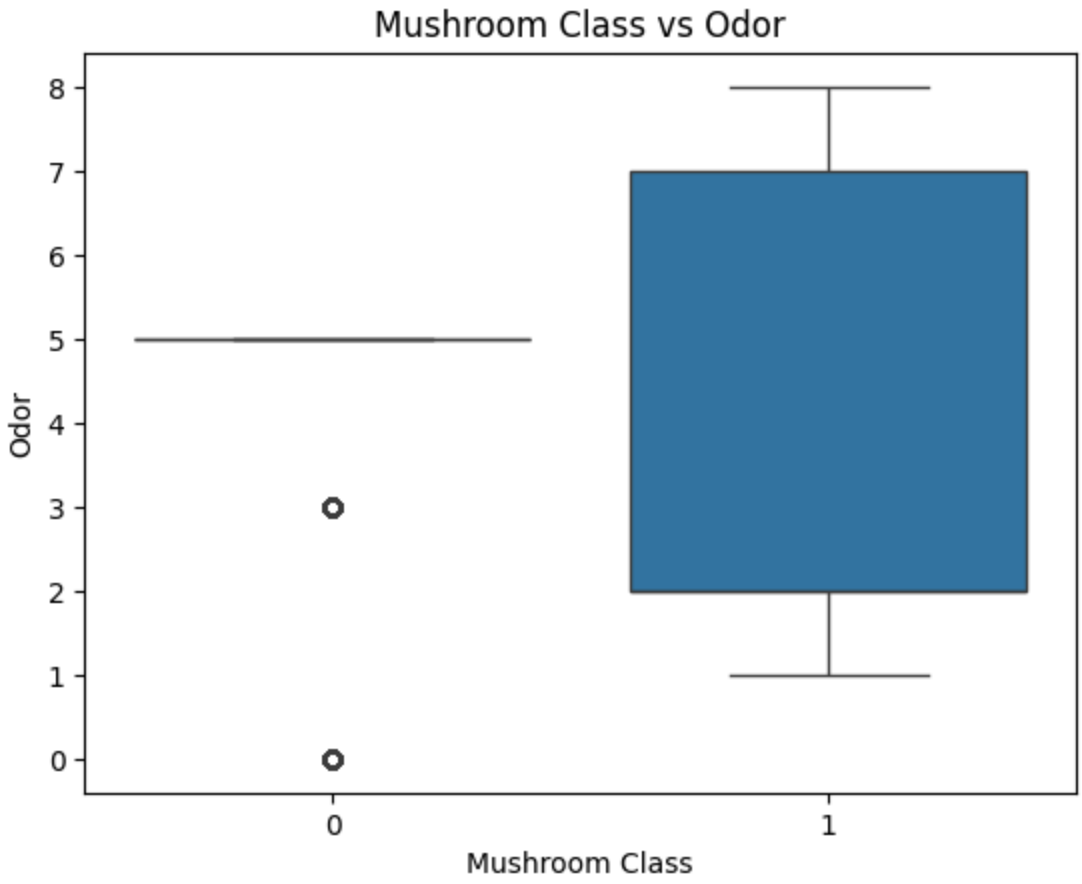
2} Scatter Plot

In [48]:
```
plt.scatter(
df["cap-color"].cat.codes,
df["gill-color"].cat.codes
)
plt.title("Cap Color vs Gill Color")
plt.xlabel("Cap Color")
plt.ylabel("Gill Color")
plt.show()
```



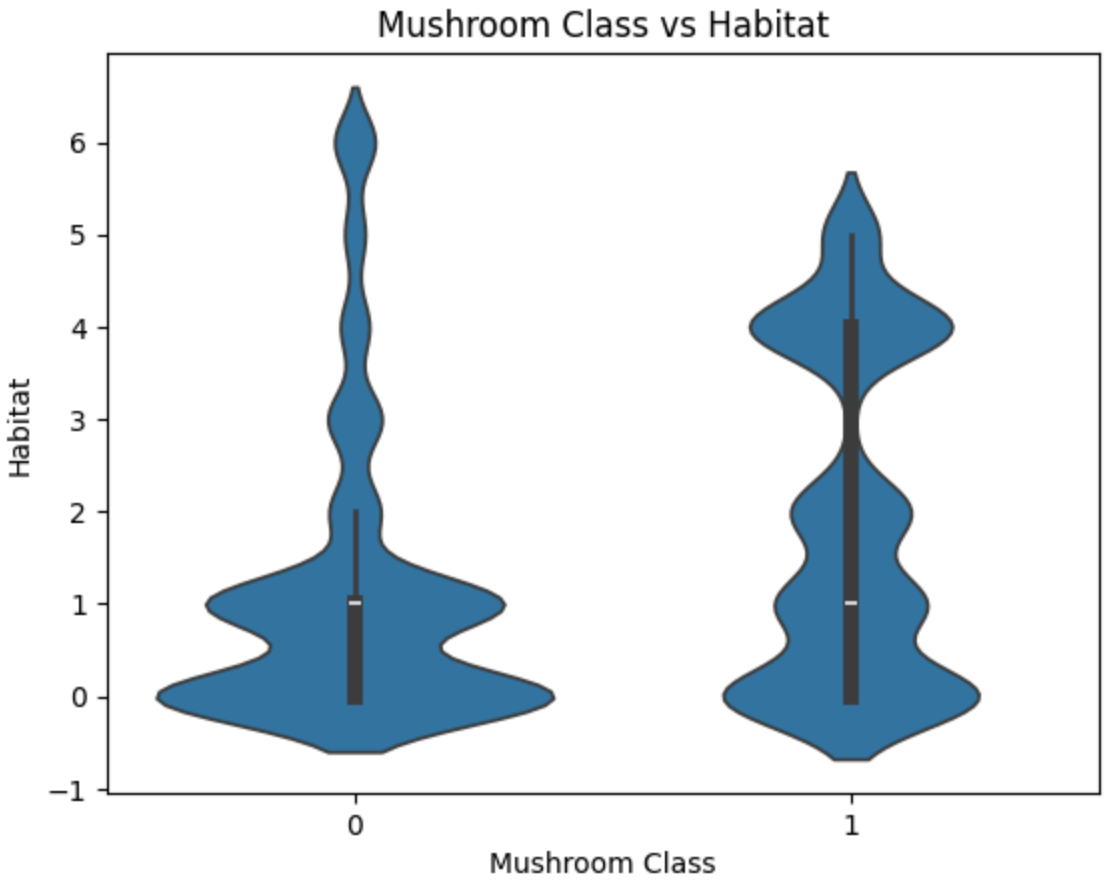
3} Box Plot

In [49]:
```
sns.boxplot(
x=df["class"].cat.codes,
y=df["odor"].cat.codes
)
plt.title("Mushroom Class vs Odor")
plt.xlabel("Mushroom Class")
plt.ylabel("Odor")
plt.show()
```



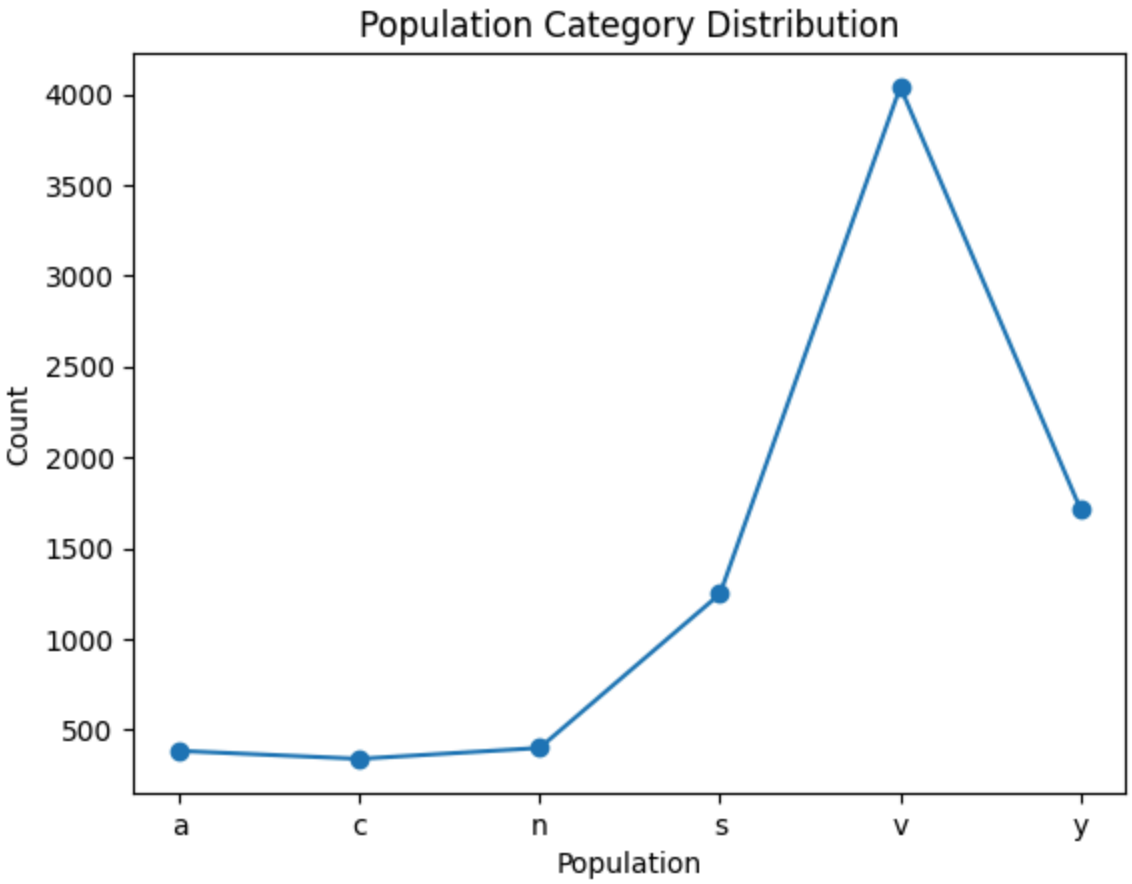
4} Violin Plot

In [50]:
```
sns.violinplot(
x=df["class"].cat.codes,
y=df["habitat"].cat.codes
)
plt.title("Mushroom Class vs Habitat")
plt.xlabel("Mushroom Class")
plt.ylabel("Habitat")
plt.show()
```



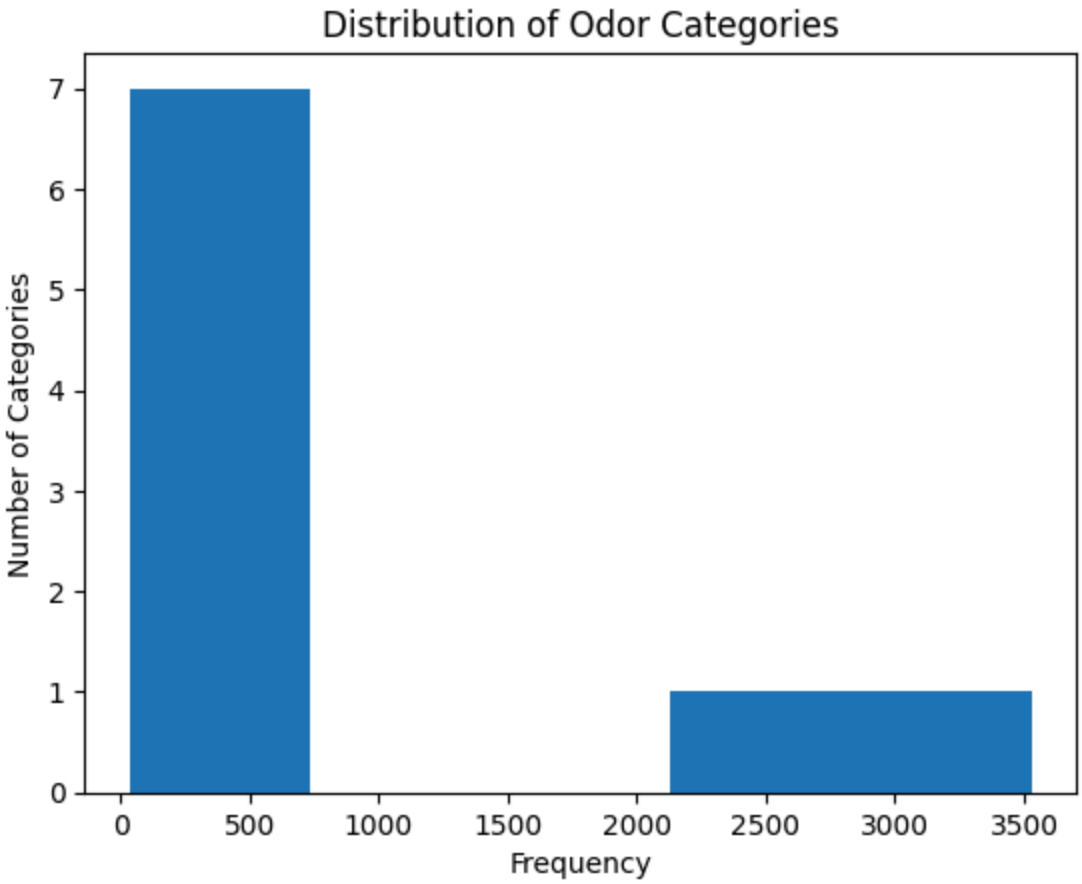
5} Line Plot

In [51]:
```
population_count = df["population"].value_counts().sort_index()
plt.plot(
population_count.index,
population_count.values,
marker="o"
)
plt.title("Population Category Distribution")
plt.xlabel("Population")
plt.ylabel("Count")
plt.show()
```



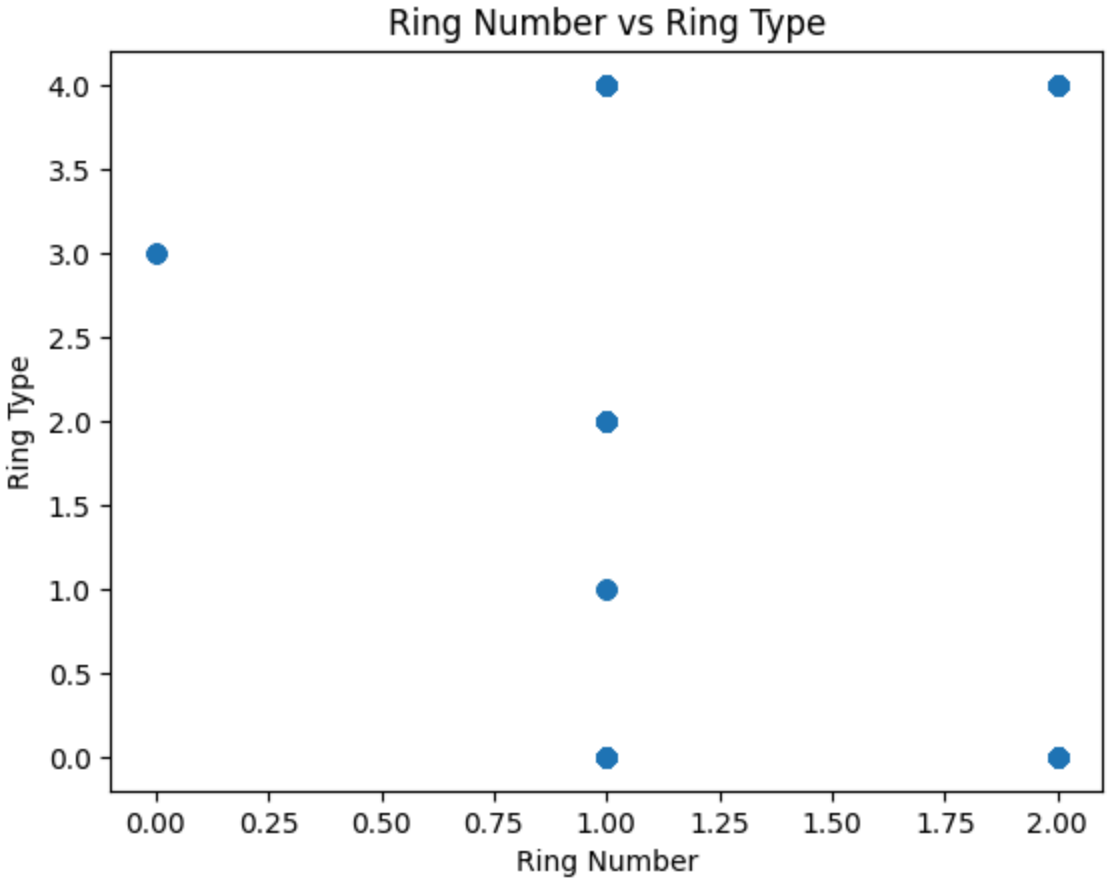
6} Histogram

In [52]:
```
odor_count = df["odor"].value_counts()
plt.hist(
odor_count.values,
bins=5
)
plt.title("Distribution of Odor Categories")
plt.xlabel("Frequency")
plt.ylabel("Number of Categories")
plt.show()
```



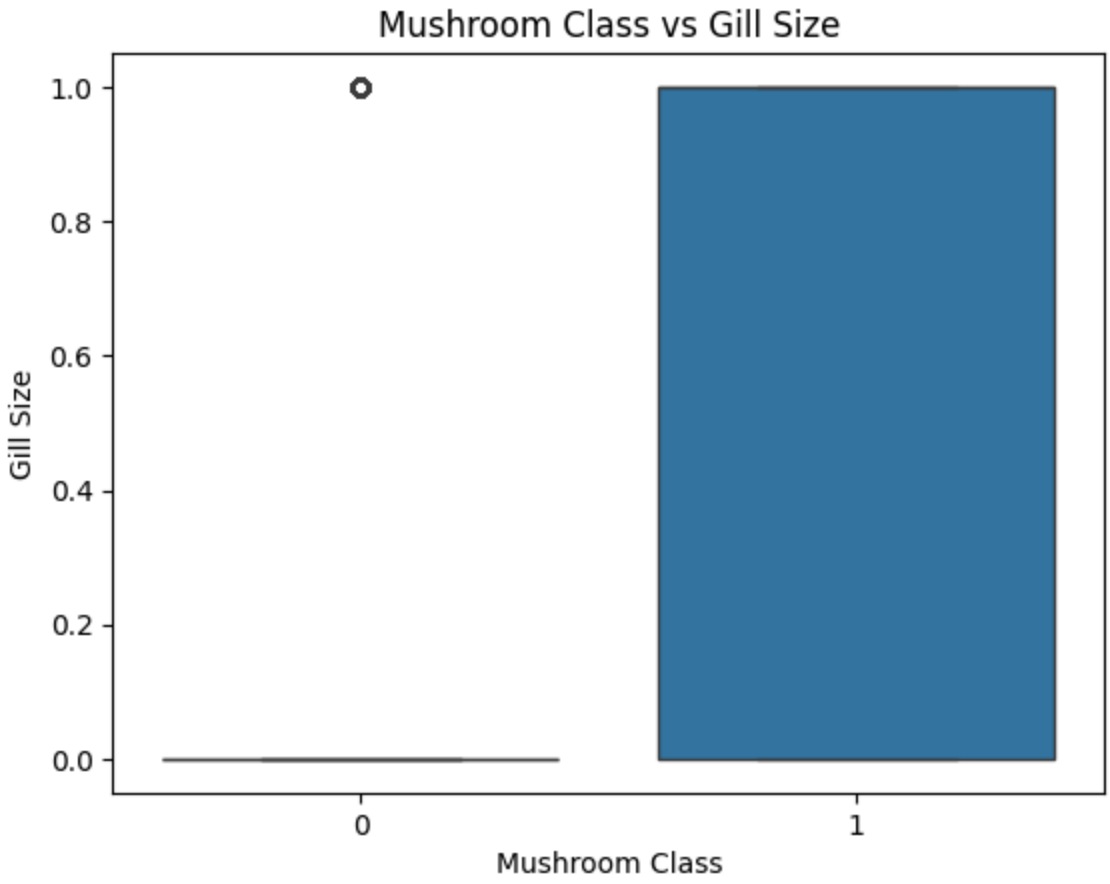
7} Scatter Plot

In [53]:
```
plt.scatter(
df["ring-number"].cat.codes,
df["ring-type"].cat.codes
)
plt.title("Ring Number vs Ring Type")
plt.xlabel("Ring Number")
plt.ylabel("Ring Type")
plt.show()
```



8} Box Plot

In [54]:
```
sns.boxplot(
x=df["class"].cat.codes,
y=df["gill-size"].cat.codes
)
plt.title("Mushroom Class vs Gill Size")
plt.xlabel("Mushroom Class")
plt.ylabel("Gill Size")
plt.show()
```

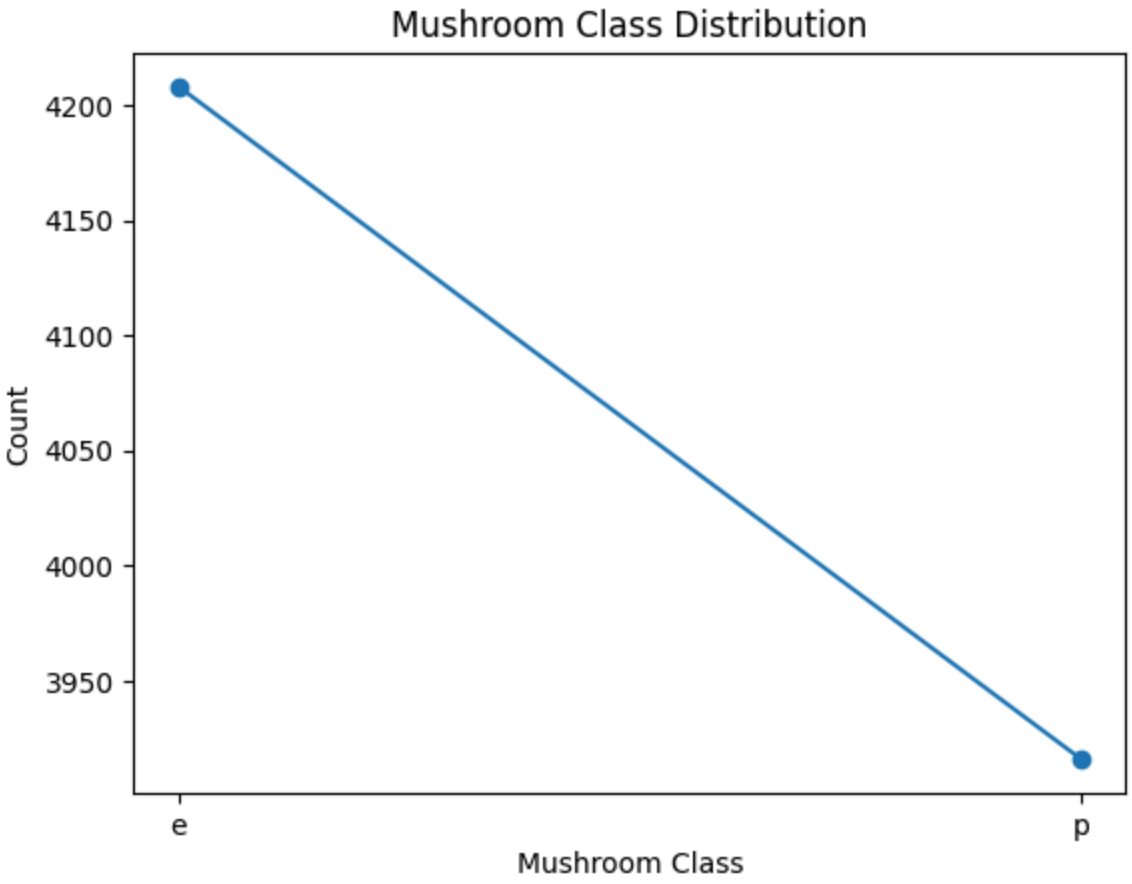


9} Violin Plot

In [55]:
```
sns.violinplot(
x=df["class"].cat.codes,
y=df["population"].cat.codes
)
plt.title("Mushroom Class vs Population")
plt.xlabel("Mushroom Class")
plt.ylabel("Population")
plt.show()
```

10} Line Plot

In [56]:
```
class_count = df["class"].value_counts().sort_index()
plt.plot(
class_count.index,
class_count.values,
marker="o"
)
plt.title("Mushroom Class Distribution")
plt.xlabel("Mushroom Class")
plt.ylabel("Count")
plt.show()
```



4] Feature Engineering

Separate x and y:

In [57]:
```
X = df.drop("class", axis=1)
y = df["class"]
```

In [58]:
```
X = pd.get_dummies(
X,
drop_first=True,
dtype=int
)
```

In [59]:
```
X.head()
```

Out[59]:

	cap-shape_c	cap-shape_f	cap-shape_k	cap-shape_s	cap-shape_x	cap-surface_g	cap-surface_s	cap-surface_y	cap-color_c	c
0	0	0	0	0	1	0	1	0	0	
1	0	0	0	0	1	0	1	0	0	
2	0	0	0	0	0	0	1	0	0	
3	0	0	0	0	1	0	0	1	0	
4	0	0	0	0	1	0	1	0	0	

5 rows × 95 columns

In [60]:
```
y = y.cat.codes
y.head()
```

Out[60]:
```
0    1
1    0
2    0
3    1
4    0
dtype: int8
```

5] Training and Testing sets

In [61]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

In [62]:
```
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (6499, 95)
X_test shape: (1625, 95)
y_train shape: (6499,)
y_test shape: (1625,)
```

In [ ]:

In [ ]:

4. Penguins Size Dataset

1] Load the Dataset

In [63]:
```
# Load the Penguins Size dataset
df = pd.read_csv("penguins_size.csv")
# Display the first 5 rows
df.head()
```

Out[63]:

	species	island	culmen_length_mm	culmen_depth_mm	flipper_length_mm	body_mas
0	Adelie	Torgersen	39.1	18.7	181.0	37
1	Adelie	Torgersen	39.5	17.4	186.0	38
2	Adelie	Torgersen	40.3	18.0	195.0	32
3	Adelie	Torgersen	NaN	NaN	NaN	N
4	Adelie	Torgersen	36.7	19.3	193.0	34

2] Data Cleaning

Check missing values

In [64]:
```
df.isnull().sum()
```

Out[64]:
```
species              0
island               0
culmen_length_mm     2
culmen_depth_mm      2
flipper_length_mm    2
body_mass_g          2
sex                 10
dtype: int64
```

2.1] Handle Missing Values

In [65]:
```
# Fill missing numerical values with the median
df['culmen_length_mm'] = df['culmen_length_mm'].fillna(df['culmen_length_mm'].media
df['culmen_depth_mm'] = df['culmen_depth_mm'].fillna(df['culmen_depth_mm'].median()
df['flipper_length_mm'] = df['flipper_length_mm'].fillna(df['flipper_length_mm'].me
df['body_mass_g'] = df['body_mass_g'].fillna(df['body_mass_g'].median())

# Fill missing categorical values with the mode
df['sex'] = df['sex'].fillna(df['sex'].mode()[0])
```

In [66]:
```
df.isnull().sum()
```

Out[66]:
```
species              0
island               0
culmen_length_mm     0
culmen_depth_mm      0
flipper_length_mm    0
body_mass_g          0
sex                  0
dtype: int64
```

In [67]:
```
df.dtypes
```

Out[67]:
```
species               object
island                object
culmen_length_mm     float64
culmen_depth_mm      float64
flipper_length_mm    float64
body_mass_g          float64
sex                   object
dtype: object
```

No data type is incorrect, so there is no need for data type conversion.!!!

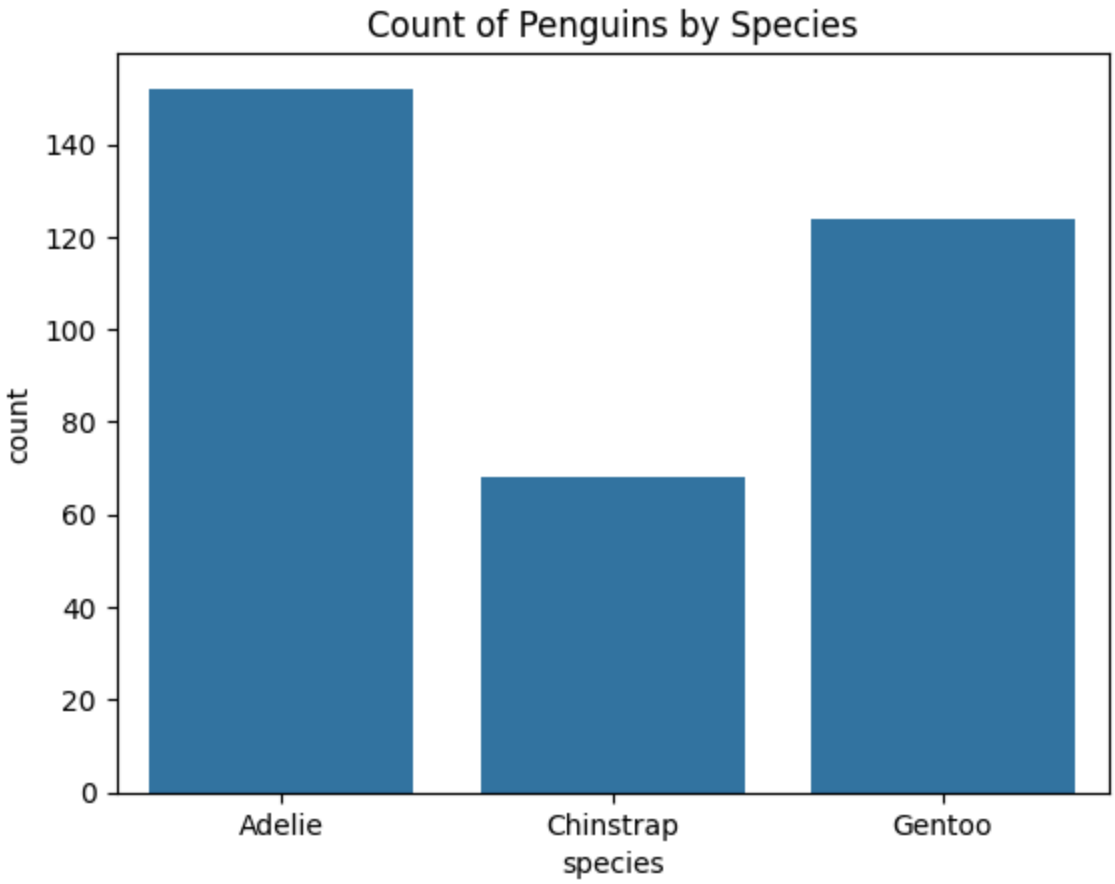
In [68]:
```
df.duplicated().sum()
```

Out[68]:
```
0
```

3] EDA

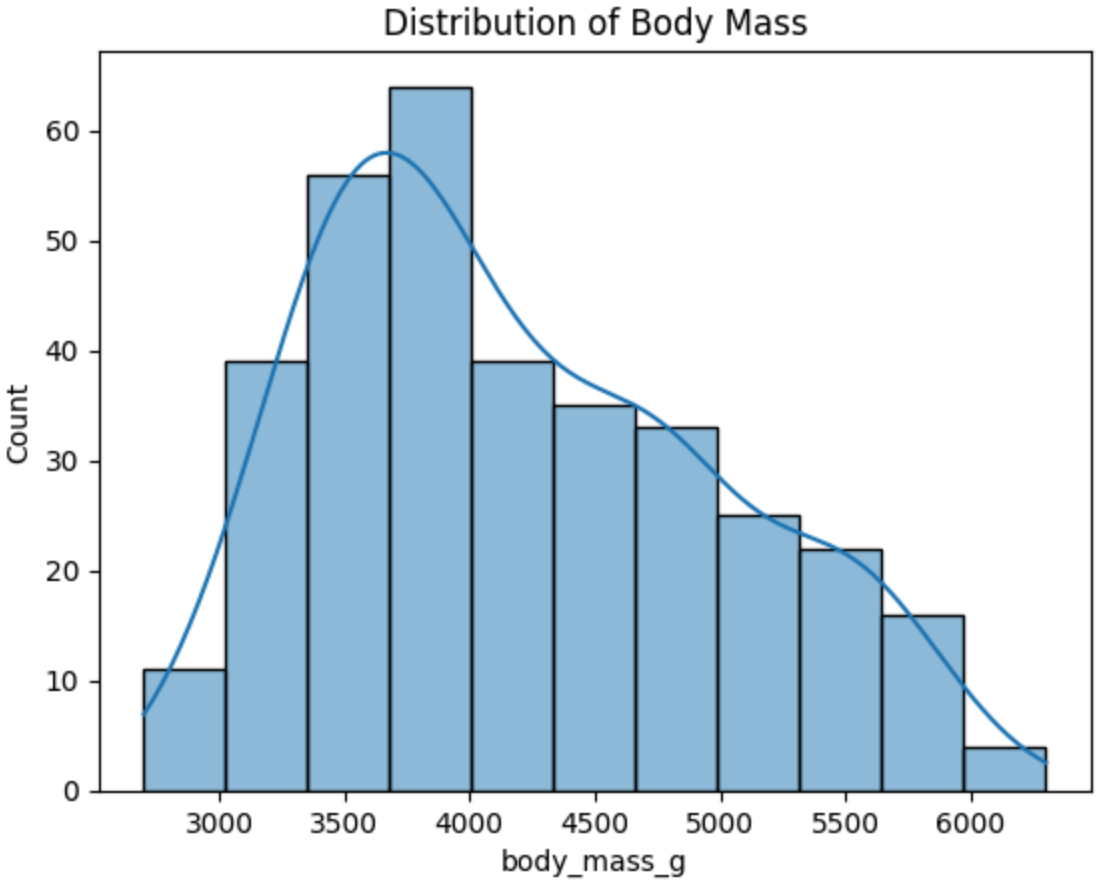
1} Count Plot

In [69]:
```
sns.countplot(x='species', data=df)
plt.title('Count of Penguins by Species')
plt.show()
```



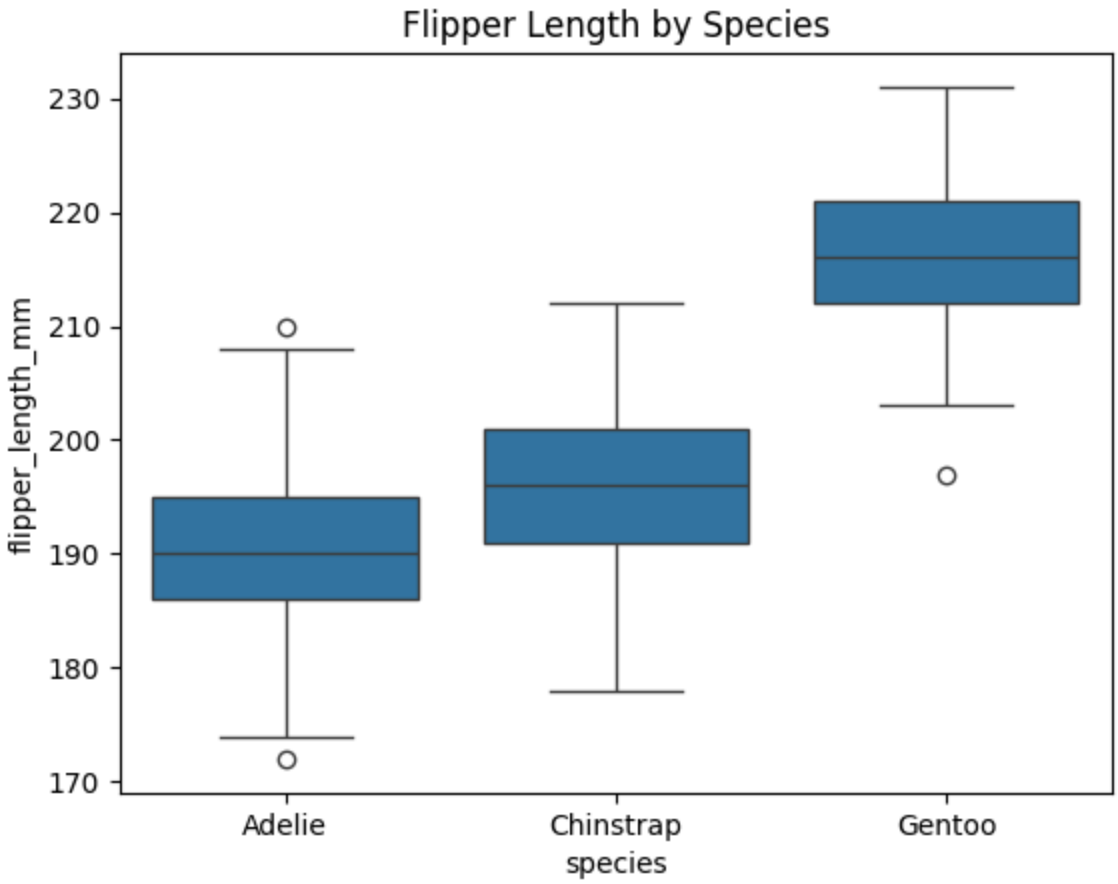
2} Histogram

In [70]:
```
sns.histplot(df['body_mass_g'], kde=True)
plt.title('Distribution of Body Mass')
plt.show()
```



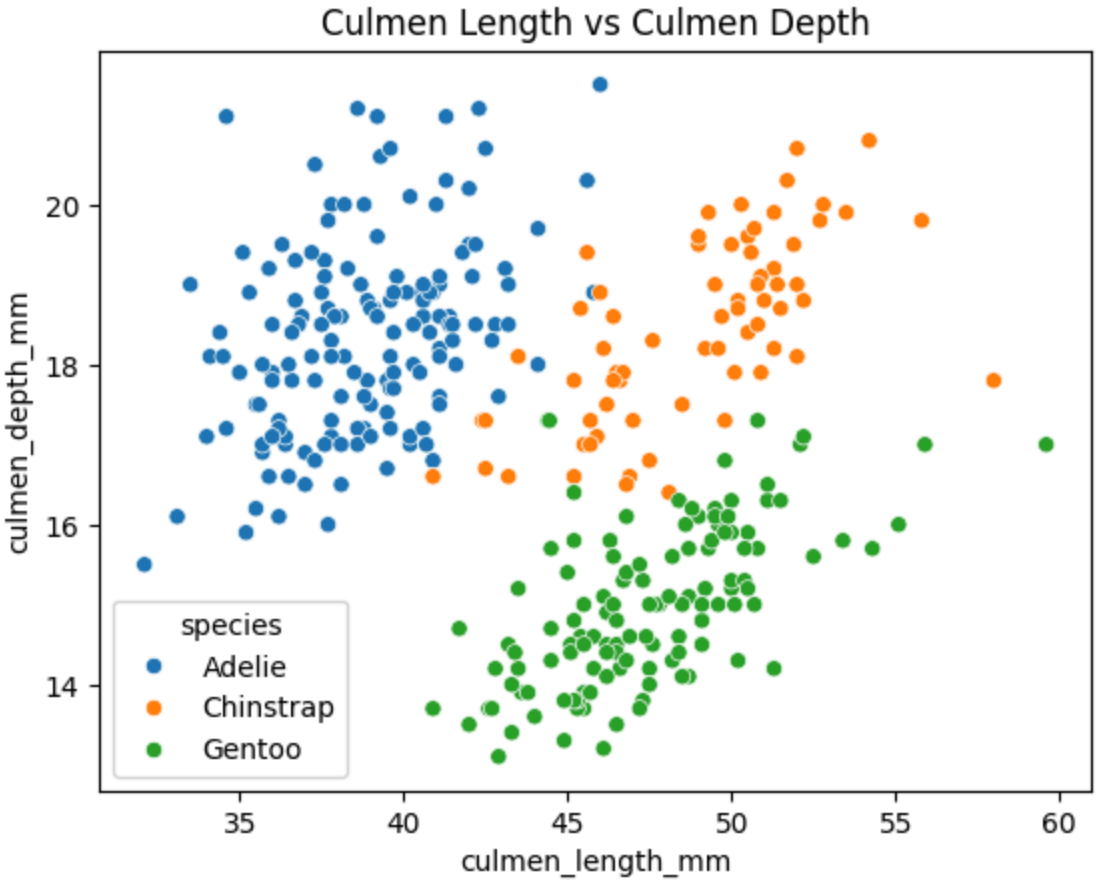
3} Box Plot

In [71]:
```
sns.boxplot(x='species', y='flipper_length_mm', data=df)
plt.title('Flipper Length by Species')
plt.show()
```



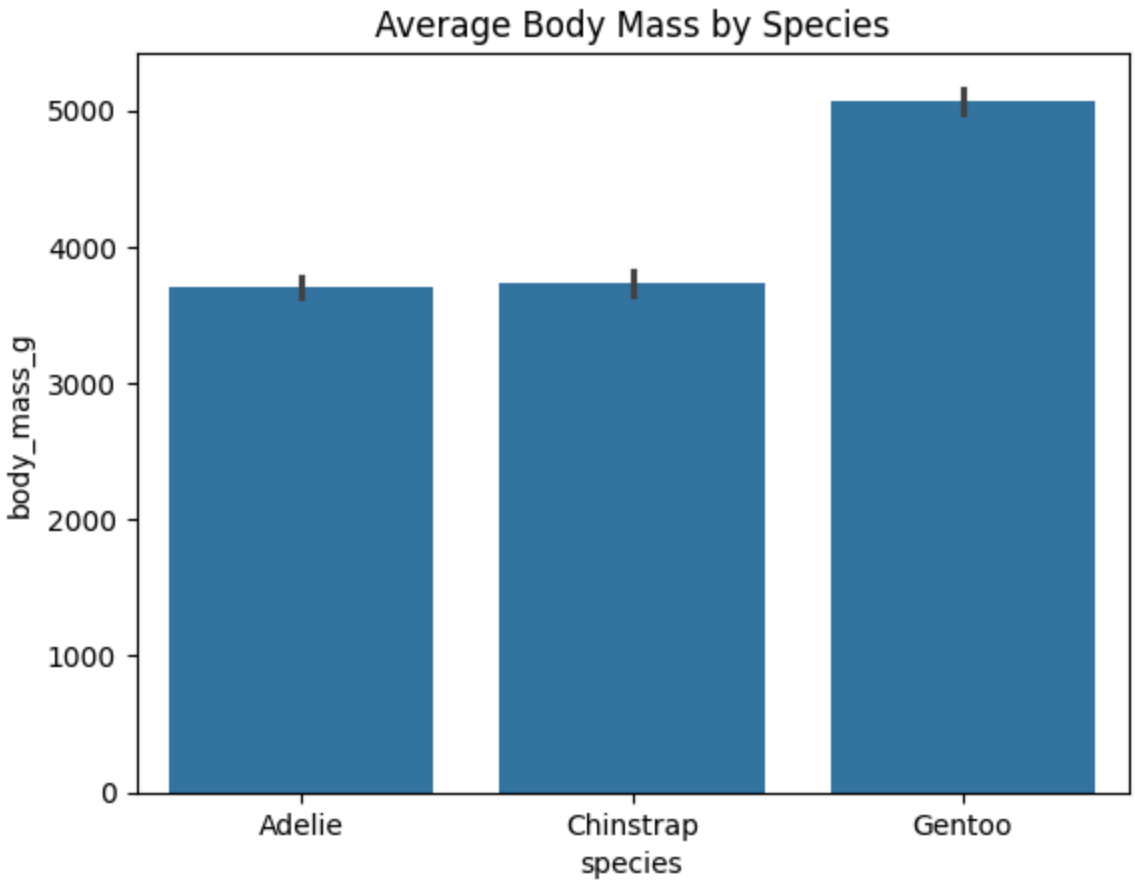
4} Scatter Plot

In [72]:
```
sns.scatterplot(
x='culmen_length_mm',
y='culmen_depth_mm',
hue='species',
data=df
)
plt.title('Culmen Length vs Culmen Depth')
plt.show()
```



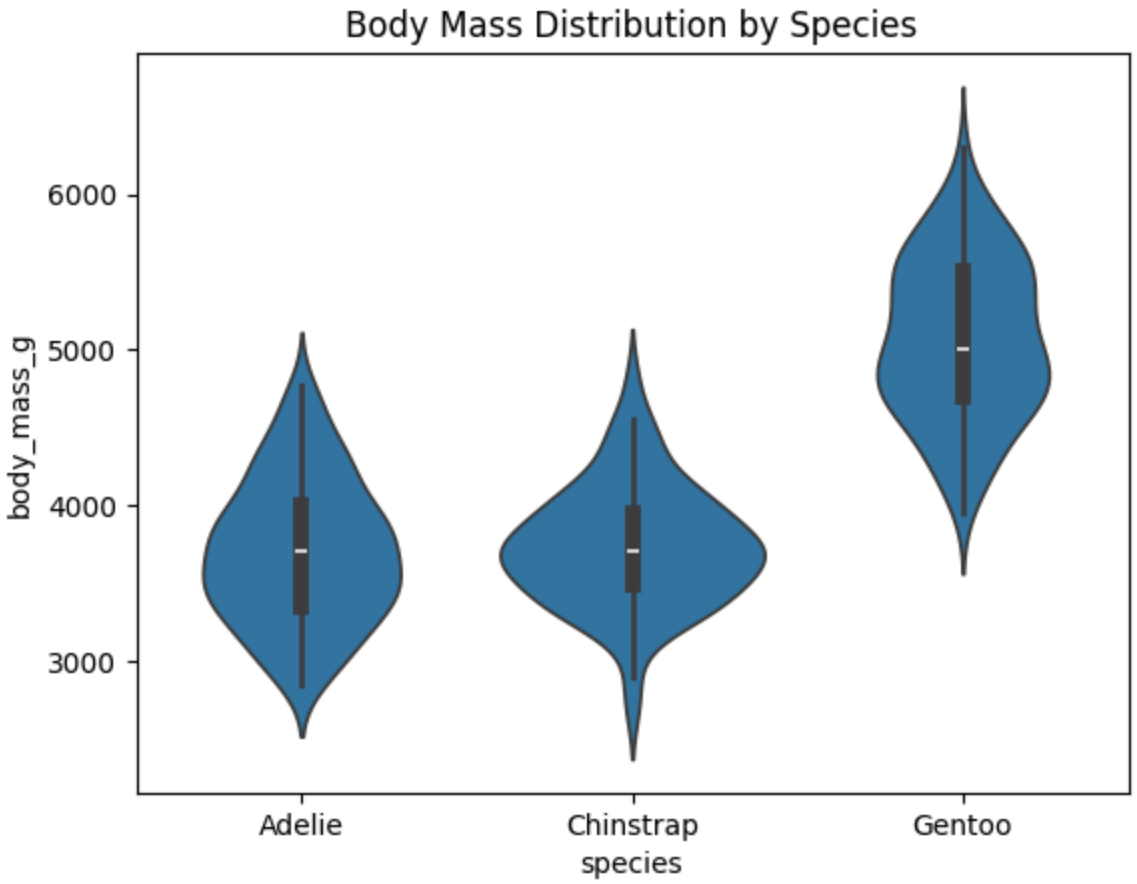
5} Bar Plot

In [73]:
```
sns.barplot(x='species', y='body_mass_g', data=df)
plt.title('Average Body Mass by Species')
plt.show()
```



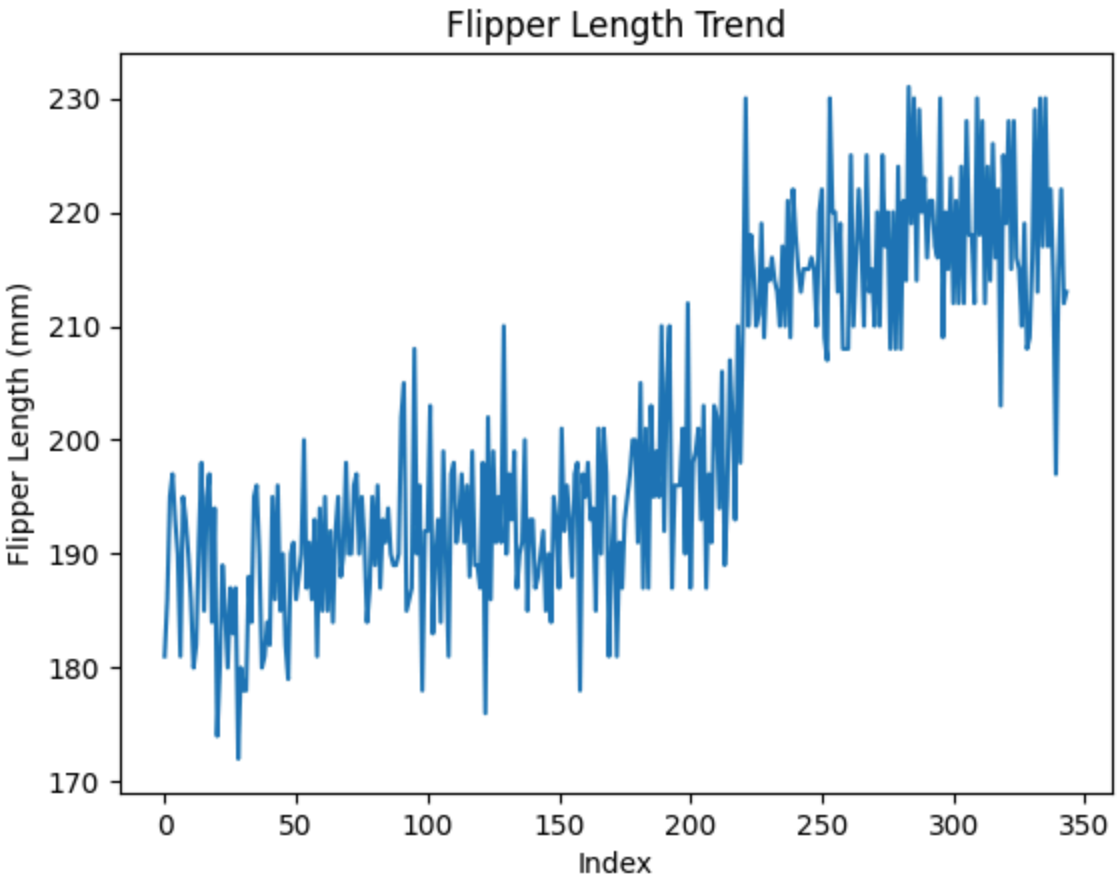
6} Violin Plot

In [74]:
```
sns.violinplot(x='species', y='body_mass_g', data=df)
plt.title('Body Mass Distribution by Species')
plt.show()
```



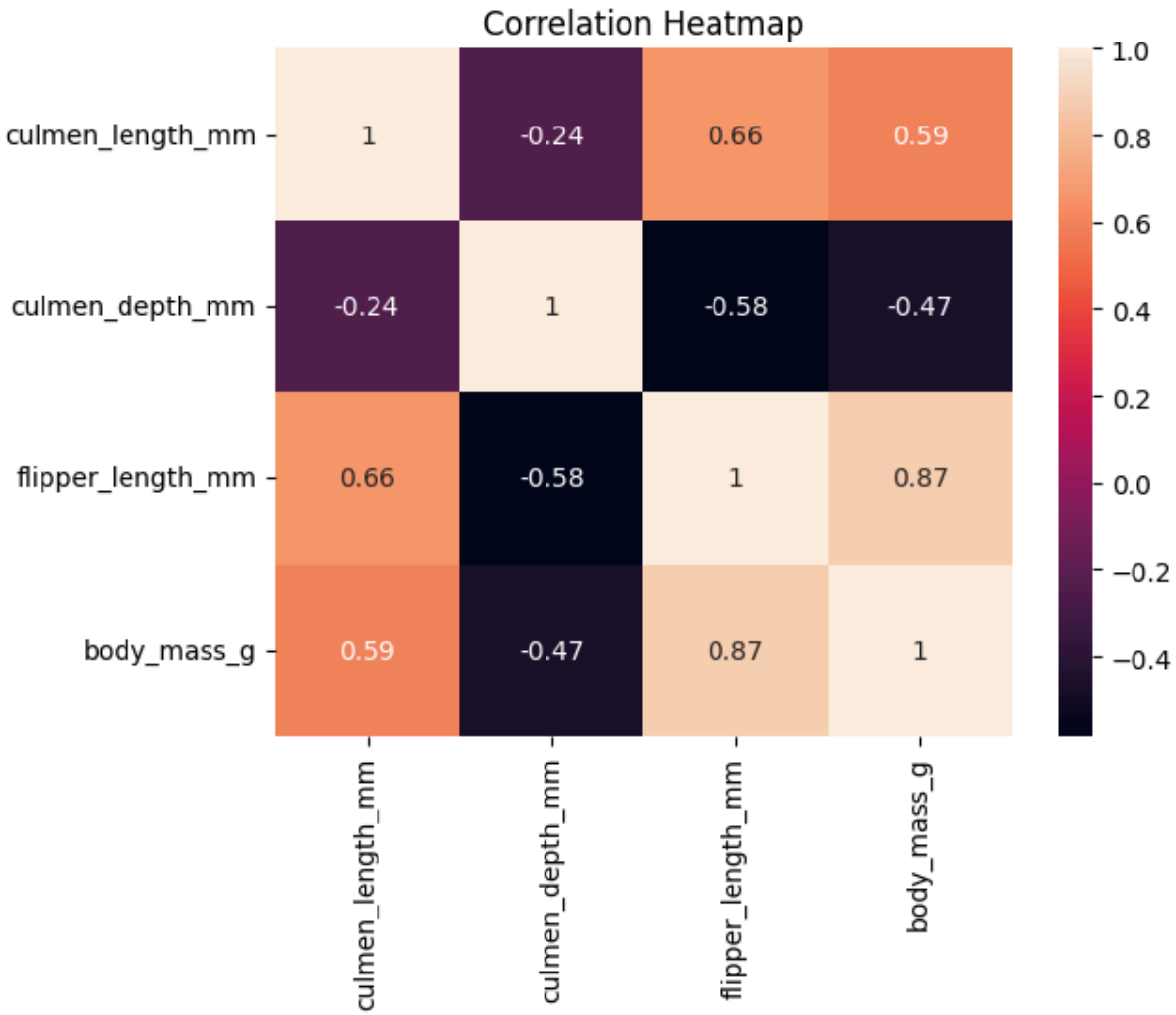
7} Line Plot

In [75]:
```
plt.plot(df['flipper_length_mm'])
plt.title('Flipper Length Trend')
plt.xlabel('Index')
plt.ylabel('Flipper Length (mm)')
plt.show()
```



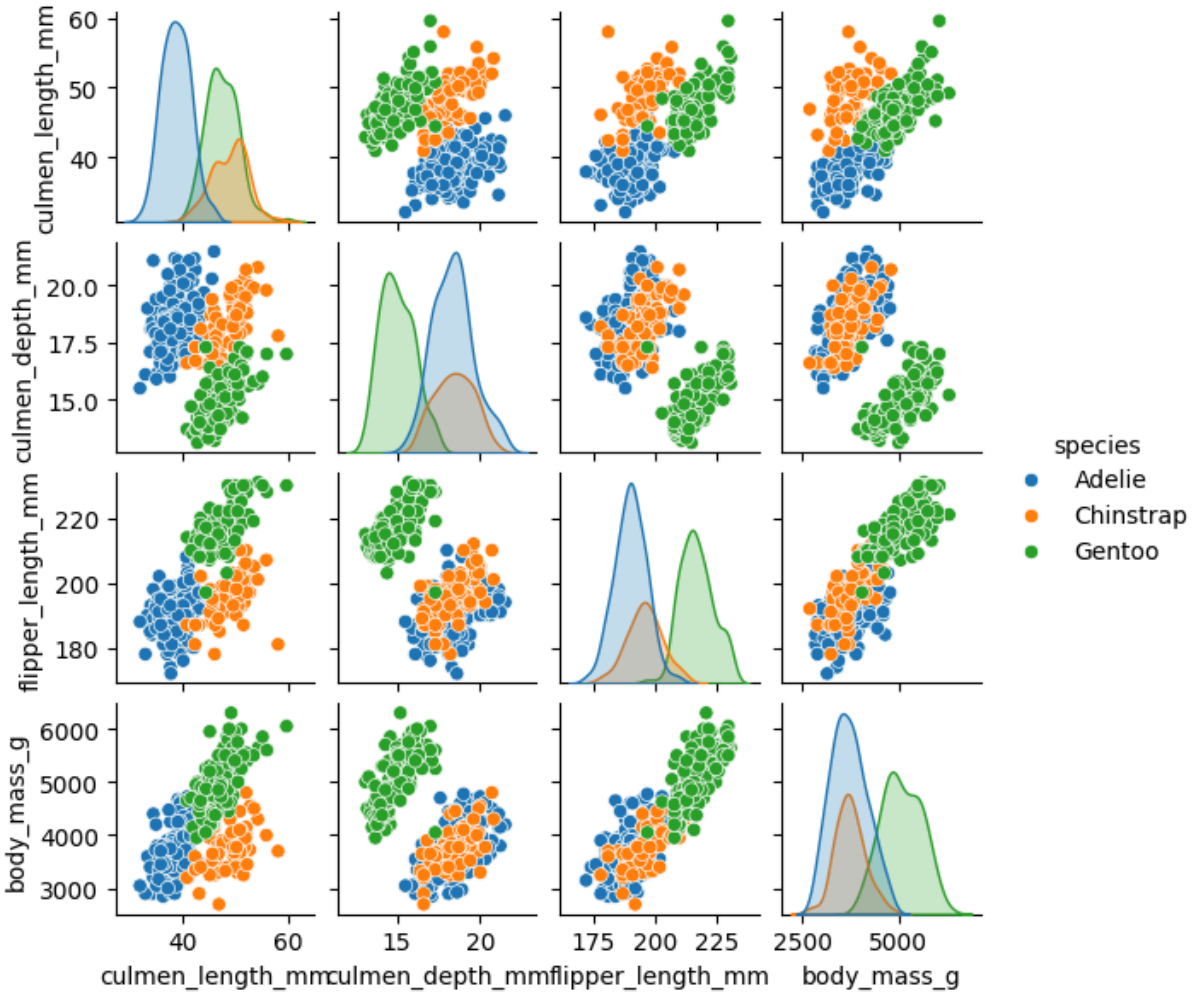
8} Heatmap

In [76]:
```
sns.heatmap(
df.select_dtypes(include='number').corr(),
annot=True
)
plt.title('Correlation Heatmap')
plt.show()
```



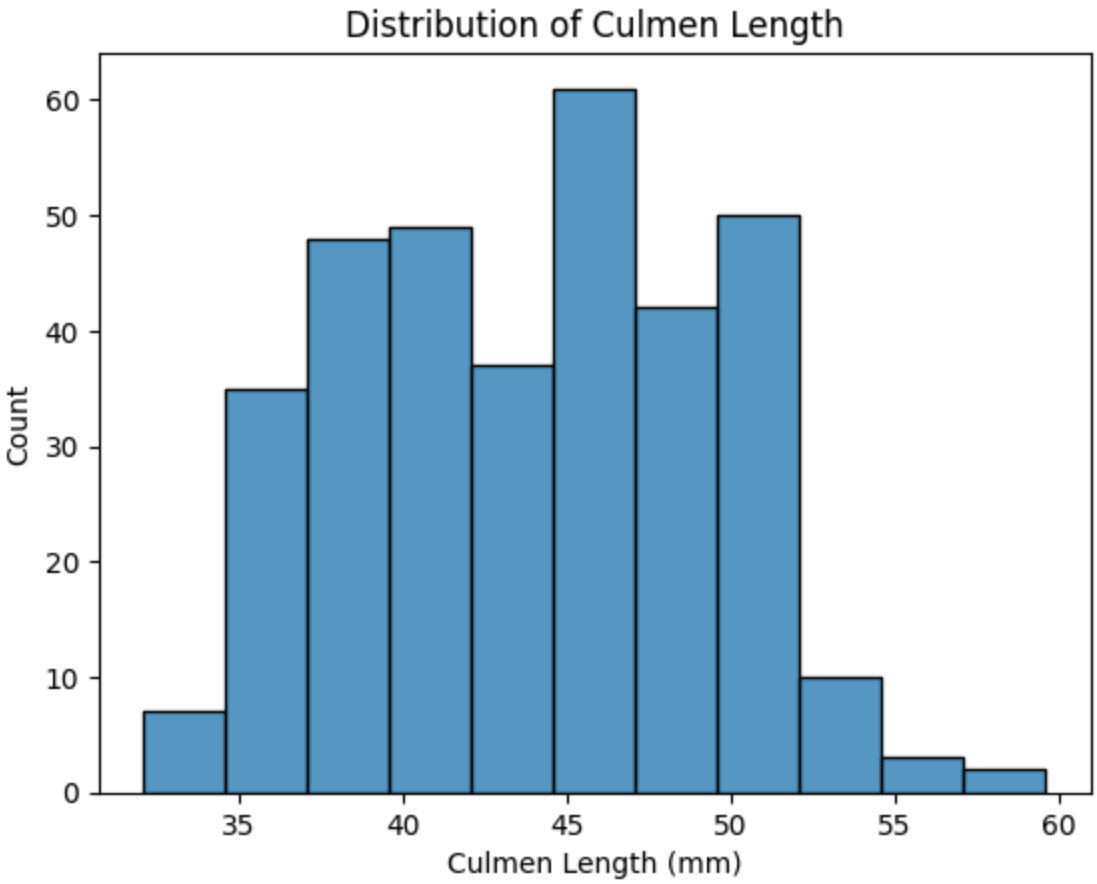
9} Pair Ploto

In [77]:
```
sns.pairplot(df, hue='species', height=1.5)
plt.show()
```



10} Histogram

In [78]:
```
sns.histplot(df['culmen_length_mm'])
plt.title('Distribution of Culmen Length')
plt.xlabel('Culmen Length (mm)')
plt.show()
```



4] Feature Engineering

Separate X and y:

In [79]:
```
X = df.drop('species', axis=1)
y = df['species']
```

In [80]:
```
# Display features
X.head()
```

Out[80]:

	island	culmen_length_mm	culmen_depth_mm	flipper_length_mm	body_mass_g	
0	Torgersen	39.10	18.7	181.0	3750.0	M
1	Torgersen	39.50	17.4	186.0	3800.0	FEM
2	Torgersen	40.30	18.0	195.0	3250.0	FEM
3	Torgersen	44.45	17.3	197.0	4050.0	M
4	Torgersen	36.70	19.3	193.0	3450.0	FEM

In [81]:
```
y.head()
```

Out[81]:
```
0    Adelie
1    Adelie
2    Adelie
3    Adelie
4    Adelie
Name: species, dtype: object
```

4.1] Encoding categorical features

In [82]:
```
X = pd.get_dummies(X, drop_first=True)
```

In [83]:
```
X.head()
```

Out[83]:

	culmen_length_mm	culmen_depth_mm	flipper_length_mm	body_mass_g	island_Dream
0	39.10	18.7	181.0	3750.0	False
1	39.50	17.4	186.0	3800.0	False
2	40.30	18.0	195.0	3250.0	False
3	44.45	17.3	197.0	4050.0	False
4	36.70	19.3	193.0	3450.0	False

4.2] Feature Scaling

In [84]:
```
from sklearn.preprocessing import StandardScaler
# Scale the numerical features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

5] Training and Testing sets

In [85]:
```
from sklearn.model_selection import train_test_split
# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y,
    test_size=0.2,
    random_state=42
)
```

In [86]:
```
# Display the shape of training and testing data
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (275, 8)
X_test shape: (69, 8)
y_train shape: (275,)
y_test shape: (69,)
```

In [ ]:

In [ ]:

5. Stroke Prediction

1] Load Dataset

In [87]:
```
df = pd.read_csv("stroke prediction.csv")
df.head()
```

Out[87]:

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residenc
0	30669	Male	3.0	0	0	No	children	
1	30468	Male	58.0	1	0	Yes	Private	
2	16523	Female	8.0	0	0	No	Private	
3	56543	Female	70.0	0	0	Yes	Private	
4	46136	Male	14.0	0	0	No	Never_worked	

2] Data Cleaning

2.1] Handle Missing Values

In [88]:
```
# Check missing v)alues
df.isnull().sum()
```

Out[88]:
```
id                   0
gender               0
age                  0
hypertension         0
heart_disease        0
ever_married         0
work_type            0
Residence_type       0
avg_glucose_level    0
bmi                1462
smoking_status    13292
stroke               0
dtype: int64
```

In [89]:
```
# Fill missing values in 'bmi' with median
df["bmi"] = df["bmi"].fillna(df["bmi"].median())

# Fill missing values in 'smoking_status' with mode
df["smoking_status"] = df["smoking_status"].fillna(df["smoking_status"].mode()[0])
```

2.2] Remove Duplicate Records

In [90]:
```
df = df.drop_duplicates()
```

2.3] Correct data types

In [91]:
```
df.dtypes
```

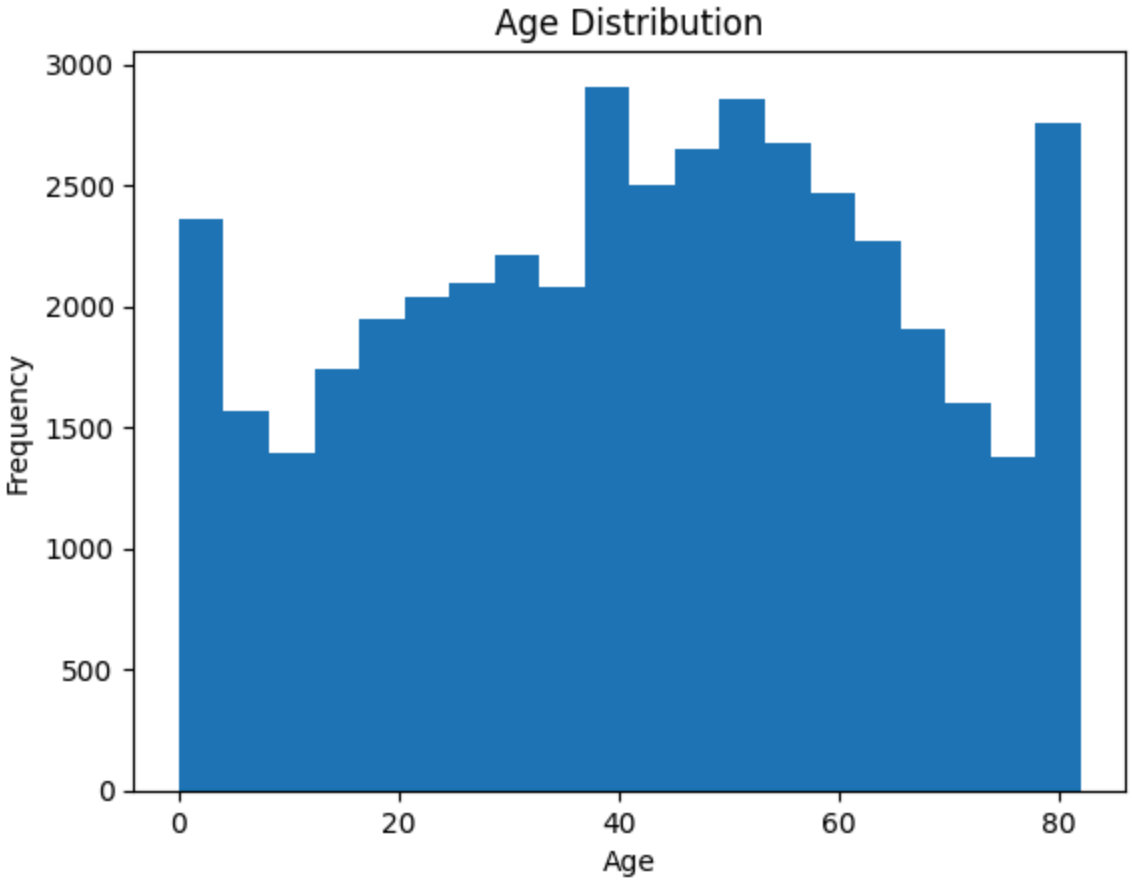
Out[91]:
```
id                   int64
gender              object
age                float64
hypertension         int64
heart_disease        int64
ever_married        object
work_type           object
Residence_type      object
avg_glucose_level  float64
bmi                float64
smoking_status      object
stroke               int64
dtype: object
```

No need of correction in data types

3] EDA

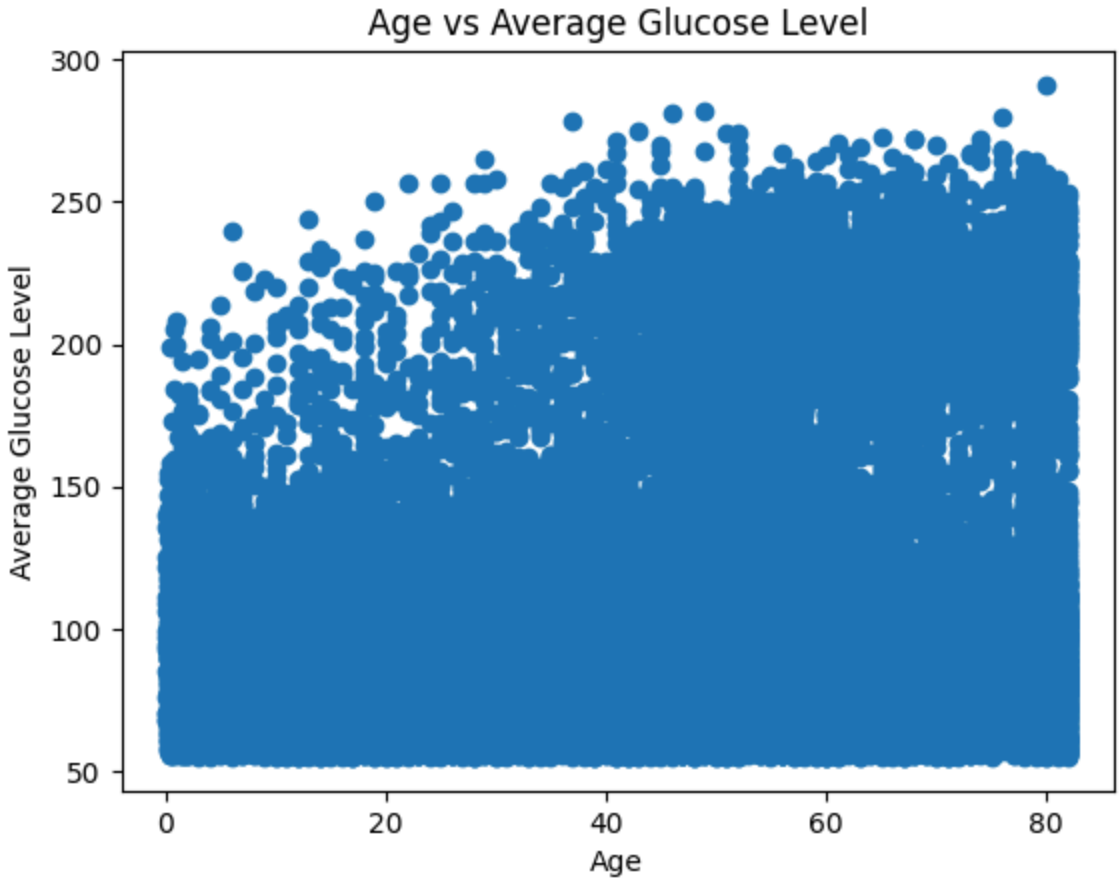
1} Histogram

In [92]:
```
plt.hist(df["age"], bins=20)
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```



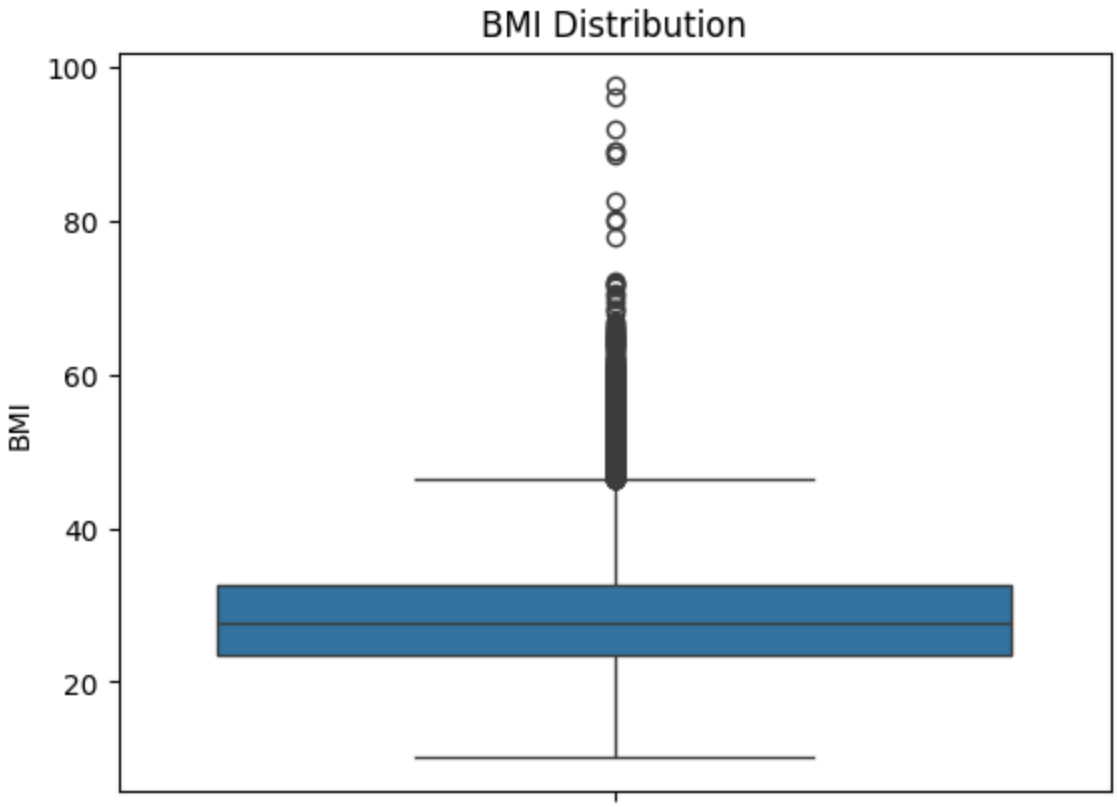
2} Scatter Plot

In [93]:
```
plt.scatter(df["age"], df["avg_glucose_level"])
plt.title("Age vs Average Glucose Level")
plt.xlabel("Age")
plt.ylabel("Average Glucose Level")
plt.show()
```



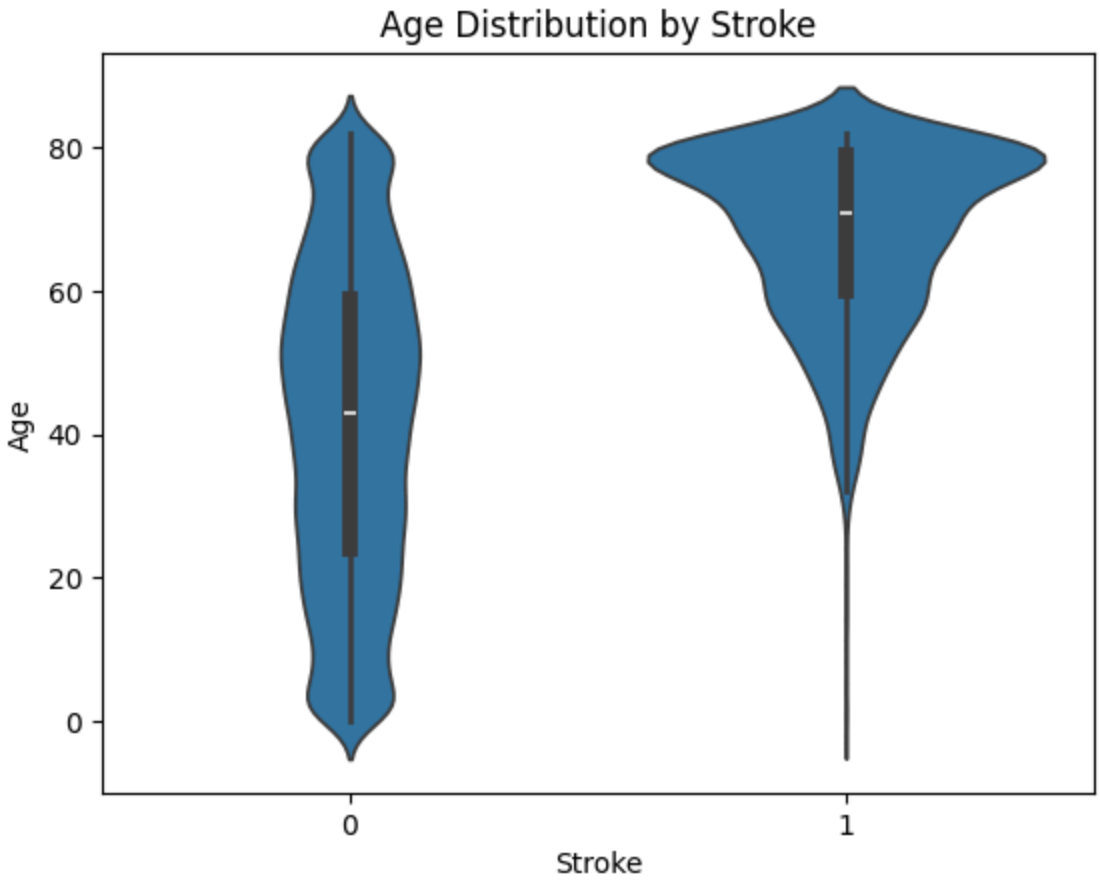
3} Box Plot

In [94]:
```
sns.boxplot(y=df["bmi"])
plt.title("BMI Distribution")
plt.ylabel("BMI")
plt.show()
```



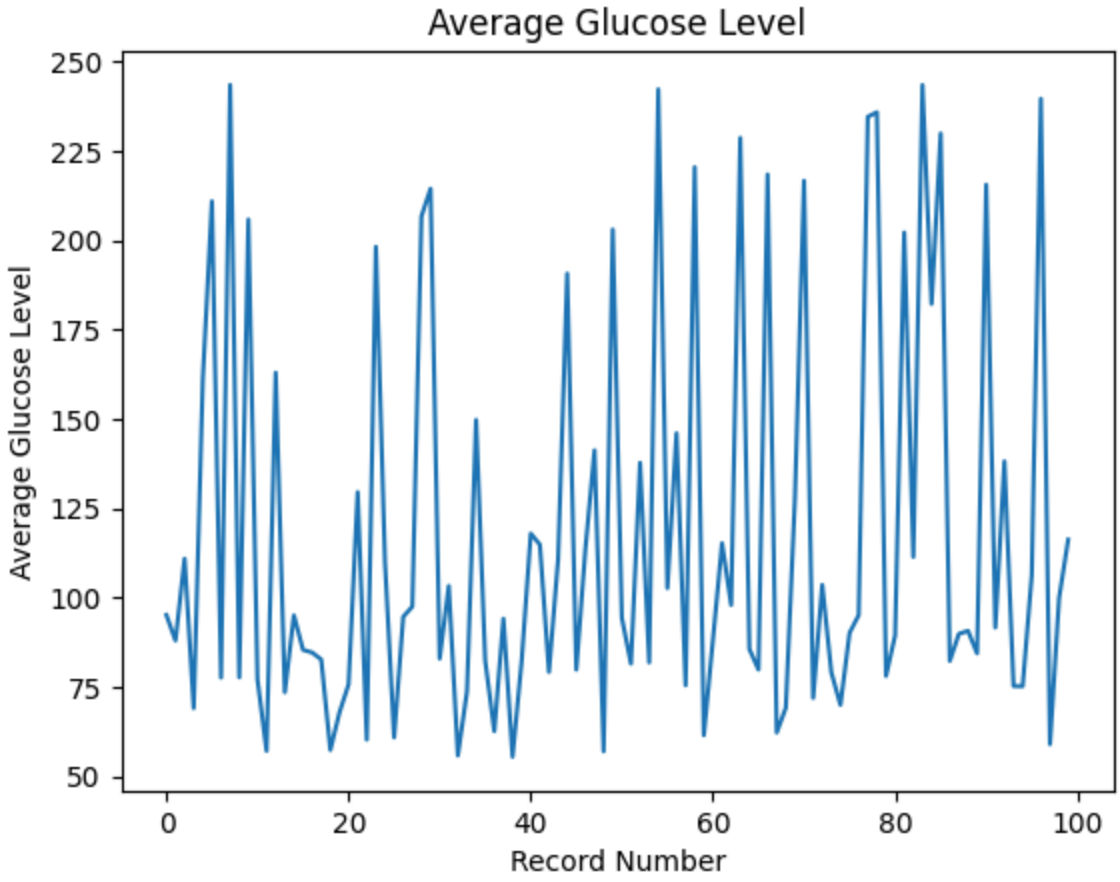
4} Violin Plot

In [95]:
```
sns.violinplot(x="stroke", y="age", data=df)
plt.title("Age Distribution by Stroke")
plt.xlabel("Stroke")
plt.ylabel("Age")
plt.show()
```



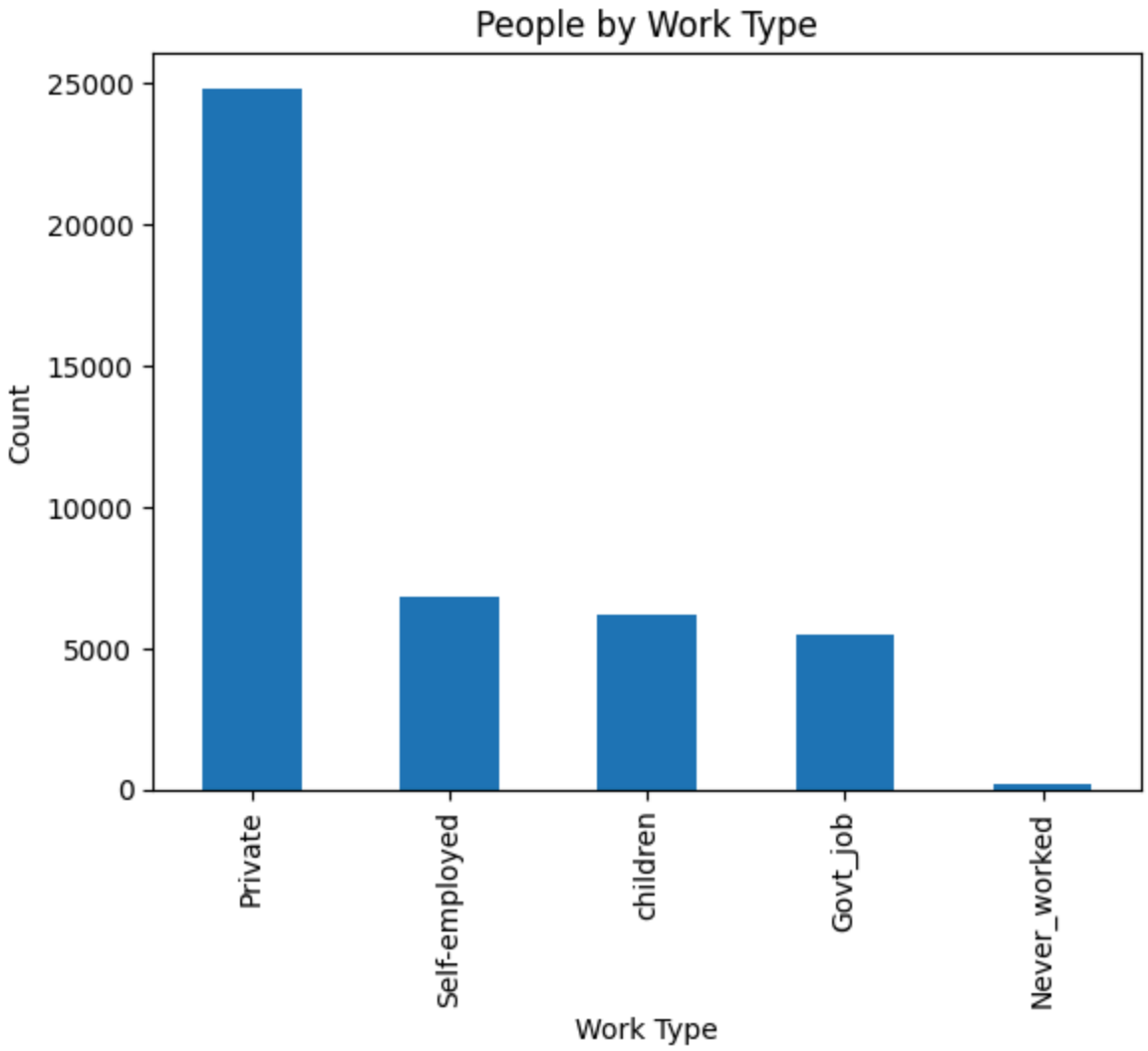
5} Line Plot

In [96]:
```
df["avg_glucose_level"].head(100).plot(kind="line")
plt.title("Average Glucose Level")
plt.xlabel("Record Number")
plt.ylabel("Average Glucose Level")
plt.show()
```



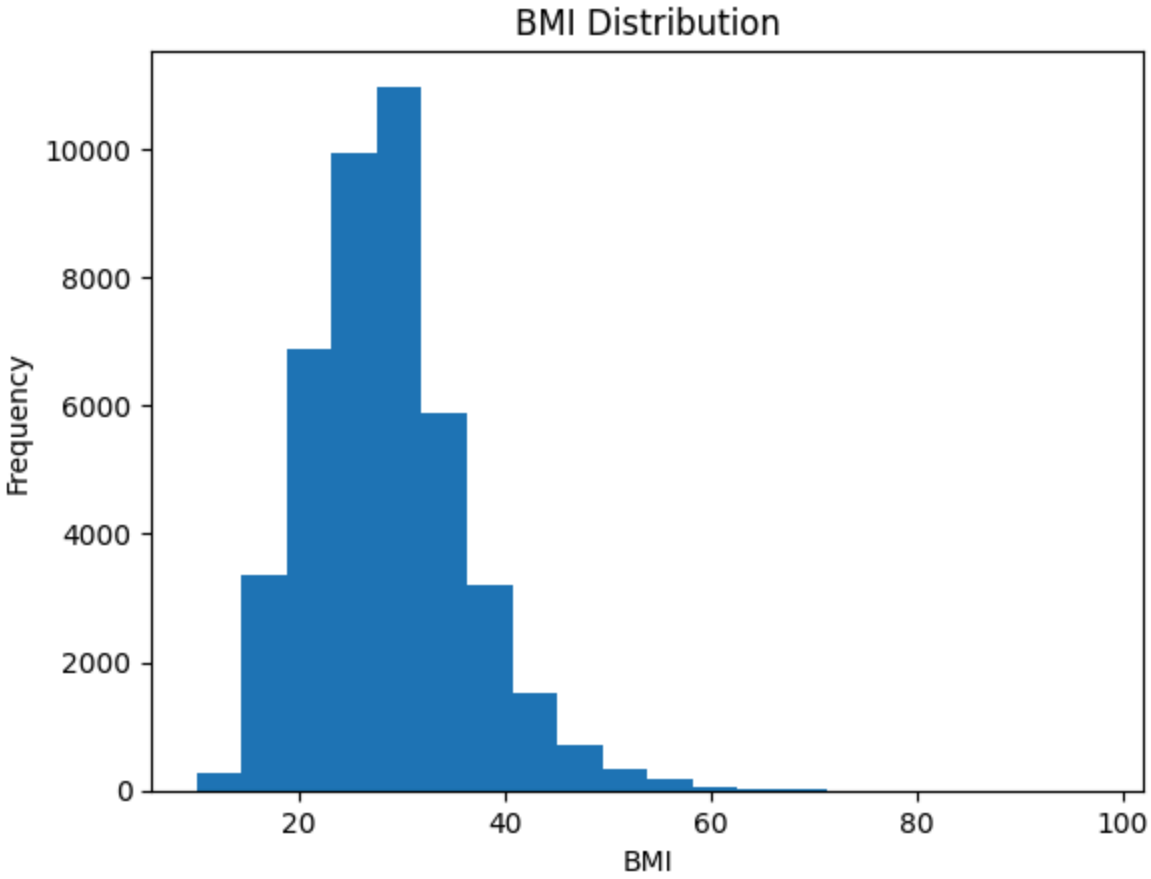
6} Bar Plot

In [97]:
```
df["work_type"].value_counts().plot(kind="bar")
plt.title("People by Work Type")
plt.xlabel("Work Type")
plt.ylabel("Count")
plt.show()
```



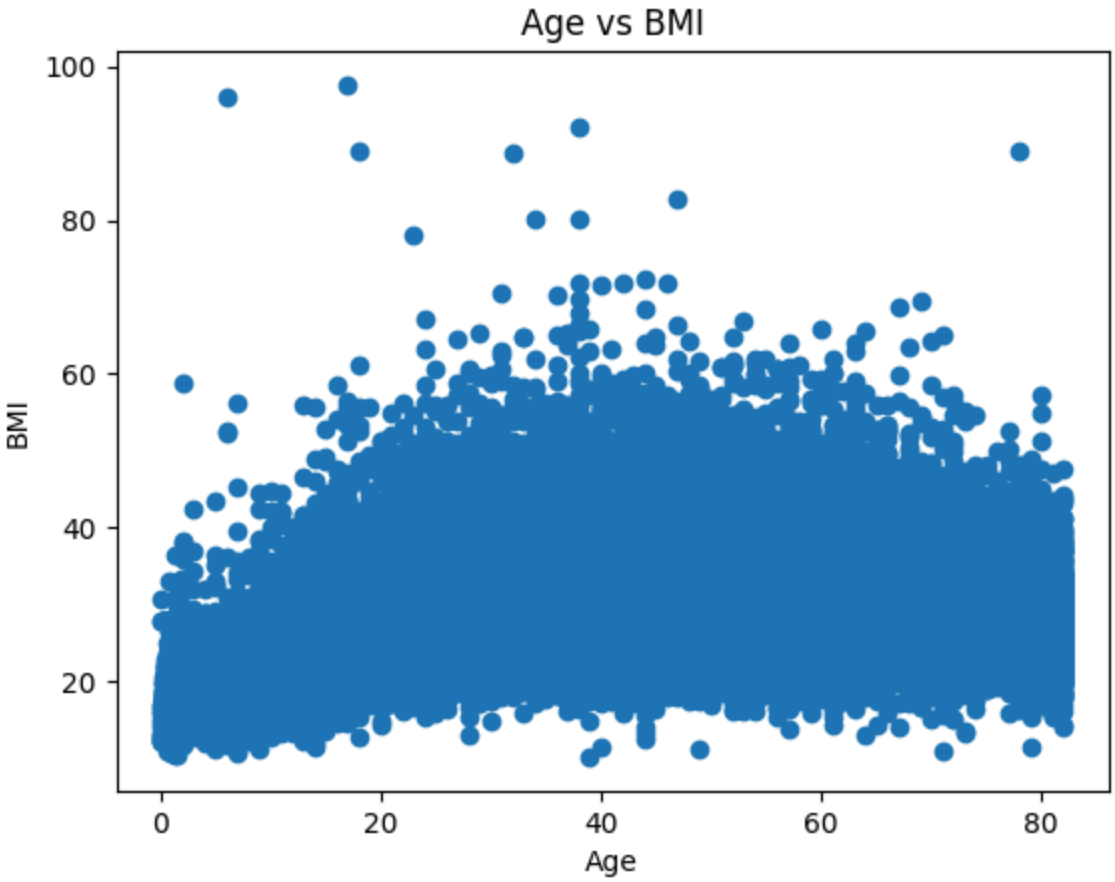
7} Histogram

In [98]:
```
plt.hist(df["bmi"], bins=20)
plt.title("BMI Distribution")
plt.xlabel("BMI")
plt.ylabel("Frequency")
plt.show()
```



8} Scatter Plot

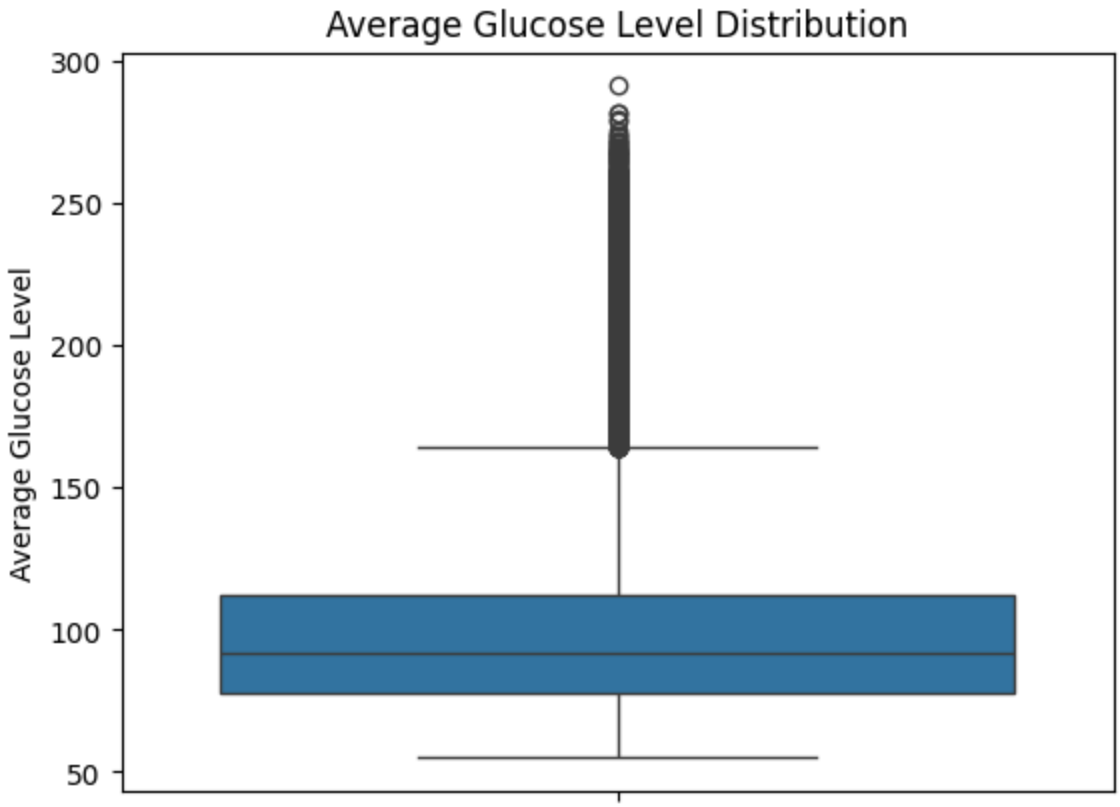
In [99]:
```
plt.scatter(df["age"], df["bmi"])
plt.title("Age vs BMI")
plt.xlabel("Age")
plt.ylabel("BMI")
plt.show()
```



9} Box Plot

In [100…

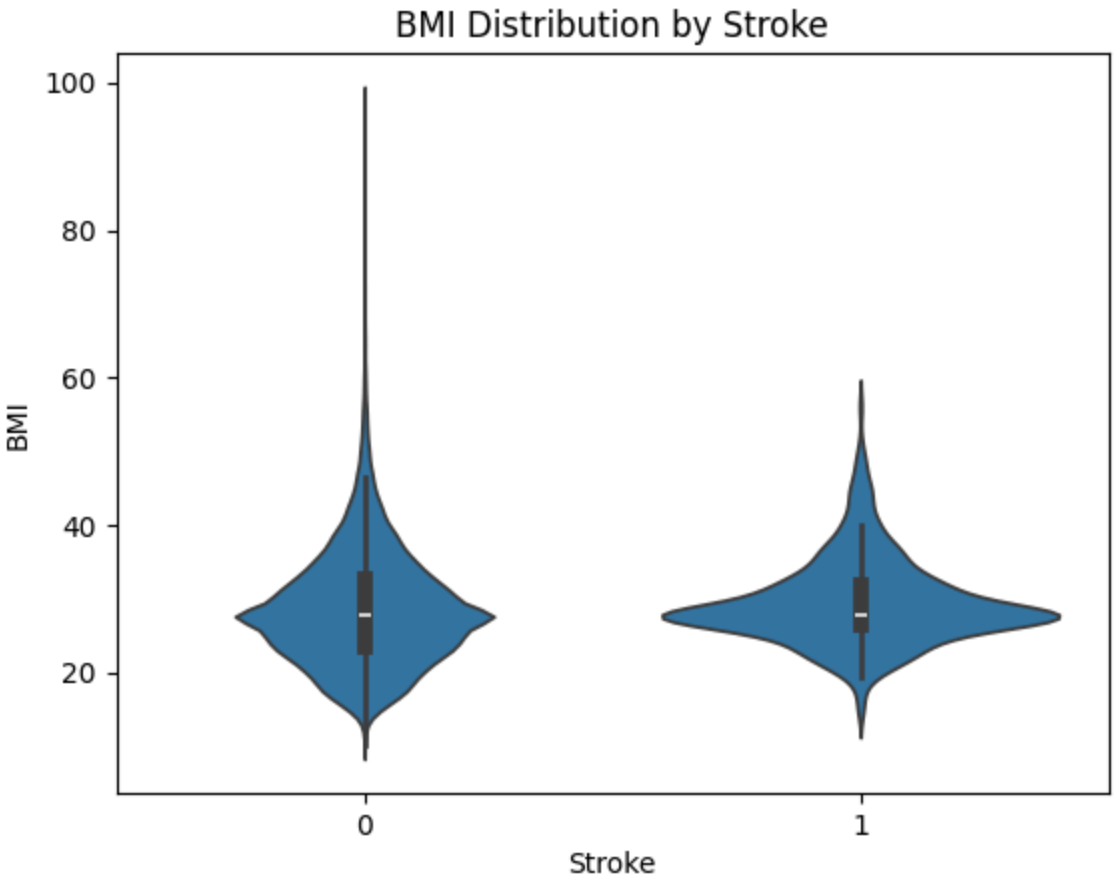
```
sns.boxplot(y=df["avg_glucose_level"])
plt.title("Average Glucose Level Distribution")
plt.ylabel("Average Glucose Level")
plt.show()
```



10} Violin Plot

In [101…

```
sns.violinplot(x="stroke", y="bmi", data=df)
plt.title("BMI Distribution by Stroke")
plt.xlabel("Stroke")
plt.ylabel("BMI")
plt.show()
```



4] Feature Engineering

X and y separation :

In [102…
```
X = df.drop("stroke", axis=1)
y = df["stroke"]
```

4.1] Encoding Categorical Features

In [103…
```
X = pd.get_dummies(X, drop_first=True)
X.head()
```

Out[103…

	id	age	hypertension	heart_disease	avg_glucose_level	bmi	gender_Male	gender_
0	30669	3.0	0	0	95.12	18.0	True	
1	30468	58.0	1	0	87.96	39.2	True	
2	16523	8.0	0	0	110.89	17.6	False	
3	56543	70.0	0	0	69.04	35.9	False	
4	46136	14.0	0	0	161.28	19.1	True	

4.2] Feature Scaling

In [104…
```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled[:2]
```

Out[104…
```
array([[-0.26846865, -1.74151677, -0.32129564, -0.22334159, -0.21717647,
        -1.38416136,  1.20360176, -0.01592233, -1.34420288, -0.06399252,
        -1.15654934, -0.43077326,  2.45968138, -1.00258398,  0.69206736,
        -0.42205601],
       [-0.27800742,  0.700823  ,  3.11239826, -0.22334159, -0.38325839,
         1.39082376,  1.20360176, -0.01592233,  0.74393532, -0.06399252,
         0.86464102, -0.43077326, -0.40655672,  0.99742268,  0.69206736,
        -0.42205601]])
```

5] Training and Testing sets

In [105…
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y,
    test_size=0.2,
    random_state=42
)
```

In [106…
```
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (34720, 16)
X_test shape: (8680, 16)
y_train shape: (34720,)
y_test shape: (8680,)
```

In [ ]:

In [ ]:

6. Titanic Dataset

1] Load Dataset

In [107…
```
df = pd.read_csv("titanic.csv")
# Display the first 5 rows
df.head()
```

Out[107…

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500

2] Data Cleaning

2.1] Handle missing values

In [108…

```
df.isnull().sum()
```

Out[108…

```
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
```

In [109…

```
df["Age"] = df["Age"].fillna(df["Age"].median())

df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])
```

```
df = df.drop("Cabin", axis=1)
```

2.2] Remove Duplicate Records

In [110…
```
df = df.drop_duplicates()
```

2.3] Correct data types

In [111…
```
df.dtypes
```

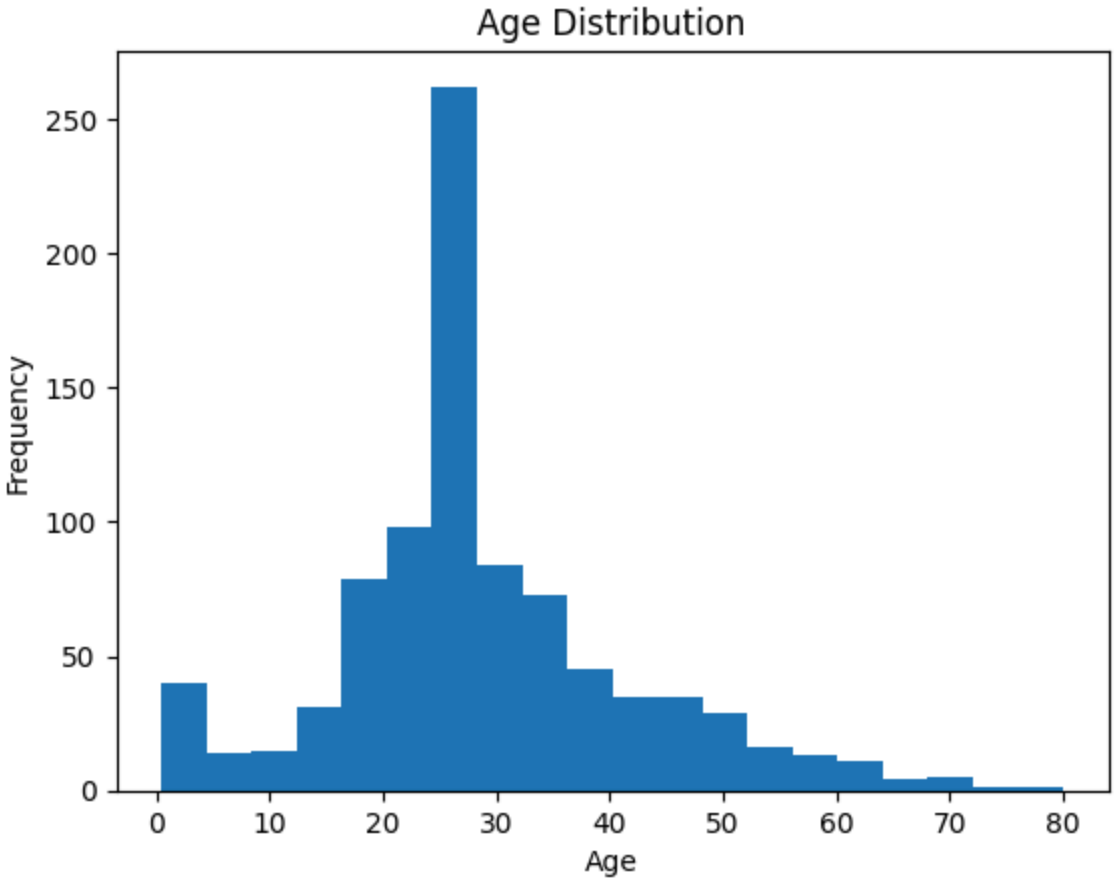
Out[111…
```
PassengerId      int64
Survived         int64
Pclass           int64
Name            object
Sex             object
Age            float64
SibSp            int64
Parch            int64
Ticket          object
Fare           float64
Embarked        object
dtype: object
```

In Data types correction not required .

3] EDA

1} Histogram

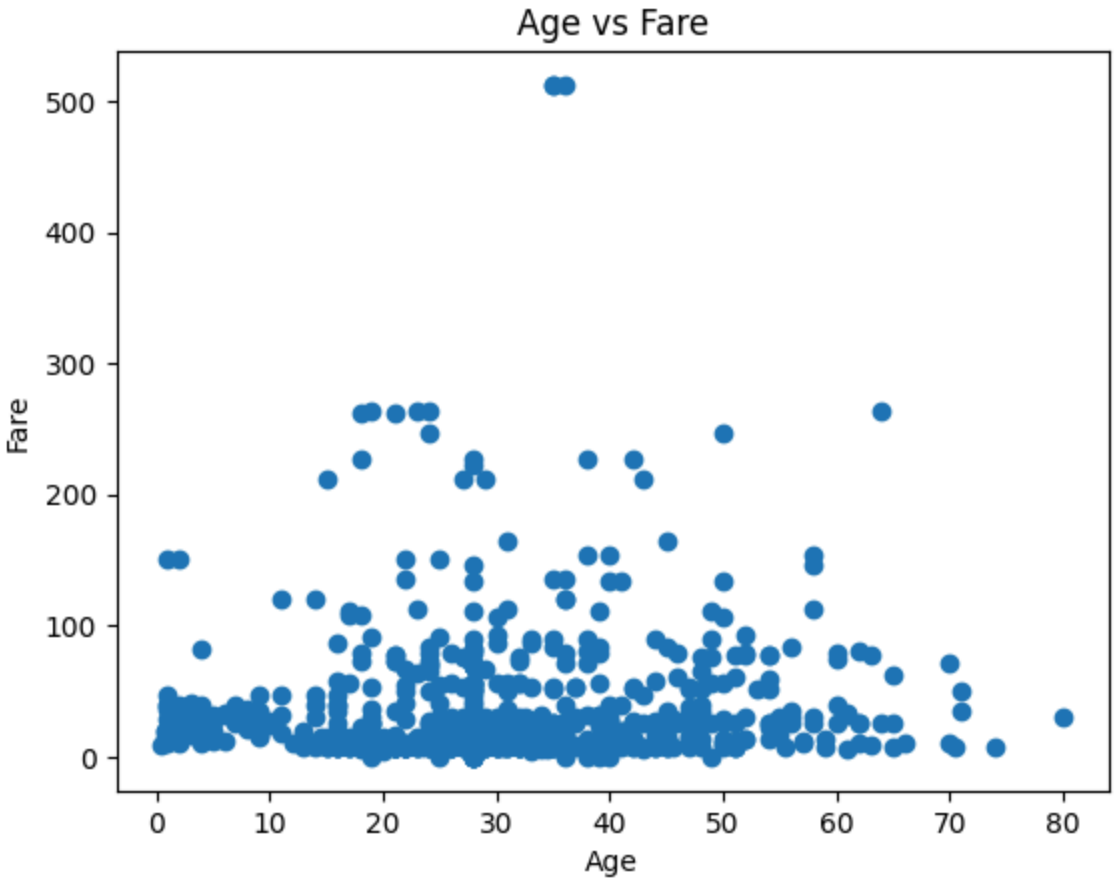
In [112…
```
plt.hist(df["Age"], bins=20)
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```



2} Scatter Plot

In [113…

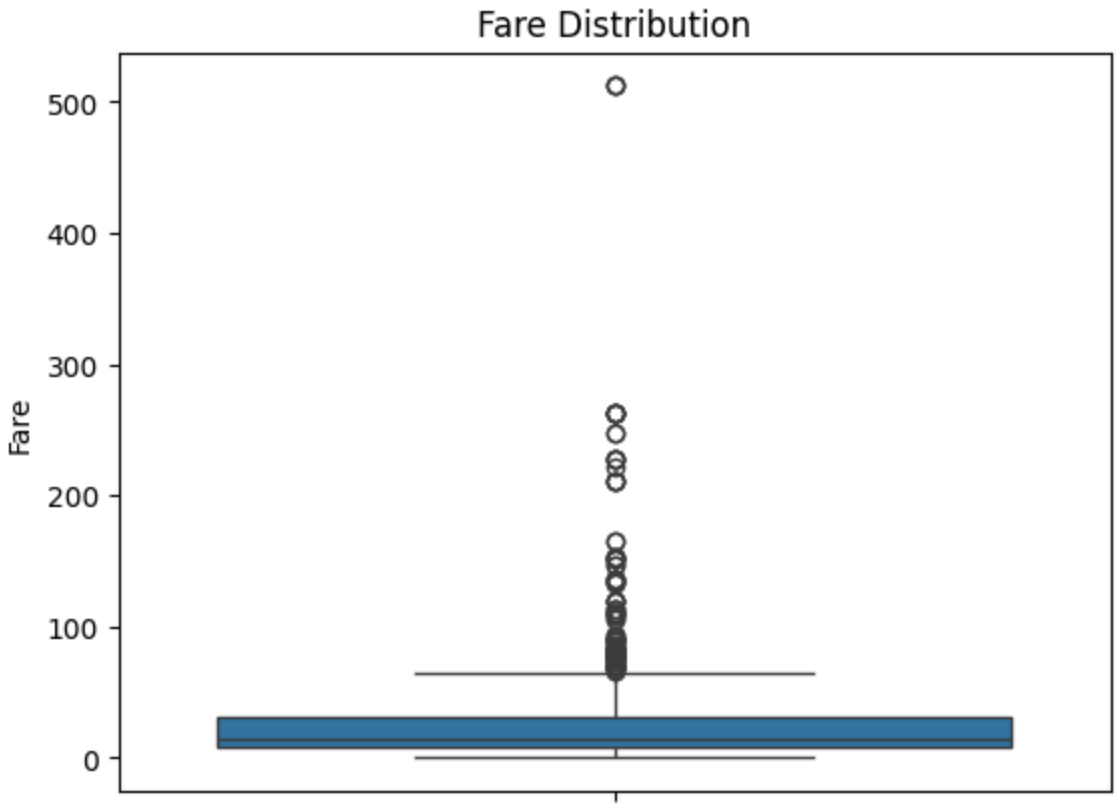
```
plt.scatter(df["Age"], df["Fare"])
plt.title("Age vs Fare")
plt.xlabel("Age")
plt.ylabel("Fare")
plt.show()
```



3} Box Plot

In [114…

```
sns.boxplot(y=df["Fare"])
plt.title("Fare Distribution")
plt.ylabel("Fare")
plt.show()
```



4} Violin Plot

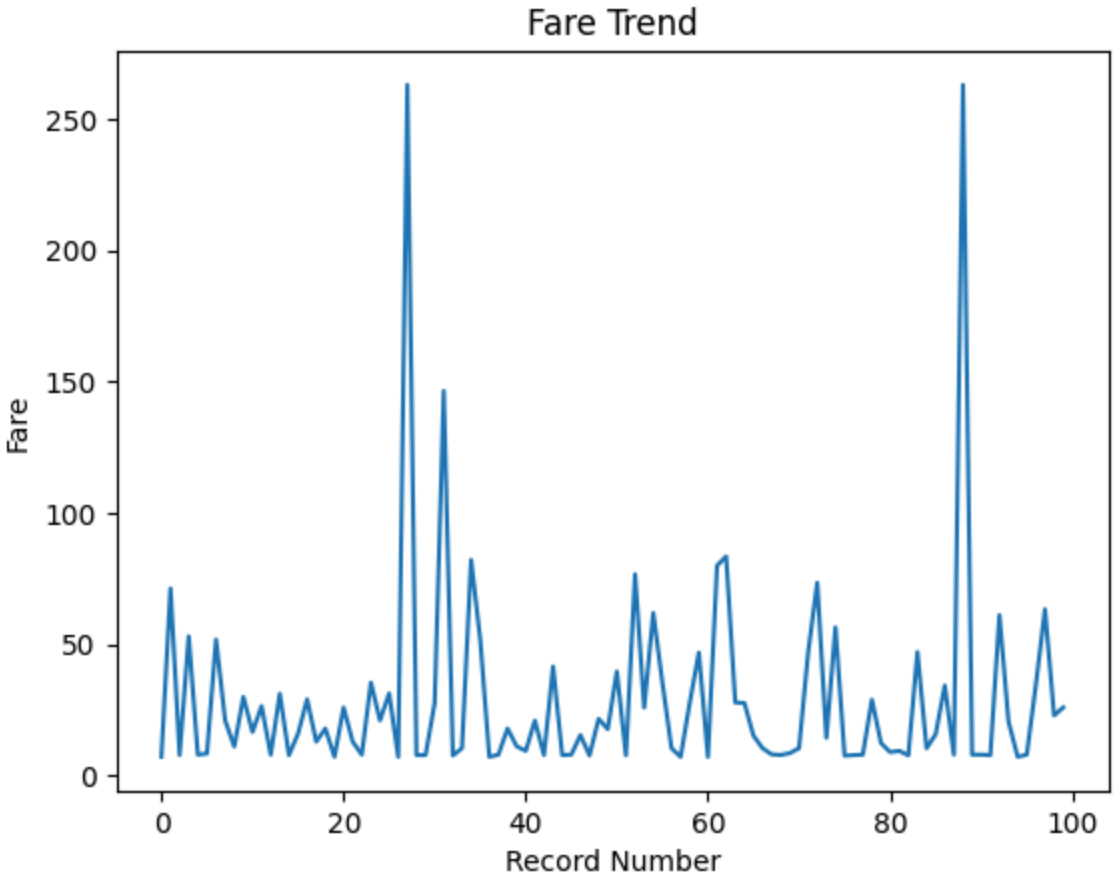
In [115…

```
sns.violinplot(x="Survived", y="Age", data=df)
plt.title("Age Distribution by Survival")
plt.xlabel("Survived")
plt.ylabel("Age")
plt.show()
```

5} Line Plot

In [116…

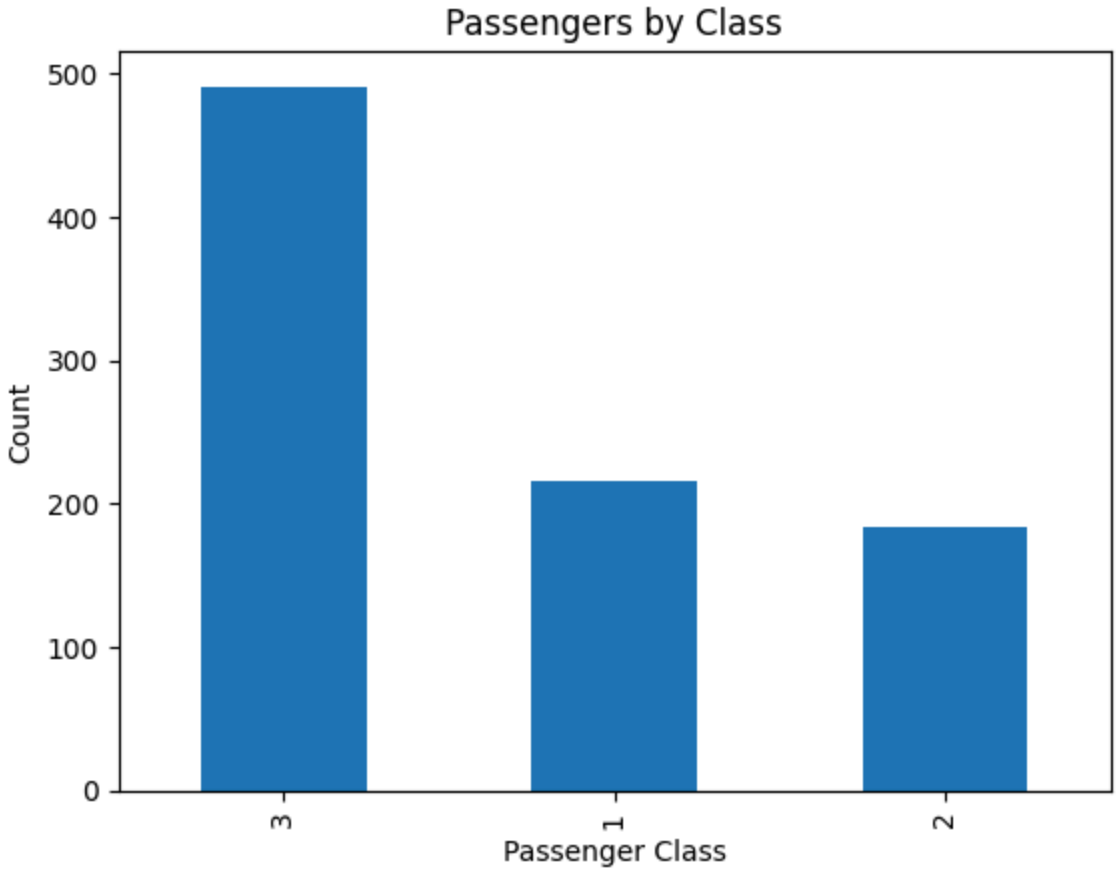
```
df["Fare"].head(100).plot(kind="line")
plt.title("Fare Trend")
plt.xlabel("Record Number")
plt.ylabel("Fare")
plt.show()
```



6} Bar Plot

In [117…

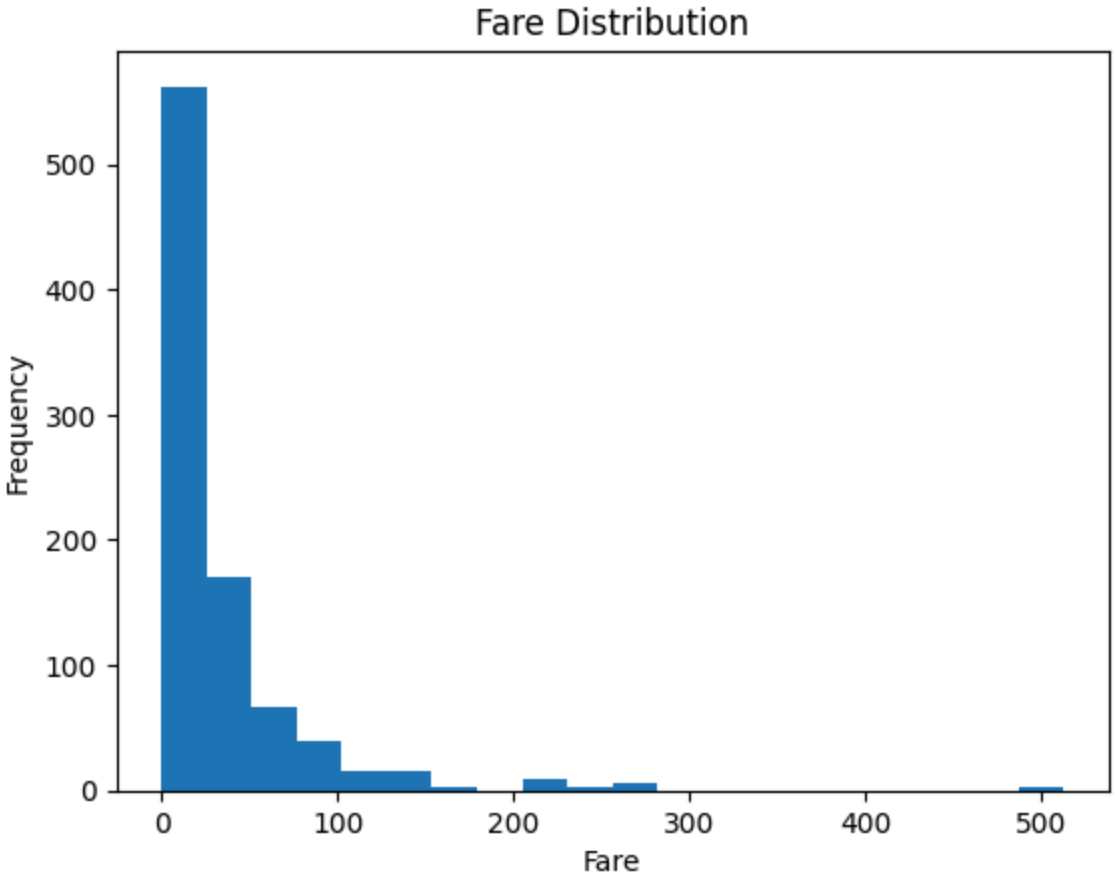
```
df["Pclass"].value_counts().plot(kind="bar")
plt.title("Passengers by Class")
plt.xlabel("Passenger Class")
plt.ylabel("Count")
plt.show()
```



7} Histogram

In [118…

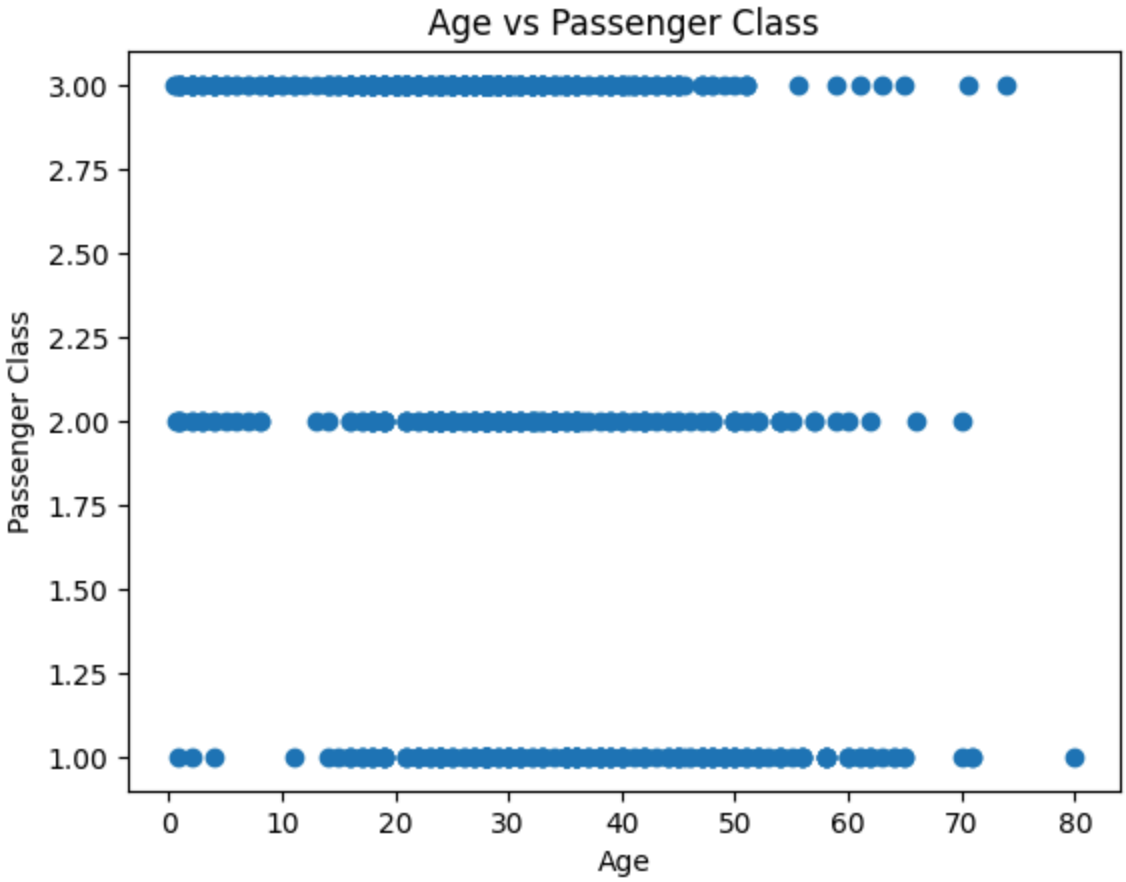
```
plt.hist(df["Fare"], bins=20)
plt.title("Fare Distribution")
plt.xlabel("Fare")
plt.ylabel("Frequency")
plt.show()
```



8} Scatter Plot

In [119…

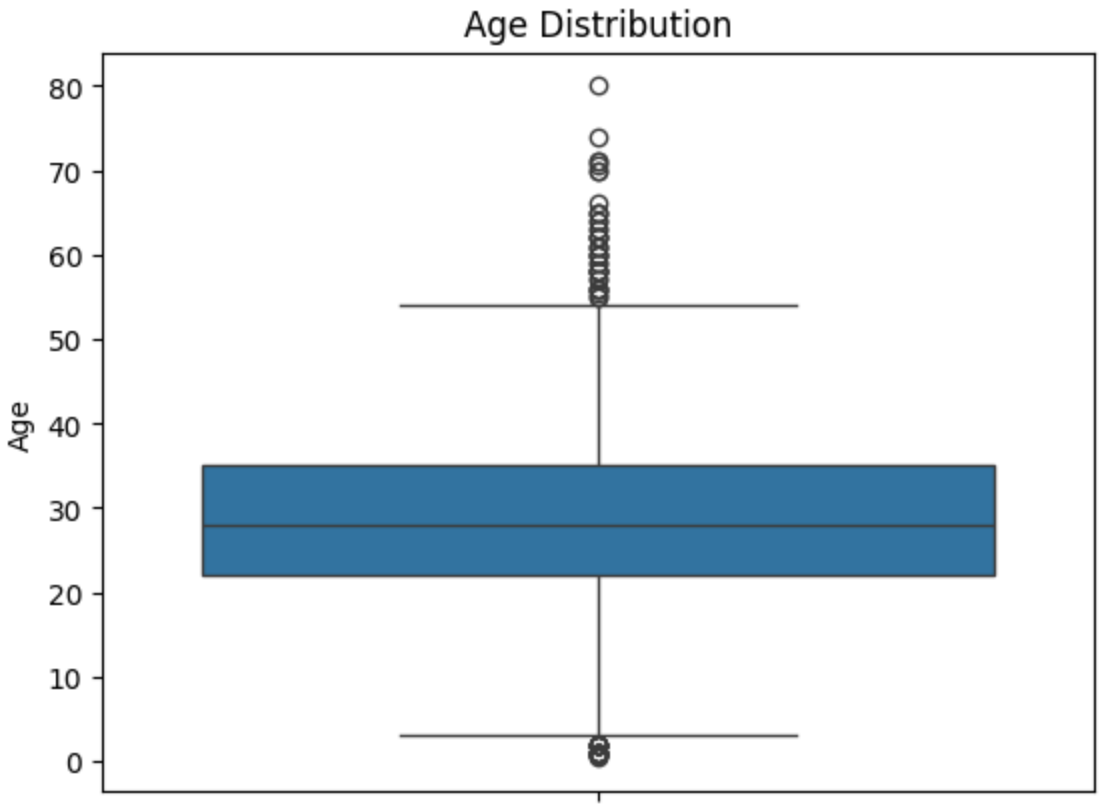
```
plt.scatter(df["Age"], df["Pclass"])
plt.title("Age vs Passenger Class")
plt.xlabel("Age")
plt.ylabel("Passenger Class")
plt.show()
```



9} Box Plot

In [120…

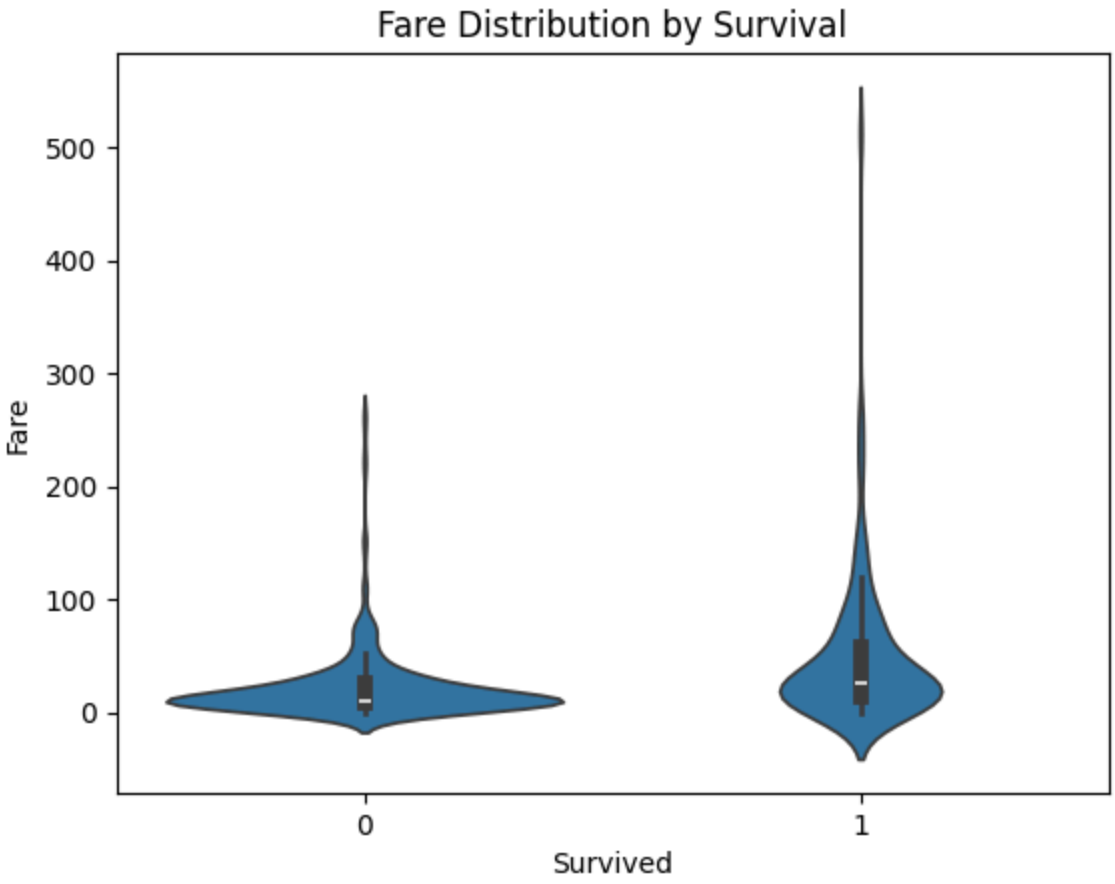
```
# Distribution of age
sns.boxplot(y=df["Age"])
plt.title("Age Distribution")
plt.ylabel("Age")
plt.show()
```



10} Violin Plot

In [121…

```
sns.violinplot(x="Survived", y="Fare", data=df)
plt.title("Fare Distribution by Survival")
plt.xlabel("Survived")
plt.ylabel("Fare")
plt.show()
```



4] Feature Engineering

X and y separation:

In [122…
```
X = df.drop("Survived", axis=1)
y = df["Survived"]
```

In [123…
```
# Display X and y
print("X:")
display(X.head())
print("y:")
display(y.head())
```

X:

	PassengerId	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Embarked
0	1	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	S
1	2	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C
2	3	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	S
3	4	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	S
4	5	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	S

```
y:
0    0
1    1
2    1
3    1
4    0
Name: Survived, dtype: int64
```

4.1] Encoding Categorical Features

In [124…

```
# Select categorical columns for encoding
categorical_columns = ["Sex", "Embarked"]

# Apply one-hot encoding
X = pd.get_dummies(
    X,
    columns=categorical_columns,
    drop_first=True
)

# Display encoded data
X.head()
```

Out[124...

	PassengerId	Pclass	Name	Age	SibSp	Parch	Ticket	Fare	Sex_male	Emba
0	1	3	Braund, Mr. Owen Harris	22.0	1	0	A/5 21171	7.2500	True	
1	2	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	38.0	1	0	PC 17599	71.2833	False	
2	3	3	Heikkinen, Miss. Laina	26.0	0	0	STON/O2. 3101282	7.9250	False	
3	4	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	35.0	1	0	113803	53.1000	False	
4	5	3	Allen, Mr. William Henry	35.0	0	0	373450	8.0500	True	

4.2] Feature Scaling

In [125...

```
X = X.drop(["Name", "Ticket"], axis=1)

X.columns
```

Out[125...

```
Index(['PassengerId', 'Pclass', 'Age', 'SibSp', 'Parch', 'Fare', 'Sex_male',
       'Embarked_Q', 'Embarked_S'],
      dtype='object')
```

In [126...

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

X_scaled[:2]
```

Out[126...

```
array([[-1.73010796,  0.82737724, -0.56573646,  0.43279337, -0.47367361,
        -0.50244517,  0.73769513, -0.30756234,  0.61583843],
       [-1.72622007, -1.56610693,  0.66386103,  0.43279337, -0.47367361,
         0.78684529, -1.35557354, -0.30756234, -1.62380254]])
```

5] Training and Testing Sets

In [127…
```
# Import train_test_split
from sklearn.model_selection import train_test_split

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
X_scaled,
y,
test_size=0.2,
random_state=42
)
```

In [128…
```
# Display the shape of training and testing data
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (712, 9)
X_test shape: (179, 9)
y_train shape: (712,)
y_test shape: (179,)
```

In [ ]:

In [ ]:

7. Fabric data

1] Load dataset

In [129…
```
#using file name
df = pd.read_excel('Fabric data.xlsx')
df.head()
```

Out[129…

	Fabric_length
0	151.2
1	160.3
2	147.5
3	149.2
4	159.2

2] Data Cleaning

2.1] Handle missing values

In [130…
```
df.isnull().sum()
```

Out[130…
```
Fabric_length    0
dtype: int64
```

2.2] Remove Duplicate Records

In [131…
```
df = df.drop_duplicates()
```

2.3] Correct data types

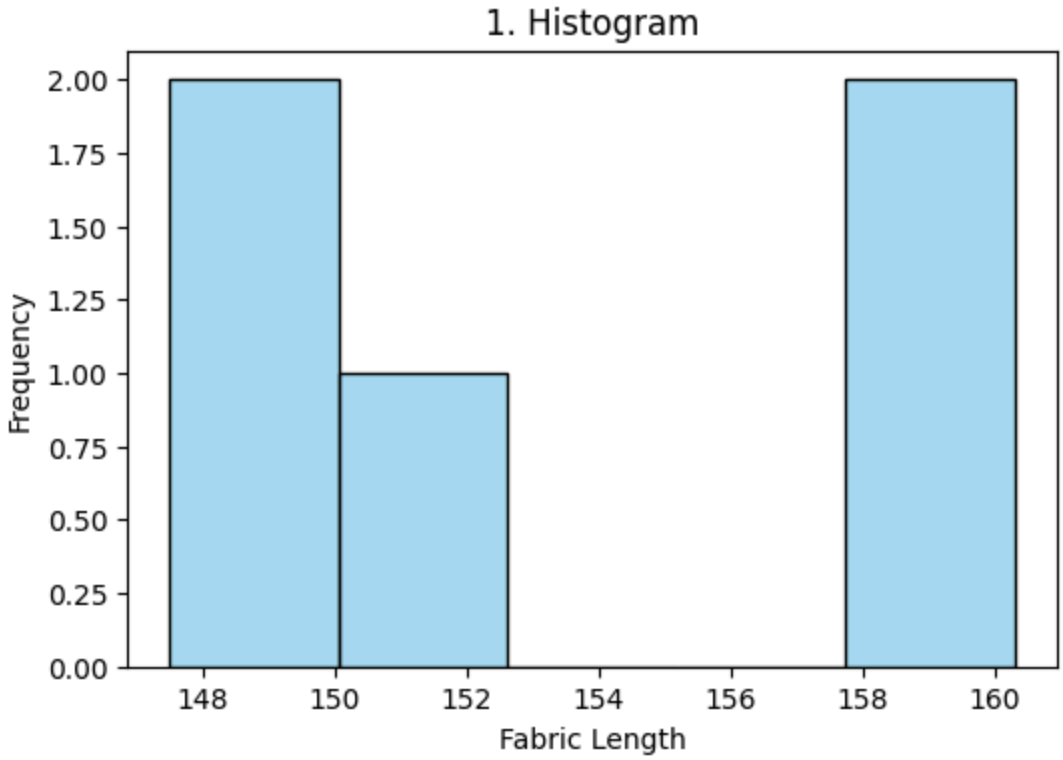
In [132…
```
df.dtypes
```

Out[132…
```
Fabric_length    float64
dtype: object
```

3] EDA

1: Histogram (Frequency Distribution)

In [133…
```
# Sample Data
df = pd.DataFrame({"Fabric_length": [151.2, 160.3, 147.5, 149.2, 159.2]})

# Plot
plt.figure(figsize=(6, 4))
sns.histplot(df["Fabric_length"], bins=5, color="skyblue", kde=False)
plt.title("1. Histogram")
plt.xlabel("Fabric Length")
plt.ylabel("Frequency")
plt.show()
```

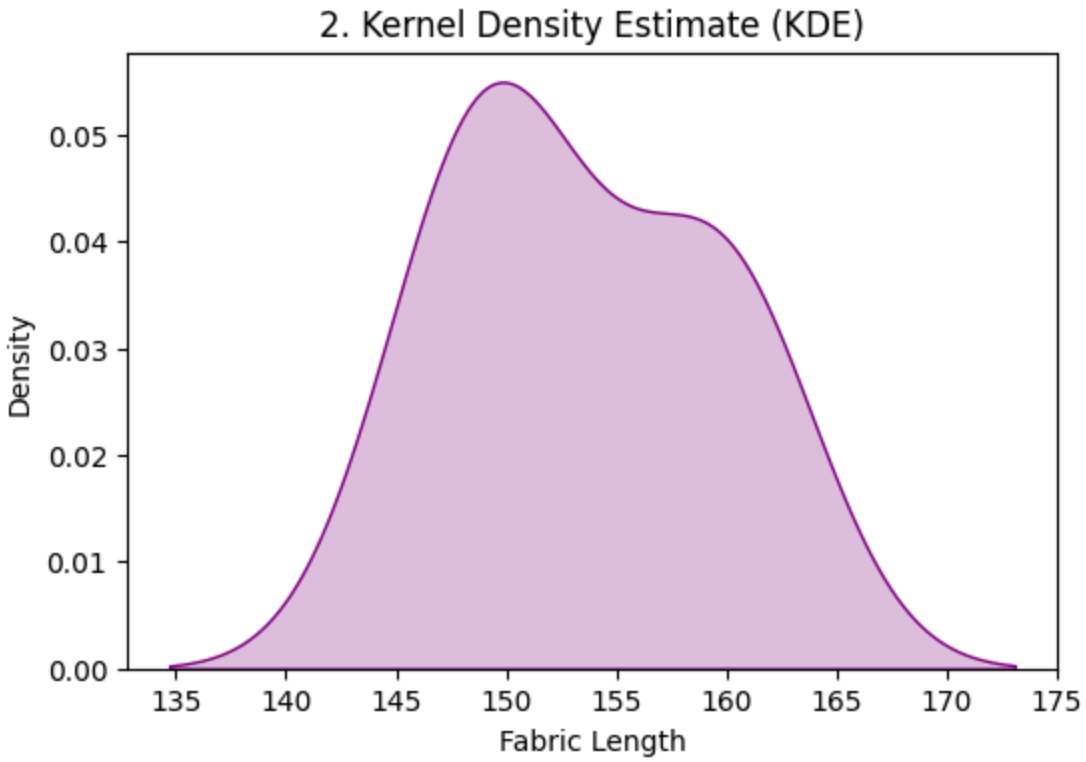


In [ ]:

2. Kernel Density Estimate (KDE) Plot

In [134…

```
# Sample Data
df = pd.DataFrame({"Fabric_length": [151.2, 160.3, 147.5, 149.2, 159.2]})

# Plot
plt.figure(figsize=(6, 4))
sns.kdeplot(df["Fabric_length"], fill=True, color="purple")
plt.title("2. Kernel Density Estimate (KDE)")
plt.xlabel("Fabric Length")
plt.ylabel("Density")
plt.show()
```

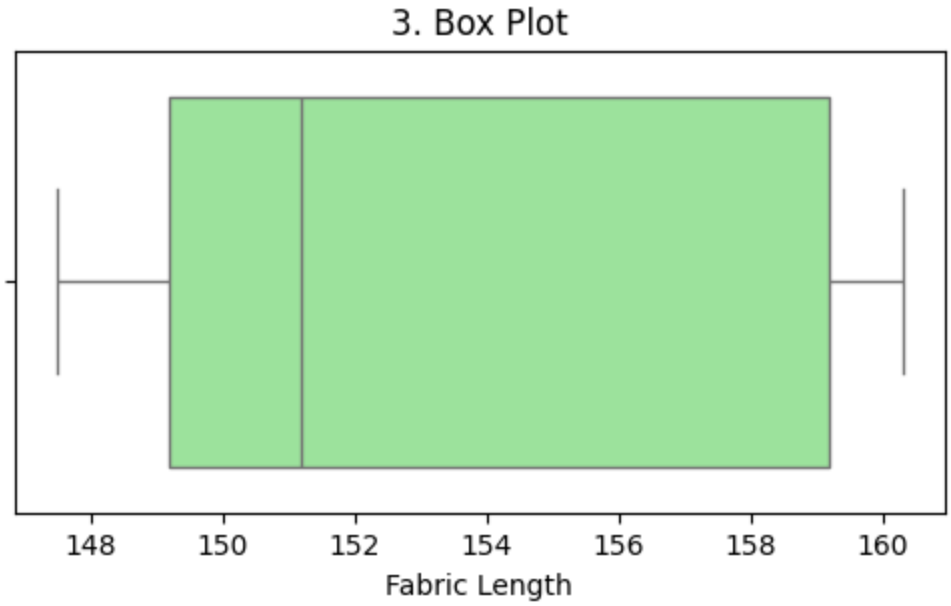


In [ ]:

3. Box Plot (Quartiles & Outliers)

In [135…

```
# Sample Data
df = pd.DataFrame({"Fabric_length": [151.2, 160.3, 147.5, 149.2, 159.2]})

# Plot
plt.figure(figsize=(6, 3))
sns.boxplot(x=df["Fabric_length"], color="lightgreen")
plt.title("3. Box Plot")
plt.xlabel("Fabric Length")
plt.show()
```

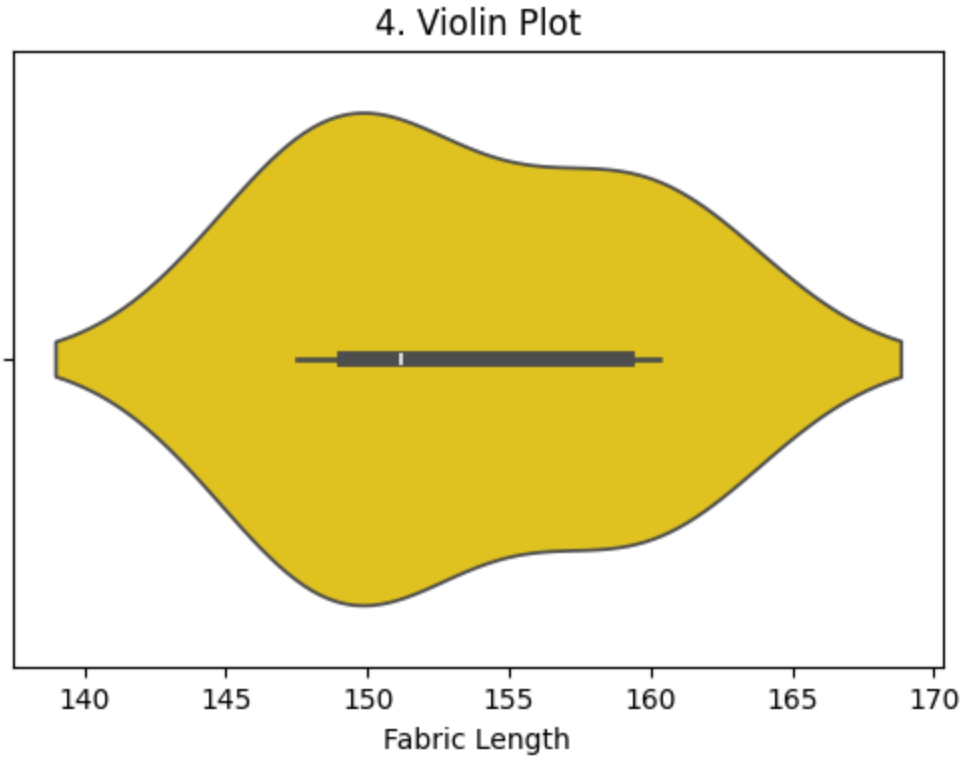


In [ ]:

4. Violin Plot (Distribution Spread)

In [136…

```
# Sample Data
df = pd.DataFrame({"Fabric_length": [151.2, 160.3, 147.5, 149.2, 159.2]})

# Plot
plt.figure(figsize=(6, 4))
sns.violinplot(x=df["Fabric_length"], color="gold")
plt.title("4. Violin Plot")
plt.xlabel("Fabric Length")
plt.show()
```

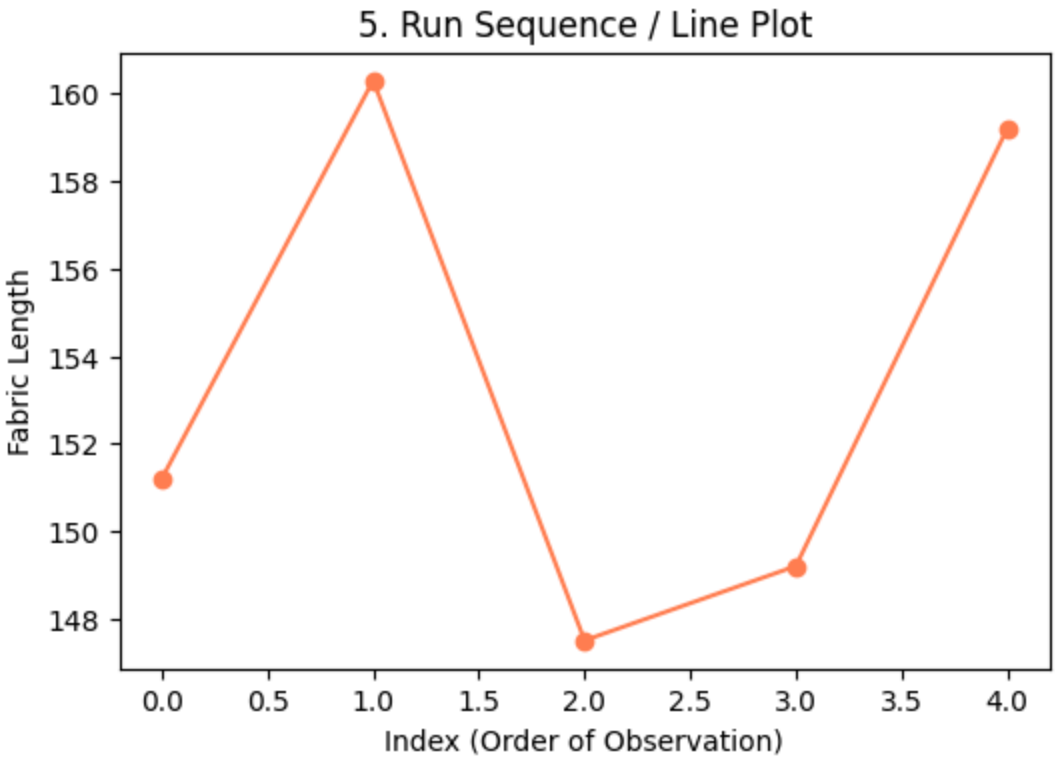


In [ ]:

5. Run Sequence Plot (Line Plot)

In [137…

```
# Sample Data
df = pd.DataFrame({"Fabric_length": [151.2, 160.3, 147.5, 149.2, 159.2]})

# Plot
plt.figure(figsize=(6, 4))
plt.plot(df.index, df["Fabric_length"], marker="o", color="coral", linestyle="-")
plt.title("5. Run Sequence / Line Plot")
plt.xlabel("Index (Order of Observation)")
plt.ylabel("Fabric Length")
plt.show()
```

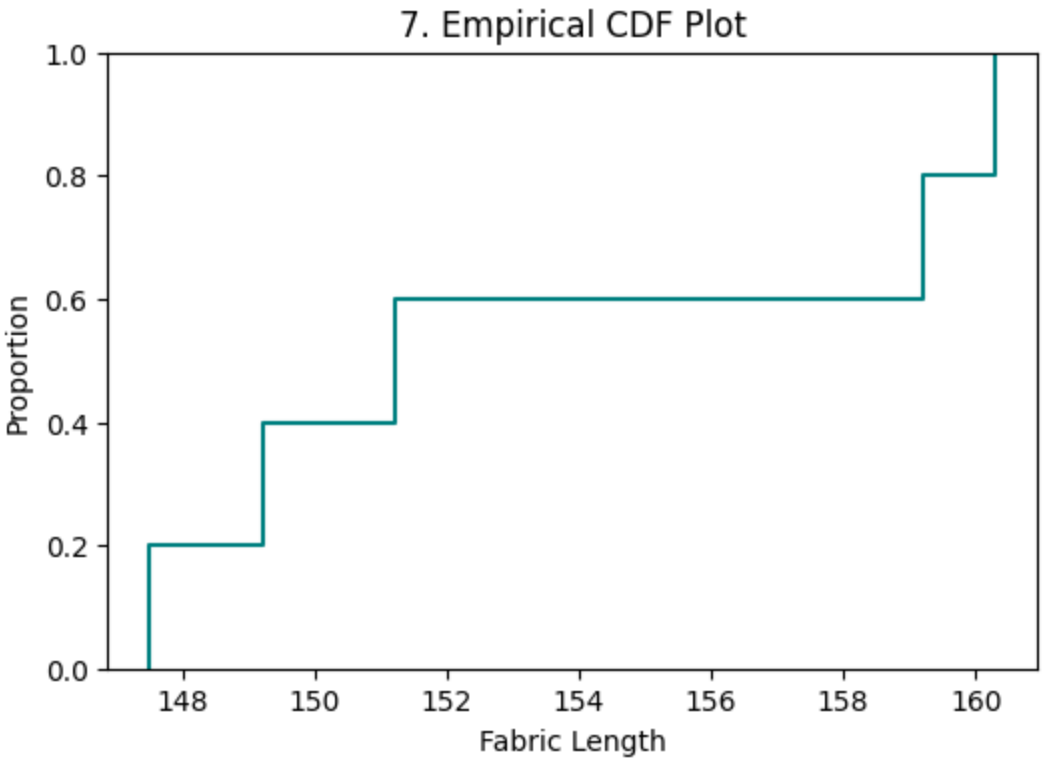


In [ ]:

6. Empirical Cumulative Distribution Function (ECDF)

In [138…

```
# Sample Data
df = pd.DataFrame({"Fabric_length": [151.2, 160.3, 147.5, 149.2, 159.2]})

# Plot
plt.figure(figsize=(6, 4))
sns.ecdfplot(data=df, x="Fabric_length", color="teal")
plt.title("7. Empirical CDF Plot")
plt.xlabel("Fabric Length")
plt.ylabel("Proportion")
plt.show()
```

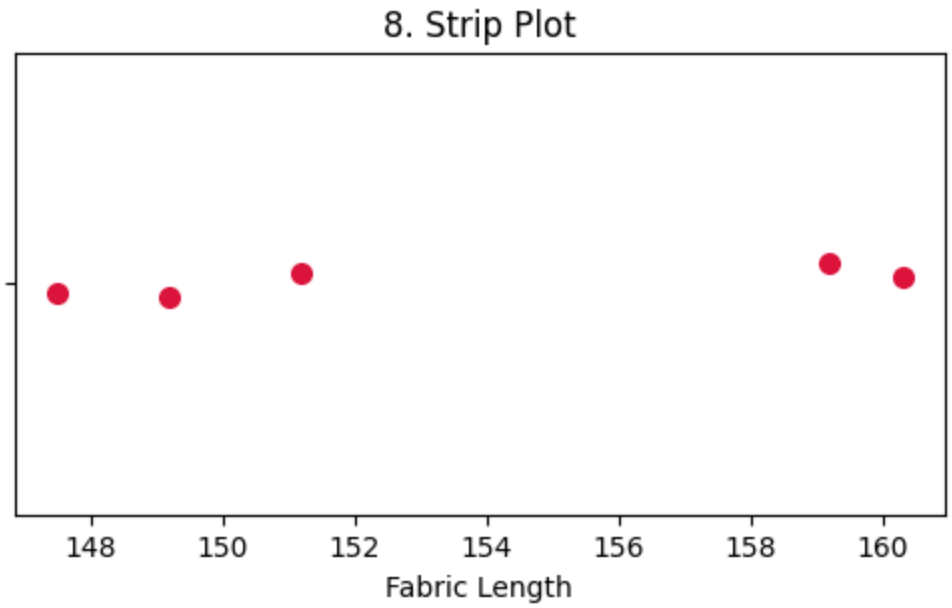


In [ ]:

7.Strip Plot (Individual Data Points)

In [139…

```
# Sample Data
df = pd.DataFrame({"Fabric_length": [151.2, 160.3, 147.5, 149.2, 159.2]})

# Plot
plt.figure(figsize=(6, 3))
sns.stripplot(x=df["Fabric_length"], color="crimson", jitter=0.05, size=8)
plt.title("8. Strip Plot")
plt.xlabel("Fabric Length")
plt.show()
```

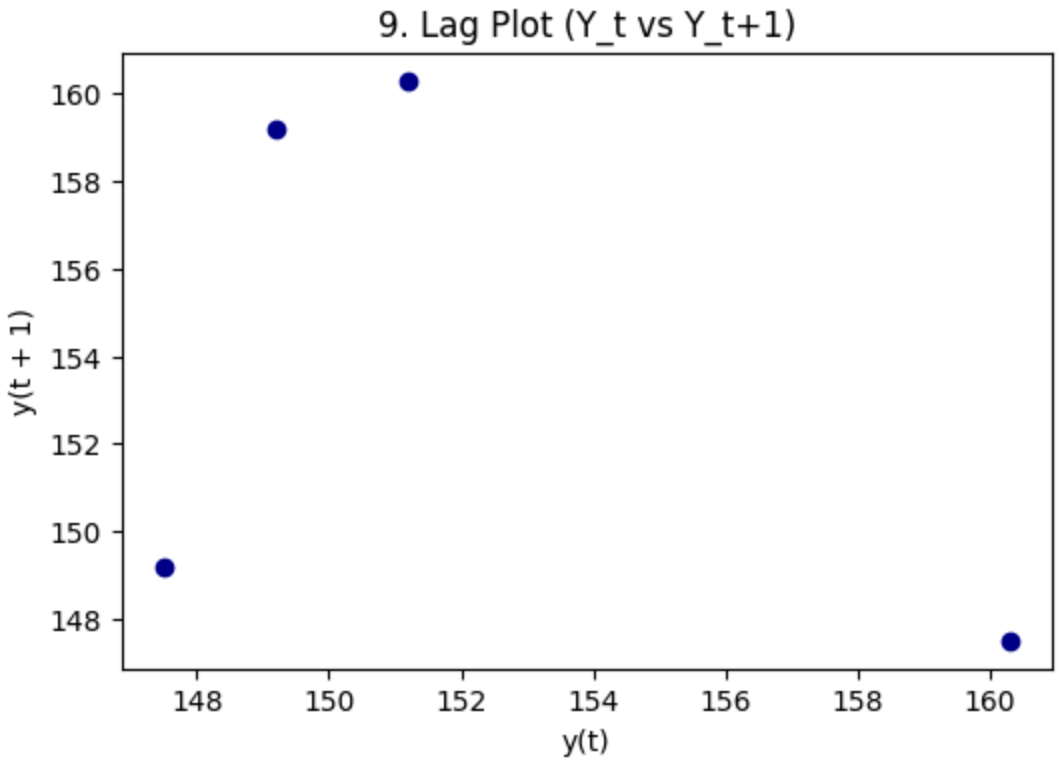


In [ ]:

8. Lag Plot (Checking for Randomness)

In [140…

```
# Sample Data
df = pd.DataFrame({"Fabric_length": [151.2, 160.3, 147.5, 149.2, 159.2]})

# Plot
plt.figure(figsize=(6, 4))
pd.plotting.lag_plot(df["Fabric_length"], c="darkblue")
plt.title("9. Lag Plot (Y_t vs Y_t+1)")
plt.show()
```

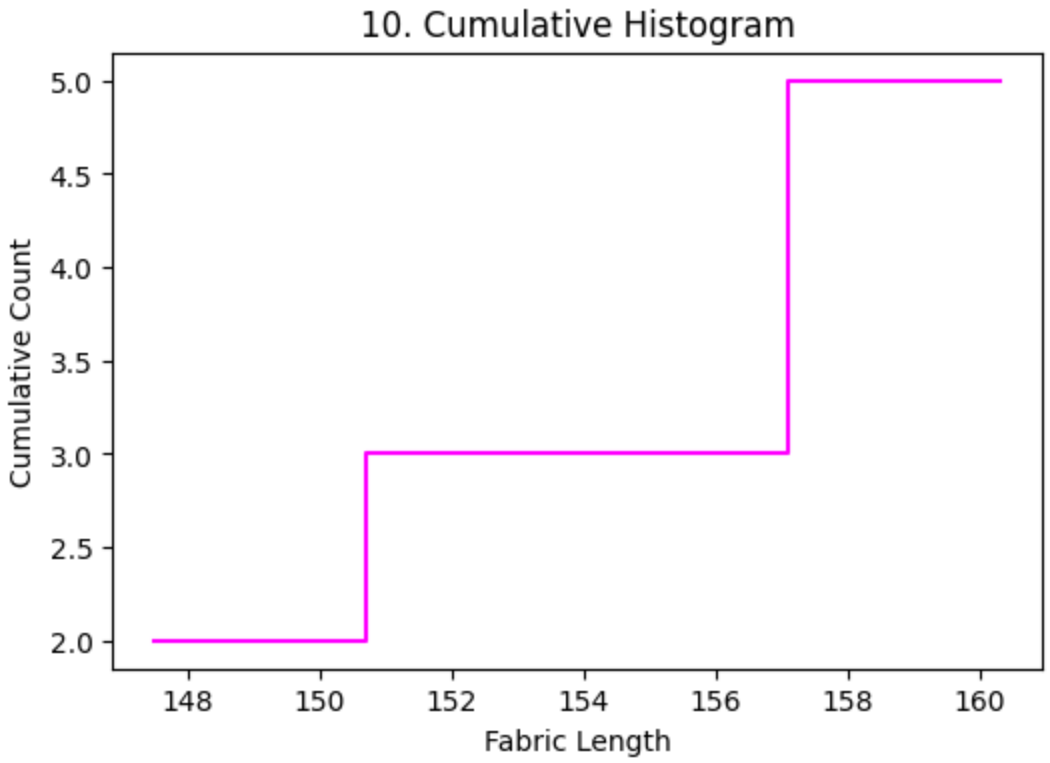


In [ ]:

9. Cumulative Histogram

In [141…

```
# Sample Data
df = pd.DataFrame({"Fabric_length": [151.2, 160.3, 147.5, 149.2, 159.2]})

# Plot
plt.figure(figsize=(6, 4))
sns.histplot(
    df["Fabric_length"], cumulative=True, element="step", fill=False, color="magent
)
plt.title("10. Cumulative Histogram")
plt.xlabel("Fabric Length")
plt.ylabel("Cumulative Count")
plt.show()
```

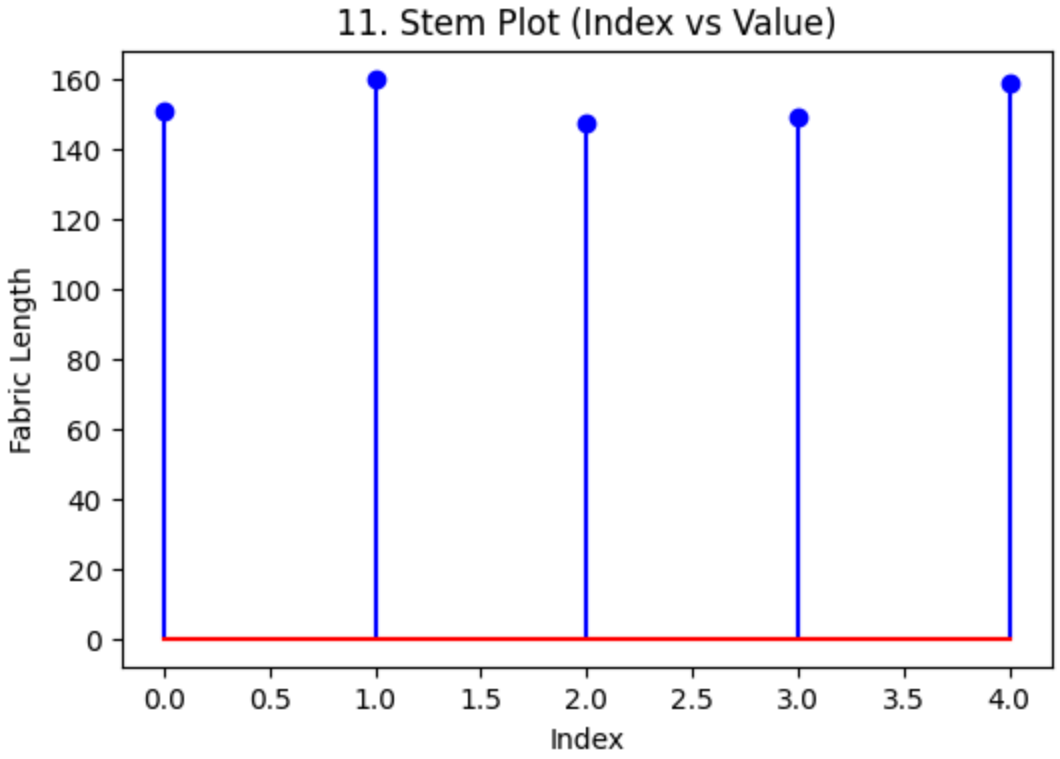


In [ ]:

10.Stem-and-Leaf Plot

In [142…

```
# Sample Data
df = pd.DataFrame({"Fabric_length": [151.2, 160.3, 147.5, 149.2, 159.2]})

# Plot
plt.figure(figsize=(6, 4))
plt.stem(df.index, df["Fabric_length"], linefmt="b-", markerfmt="bo", basefmt="r-")
plt.title("11. Stem Plot (Index vs Value)")
plt.xlabel("Index")
plt.ylabel("Fabric Length")
plt.show()
```


11. Stem Plot (Index vs Value)
Fabric Length
Index
160
140
120
100
80
60
40
20
0
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0

In [ ]:

In [ ]: